ỦY BAN NHÂN DÂN
THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN



TÀI LIỆU HỌC TẬP CẤP KHOA

TÀI LIỆU HỌC TẬP:
HƯỚNG DẪN TỰ HỌC VÀ NGHIÊN CỨU
HỌC PHẦN CHUYÊN ĐỀ J2EE (841468)

Mã số: CK2025 - 03

Chủ biên tài liệu: ThS. Nguyễn Thanh Phước

Tp Hồ Chí Minh – 07/2026

ỦY BAN NHÂN DÂN
THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN

TÀI LIỆU HỌC TẬP CẤP KHOA

TÀI LIỆU HỌC TẬP:
HƯỚNG DẪN TỰ HỌC VÀ NGHIÊN CỨU
HỌC PHẦN CHUYÊN ĐỀ J2EE (841468)

Mã số: CK2025 - 03

Xác nhận của
Chủ tịch hội đồng thẩm định
(ký, họ tên)

Chủ biên
(ký, họ tên)

Tp Hồ Chí Minh - Tháng 7/2026

Mục lục

MỤC LỤC 3

DANH MỤC HÌNH 6

LỜI NÓI ĐẦU 7

CHƯƠNG 1: KIẾN TRÚC SPRING BOOT 9

A. MỤC TIÊU 9

B. KẾT CẤU CHƯƠNG 9

C. NỘI DUNG CHƯƠNG 9

1.1. PHẦN 1: SPRING FRAMEWORK 9

1.1.1. Ôn tập lý thuyết với các PROMPT 9

1.1.2. Bài làm mẫu 10

1.1.3. Bài tập làm thêm 17

1.2. PHẦN 2: SPRING BOOT 18

1.2.1. Ôn tập lý thuyết với các PROMPT 18

1.2.2. Bài làm mẫu 19

1.2.2. Bài tập làm thêm 25

1.3. PHẦN 3: SPRING PROJECT 26

1.3.1. Ôn tập lý thuyết với các PROMPT 26

1.3.2. Bài làm mẫu 27

1.3.3. Bài tập làm thêm 32

D. TÓM TẮT CHƯƠNG 1 32

CHƯƠNG 2: RESTFUL API 34

A. MỤC TIÊU CHƯƠNG 34

B. KẾT CẤU CHƯƠNG 34

C. NỘI DUNG CHƯƠNG 34

2.1. PHẦN 1: RESTFUL API 34

2.1.1. Ôn tập lý thuyết với các PROMPT 34

2.1.2. Bài làm mẫu 35

2.1.3. Bài tập làm thêm 43

2.2. PHẦN 2: SWAGGER & OPEN API 43

2.2.1. Ôn tập lý thuyết với các PROMPT 43

2.2.2. Bài làm mẫu 44

2.2.3. Bài tập làm thêm 47

2.3. PHẦN 3: HATEOAS (HYPERMEDIA AS THE ENGINE OF APPLICATION STATE) 48

2.3.1. Ôn tập lý thuyết với các PROMPT 48

2.3.2. Bài làm mẫu 48
2.3.3. Bài tập làm thêm 50
D. TÓM TẮT CHƯƠNG 51
CHƯƠNG 3 VALIDATE DỮ LIỆU - ENUM VÀ RESPONSE 52
A. MỤC TIÊU CHƯƠNG 52
B. KẾT CẤU CHƯƠNG 52
C. NỘI DUNG CHƯƠNG 52
3.1. PHẦN 1: VALIDATE DỮ LIỆU 52
3.1.1. Ôn tập lý thuyết với các PROMPT 52
3.1.2. Bài tập mẫu 53
3.1.3. Bài tập làm thêm 55
3.2. PHẦN 2: RESPONSE 56
3.2.1. Ôn tập lý thuyết với các PROMPT 56
3.2.2. Bài làm mẫu 56
3.2.3. Bài tập làm thêm 60
D. TÓM TẮT CHƯƠNG 60
CHƯƠNG 4: XỬ LÝ NGOẠI LỆ 61
A. MỤC TIÊU CHƯƠNG 61
B. KẾT CẤU CHƯƠNG 61
C. NỘI DUNG CHƯƠNG 61
4.1. PHẦN 1: XỬ LÝ NGOẠI LỆ VỚI @CONTROLLERADVICED VÀ @EXCEPTIONHANDLER 61
4.1.1. Ôn tập lý thuyết với các PROMPT 61
4.1.2. Bài làm mẫu 62
4.1.3. Bài tập làm thêm 67
4.2. PHẦN 2: XỬ LÝ LOGGING 67
4.2.1. Ôn tập lý thuyết với các PROMPT 67
4.2.2. Bài làm mẫu 68
4.2.3. Bài tập làm thêm 69
D. TÓM TẮT CHƯƠNG 70
CHƯƠNG 5: LOMBOK VÀ MAPSTRUCT 71
A. MỤC TIÊU CHƯƠNG 71
B. KẾT CẤU CHƯƠNG 71
C. NỘI DUNG CHƯƠNG 71
5.1. PHẦN 1: SỬ DỤNG LOMBOK 71
5.1.1. Ôn tập lý thuyết với các PROMPT 71
5.1.2. Bài làm mẫu 72

5.1.3. Bài tập làm thêm 76
5.2. PHẦN 2: XỬ LÝ MAPSTRUCT 76
5.2.1. Ôn tập lý thuyết với các PROMPT 76
5.2.2. Bài làm mẫu 77
5.2.3. Bài tập làm thêm 85
D. TÓM TẮT CHƯƠNG 85
CHƯƠNG 6: JPA (JAVA PERSISTENCE API) VÀ HIBERNATE 86
A. MỤC TIÊU CHƯƠNG 86
B. KẾT CẤU CHƯƠNG 86
C. NỘI DUNG CHƯƠNG 86
6.1. PHẦN 1: JPA (JAVA PERSISTENCE API) 86
6.1.1. Ôn tập lý thuyết với các PROMPT 87
6.1.2. Bài làm mẫu 87
6.1.3. Bài tập làm thêm 89
6.2. PHẦN 2: HIBERNATE & SPRING DATA JPA 90
6.2.1. Ôn tập lý thuyết với các PROMPT 90
6.2.2. Bài làm mẫu 90
6.2.3. Bài tập làm thêm 106
D. TÓM TẮT CHƯƠNG 106
TÀI LIỆU THAM KHẢO 107

Danh mục hình

Hình Tổng quan về Spring Framework 6.x 9
Hình Spring Initializr cho phép tạo dự án Spring Boot trên web 10
Hình Dự án Spring boot khi được mở bằng ứng dụng IntelliJ Community 11
Hình Tổng quan về Spring Boot 3.x 18
Hình Dự án Spring Boot chạy trên IntelliJ Community 20
Hình Kiểm tra kết quả thực hiện các endpoint trên trình duyệt 20
Hình Trang chủ của website 25
Hình Liệt kê danh sách Sách 25
Hình Thông tin chi tiết quyển sách có ISBN là 9780451524935 25
Hình Liệt kê danh sách các thành viên của công ty 26
Hình Cấu trúc dự án Spring Project với Spring Boot 26
Hình Trang chủ dự án 31
Hình Liệt kê danh sách các khóa học 32
Hình Hiển thị danh sách Restful API trên thực thể TODO 34
Hình Liệt kê danh sách quyển sách khi sử dụng POSTMAN để gọi endpoint 39
Hình Thông tin của 1 quyển sách 39
Hình Thêm một quyển sách bằng ứng dụng POSTMAN 40
Hình Các tính năng quan trọng của SWAGGER 43
Hình Liệt kê danh sách các RESTful API có trong ứng dụng sử dụng Swagger-UI 44
Hình Liệt kê danh sách các Endpoint có trong ứng dụng bằng Swagger UI 46
Hình Hiển thị thông tin mô tả một Endpoint 47
Hình Các tính năng chính HATEOAS 48
Hình Bốn mức độ trưởng thành của RESTful API 51
Hình Các tính năng kiểm tra dữ liệu có trong Spring Boot 52
Hình Hỗ trợ các Response có trong Spring Boot 56
Hình Tổng quan về xử lý ngoại lệ trong Spring 61
Hình Tổng quan xử lý Logging mà Spring hỗ trợ 67
Hình Tổng quan các tính năng của Lombok 71
Hình Trang chủ của ứng dụng 75
Hình Trang đăng nhập của ứng dụng 75
Hình Trang holidays của website 76
Hình Tổng quan các tính năng được hỗ trợ bởi Spring Data JPA 86
Hình So sánh giữa Hibernate và cài đặt JPA mặc định 90

Lời nói đầu

MỤC TIÊU HỌC LIỆU

Tài liệu hướng dẫn tự học và nghiên cứu học phần Chuyên đề Java Enterprise Edition (Java EE) được biên soạn nhằm chuẩn hóa hệ thống học liệu, tạo nền tảng vững chắc giúp sinh viên chủ động tiếp cận kiến thức chuyên sâu về phát triển ứng dụng doanh nghiệp. Thông qua nội dung được hệ thống hóa bài bản, tài liệu hỗ trợ người học nắm vững các khái niệm cốt lõi từ kiến trúc đa tầng đến các công nghệ quan trọng như cách tổ chức dự án Spring, xử lý ngoài lệ, sử dụng Java Persistence API để làm việc với các hệ quản trị cơ sở dữ liệu, … Bên cạnh việc củng cố lý thuyết thông qua các câu hỏi Gemini, tài liệu còn chú trọng vào việc rèn luyện kỹ năng thực hành và giải quyết các bài toán thực tế theo đúng yêu cầu chuẩn đầu ra trong đề cương học phần Chuyên đề J2EE. Đây được xem là người bạn đồng hành thiết yếu, giúp sinh viên không chỉ vượt qua kỳ thi mà còn hình thành tư duy lập trình chuyên nghiệp trong môi trường Java thuần thục. Cuối cùng, việc bám sát lộ trình tự học này sẽ giúp sinh viên tối ưu hóa thời gian nghiên cứu, tạo đà thuận lợi cho việc tiếp cận các framework hiện đại trong tương lai.

CẤU TRÚC HỌC LIỆU

Tài liệu được thiết kế gồm 6 chương dẫn dắt sinh viên từ nền tảng kiến trúc đến kỹ năng triển khai ứng dụng thực tế. Mỗi chương được chia thành hai hoặc ba phần nhỏ với cấu trúc gồm 3 nội dung sau:

+ Hệ thống hóa kiến thức: Sử dụng AI (Gemini) để truy vấn, giải thích và ôn tập lý thuyết một cách trực quan.

+ Bài tập mẫu: Các ví dụ minh họa chi tiết kèm giải thích mã nguồn.

+ Bài tập làm thêm: Các bài tập tự luyện để củng cố, nâng cao tư duy lập trình và gợi ý cách áp dụng kiến thức này vào dự án cuối kỳ.

Chương 1. Kiến trúc Spring Boot: Giúp sinh viên hiểu rõ cơ chế Inversion of Control (IoC), Dependency Injection (DI) và cách Spring Boot tự động cấu hình (Auto-configuration) để khởi tạo một dự án Enterprise nhanh chóng.

Chương 2. RESTful API: Trang bị tư duy thiết kế hệ thống theo kiến trúc hướng dịch vụ, nắm vững các phương thức HTTP (GET, POST, PUT, DELETE) và cách xây dựng các điểm cuối (Endpoints) chuẩn hóa.

Chương 3. Validate Data, Enum và Response: Hướng dẫn cách kiểm soát dữ liệu đầu vào (Validation), sử dụng kiểu dữ liệu liệt kê (Enum) để code sạch hơn và chuẩn hóa cấu trúc phản hồi (Response) đồng nhất cho mọi API.

Chương 4. Xử lý ngoại lệ (Exception Handling): Giúp sinh viên xây dựng cơ chế quản lý lỗi tập trung (@ControllerAdvice), đảm bảo hệ thống không bị "sập" khi gặp sự cố và trả về thông báo lỗi thân thiện với người dùng.

Chương 5. Lombok và MapStruct: Giới thiệu các công cụ tối ưu hóa năng suất lập trình; sử dụng Lombok

để loại bỏ "code rác" (Boilerplate code) và MapStruct để chuyển đổi dữ liệu mượt mà giữa các đối tượng DTO và Entity.

Chương 6. JPA và Hibernate: Nắm vững kỹ thuật ORM (Object-Relational Mapping), cách quản trị cơ sở dữ liệu thông qua mã Java và tối ưu hóa các câu truy vấn phức tạp trong môi trường doanh nghiệp.

YÊU CẦU ĐỐI VỚI NGƯỜI HỌC

Người học cần trang bị một nền tảng lập trình Java cơ bản và tham gia học học phần "Chuyên đề J2EE (841468) tại Trường Đại học Sài Gòn để có thể tiếp thu các khái niệm chuyên sâu một cách hiệu quả. Bên cạnh kiến thức chuyên môn, việc thành thạo công cụ lập trình IntelliJ IDEA Community là một điểm cộng để thực hiện các bài tập và dự án thực tế trong tài liệu. Đặc biệt, sinh viên cần biết cách khai thác sức mạnh của trợ lý AI Gemini như một công cụ hỗ trợ tư duy, giúp giải đáp thắc mắc và mở rộng lý thuyết một cách chủ động. Sự kết hợp giữa kỹ năng lập trình vững chắc, công cụ làm việc chuyên nghiệp và khả năng tương tác với AI sẽ đảm bảo người học đạt được kết quả tốt.

Trân trọng cảm ơn!

CHƯƠNG 1: KIẾN TRÚC SPRING BOOT

A. MỤC TIÊU

Hỗ trợ việc luyện tập và nghiên cứu nội dung chương 2 có trong đề cương học phần Chuyên đề J2EE (841468) bao gồm Spring Framework, Spring Boot và cách triển khai một dự án với kiến trúc sử dụng Spring Boot làm Backend và sử dụng Spring Boot + Spring MVC + Thymeleaf làm Frontend.

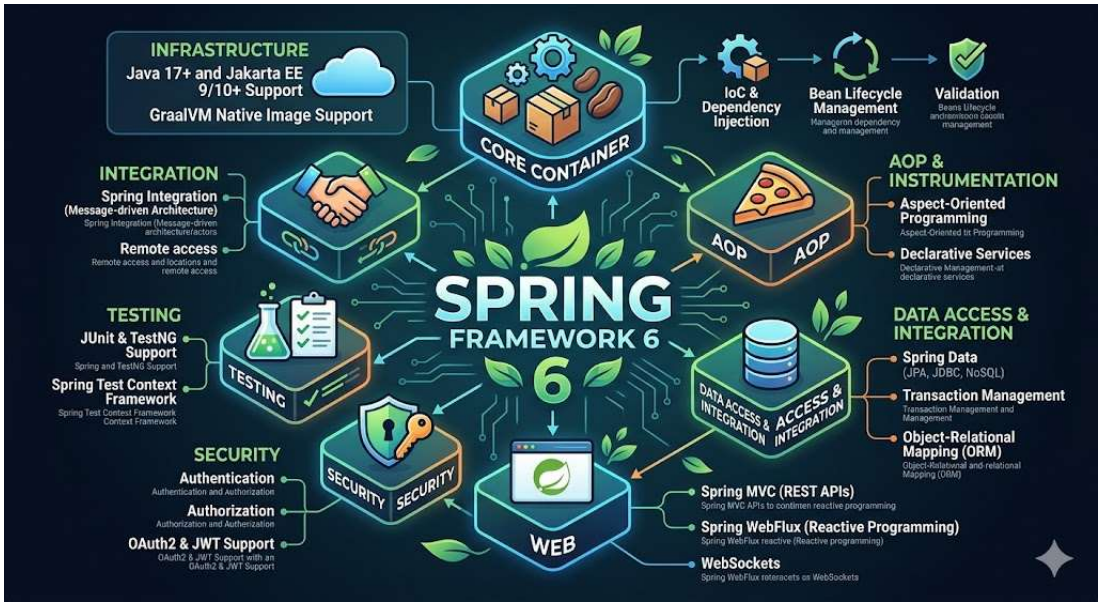
B. KẾT CẤU CHƯƠNG

Chương này trình bày ba phần bao gồm

- Phần 1: Spring Framework 6.x
- Phần 2: Spring Boot 3.x
- Phần 3: Spring Project 3.x

C. NỘI DUNG CHƯƠNG

1.1. PHẦN 1: SPRING FRAMEWORK



Hình Tổng quan về Spring Framework 6.x

1.1.1. Ôn tập lý thuyết với các PROMPT

+ Java EE là gì? Do công ty nào phát triển và phiên bản mới nhất?

+ Spring Framework là gì? Do công ty nào phát triển và phiên bản mới nhất?

+ Giải thích 4 cột trụ cốt lõi của Spring Framework bao gồm: IoC/DI, AOP, Modularity và Transaction Management. Với mỗi tính năng, hãy đưa ra một ví dụ code minh họa đơn giản bằng Annotation để một sinh viên mới bắt đầu có thể hiểu được

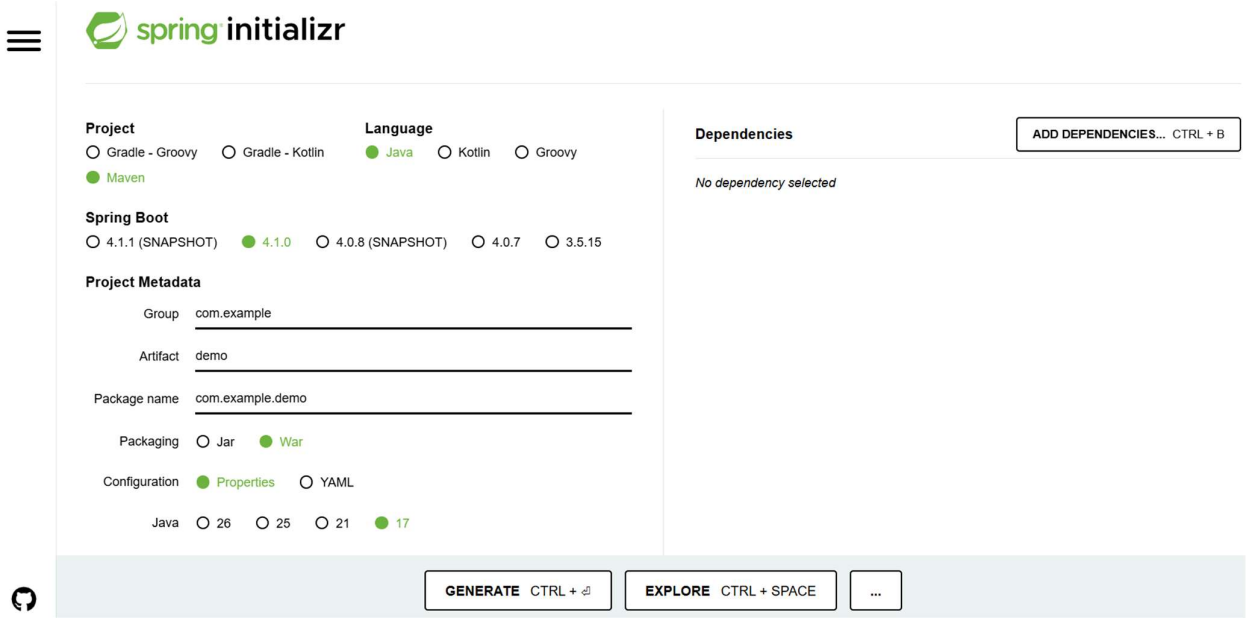
+ So sánh sự khác biệt giữa Spring Framework hiện đại (phiên bản 7) và Java EE 6 (J2EE). Hãy tập trung vào sự khác biệt về triết lý thiết kế, cách cấu hình (XML vs Annotations), và tại sao Spring lại trở thành lựa chọn phổ biến hơn trong phát triển ứng dụng Microservices ngày nay?

+ Hãy giải thích khái niệm Dependency Injection trong Spring Boot. Sử dụng một ví dụ thực tế trong đời sống để minh họa thay vì dùng thuật ngữ kỹ thuật.

1.1.2. Bài làm mẫu

Bài 1. Tạo dự án Spring sử dụng Spring Initializr

Bước 1: Đến trang web https://start.spring.io/



Hình Spring Initializr cho phép tạo dự án Spring Boot trên web

- Thực hiện các chọn lựa như sau:

  Project: Maven

  Language: Java

  Spring Boot: 3.5.15

  Project Metadata: nhập tên dự án

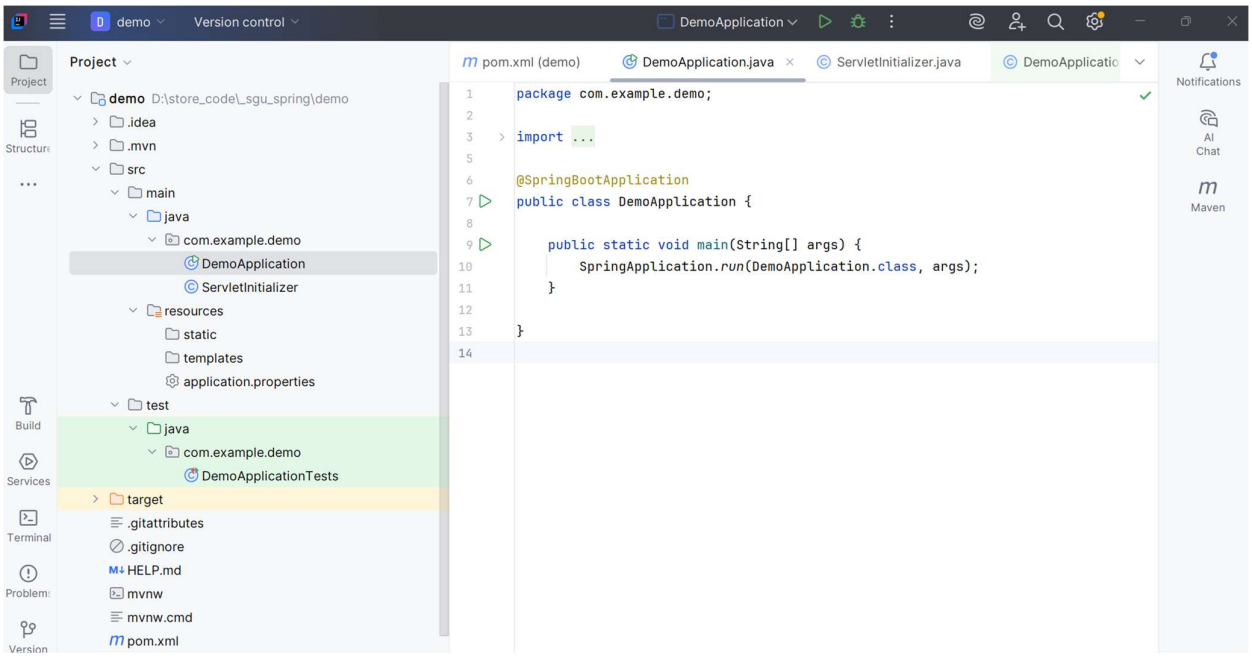
  Packaging: War

  Configuration: Properties

  Java: 17

- Click GENERATE để tạo dự án và tải dự án về máy tính

Bước 2**:** Mở dự án bằng phần mềm **IntelliJ IDEA 2025.2.3 (Community Edition)** nên chọn tải phiên bản mới nhất.



Hình Dự án Spring boot khi được mở bằng ứng dụng IntelliJ Community

Bước 3**:** Cập nhật lớp DemoApplication với nội dung như sau:

```
@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
       SpringApplication.run(DemoApplication.class, args);

        System.out.println("Welcome to Spring Boot!");

        try (var ctx = SpringApplication.run(DemoApplication.class, args)) {

            System.out.println("# Beans: " + ctx.getBeanDefinitionCount());

            var names = ctx.getBeanDefinitionNames();
            Arrays.sort(names);
            Arrays.asList(names).forEach(System.out::println);
        }
    }
}
```

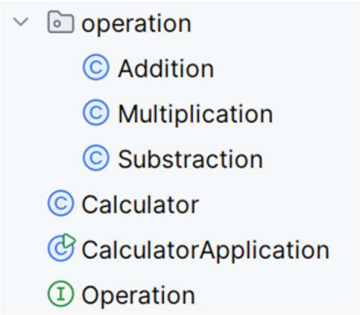
Bước 4**:** Chạy ứng dụng và quan sát kết quả

Bước 5**:** Tìm hiểu ý nghĩa đoạn code trên

Bài 2: Cách tạo bean và tìm hiểu khái niệm IoC và DI của Spring

Bước 1: Tạo dự án Spring từ Spring Initializr

Bước 2: Lần lượt tạo các lớp và interface như hình sau



Bước 3: Viết mã cho Operation interface và các lớp cài đặt của nó là Addition, Multiplication và Substraction như sau

```
public interface Operation {
    int apply(int lhs, int rhs);
    boolean handles(char op);
}


@Component
class Addition implements Operation {
    @Override
    public int apply(int lhs, int rhs) {
        return lhs + rhs;
    }
    @Override
    public boolean handles(char op) {
        return '+' == op;
    }
}


@Component
class Multiplication implements Operation {
    @Override
    public int apply(int lhs, int rhs) {
        return lhs * rhs;
    }
    @Override
    public boolean handles(char op) {
        return '*' == op;
    }
}


@Component
class Substraction implements Operation {
    @Override
    public int apply(int lhs, int rhs) {
       return lhs - rhs;
    }
    @Override
    public boolean handles(char op) {
       return '-' == op;
    }
}
```

Bước 4: Viết mã cho lớp Calculator (sinh viên tự tìm hiểu thêm cách sử dụng Stream API)

```
@Component
public class Calculator {

    private final Collection<Operation> operations;

    public Calculator(Collection<Operation> operations) {
       this.operations = operations;

       System.out.println("OPERATION" + operations);
    }
```

```
    public void calculate(int lhs, int rhs, char op) {
       operations.stream()
         .filter((operation) -> operation.handles(op))
         .map((operation) -> operation.apply(lhs, rhs))
         .peek( (result)
            -> System.out.printf("%d %s %d = %s%n", lhs, op, rhs, result))
         .findFirst()
         .orElseThrow(() ->
            new IllegalArgumentException("Unknown operation " + op));
    }
}
```

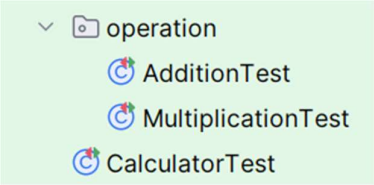
Bước 5: Viết mã cho lớp CalculatorApplication cho ứng dụng Spring này. Lớp này cũng chính là lớp thực thi cho ứng dụng sử dụng công nghệ Spring

```
@SpringBootApplication
public class CalculatorApplication {

    public static void main(String[] args) {
       SpringApplication.run(CalculatorApplication.class, args);
    }

    @Bean
    public ApplicationRunner calculationRunner(Calculator calculator) {
       return args -> {
          calculator.calculate(137, 21, '+');
          calculator.calculate(137, 21, '*');
          calculator.calculate(137, 21, '-');
       };
    }
}
```

Bước 6: Viết mã kiểm tra đơn vị (unit test) cho dự án Spring. Vào phần test và tạo ra các lớp như hình bên dưới:



Viết mã cho các lớp trên như sau (sinh viên tự tìm hiểu thêm cách sử dụng JUnit 5 và Mockito để thực hiện việc kiểm thử đơn vị một cách tự động; Java Lambda Expressions để biết cách áp dụng kỹ thuật lập trình hàm (functional programming) vào dự án

```
class AdditionTest {
    private final Addition addition = new Addition();
    @Test
    void shouldMatchPlusSign() {
        assertThat(addition.handles('+')).isTrue();
        assertThat(addition.handles('/')).isFalse();
    }
    @Test
    void shouldCorrectlyApplyFormula() {
        assertThat(addition.apply(2, 2)).isEqualTo(4);
        assertThat(addition.apply(12, 88)).isEqualTo(100);
    }
}


class MultiplicationTest {
    private final Multiplication addition = new Multiplication();
    @Test
    void shouldMatchSign() {
       assertThat(addition.handles('*')).isTrue();
       assertThat(addition.handles('/')).isFalse();
    }
```

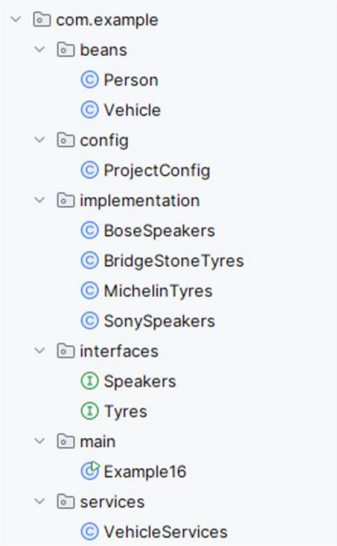
```
    @Test
    void shouldCorrectlyApplyFormula() {
       assertThat(addition.apply(2, 2)).isEqualTo(4);
       assertThat(addition.apply(12, 10)).isEqualTo(120);
    }
}


class CalculatorTest {
    private Calculator calculator;
    private Operation mockOperation;
    @BeforeEach
    public void setup() {
        mockOperation = Mockito.mock(Operation.class);
        calculator = new Calculator(Collections.singletonList(mockOperation));
    }
    @Test
    void throwExceptionWhenNoSuitableOperationFound() {
        when(mockOperation.handles(anyChar())).thenReturn(false);
        Assertions.assertThrows(IllegalArgumentException.class,
             () ->calculator.calculate(2, 2, '*'));
    }
    @Test
    void shouldCallApplyMethodWhenSuitableOperationFound() {
        when(mockOperation.handles('*')).thenReturn(true);
        when(mockOperation.apply(2, 2)).thenReturn(4);
        calculator.calculate(2, 2, '*');
        verify(mockOperation, times(1)).apply(2,2);
    }
}
```

Bước 7: Chạy ứng dụng tại lớp CalculatorApplication và chạy tất cả test case tại lớp CalculatorTest.

Bài 3: Tổ chức dự án Spring và tìm hiểu về bean scope

Bước 1: Tạo dự án và tạo các lớp và interface như hình bên dưới



Bước 2: Viết mã cho interface Speakers và các lớp Speakers cụ thể là BoseSpeakers và SonySpeakers

```
public interface Speakers {
    public String makeSound();
}


@Component
public class BoseSpeakers implements Speakers {
```

```
    public String makeSound(){
        return "Playing music with Bose speakers";
    }
}


@Component
@Primary
public class SonySpeakers implements Speakers {
    public String makeSound(){
        return "Playing music with Sony speakers";
    }
}
```

Bước 3: Viết mã cho interface Tyres và các lớp Tyres cụ thể là BridgeStoneTyres và MichelinTyres

```
public interface Tyres {
    public String rotate();
}


@Component
public class BridgeStoneTyres implements Tyres {
    public String rotate(){
        return "Vehicle moving with BridgeStone tyres";
    }
}


@Component
@Primary
public class MichelinTyres implements Tyres {
    public String rotate(){
        return "Vehicle moving with Michelin tyres";
    }
}
```

Bước 4: Viết mã cho 2 bean là Person và Vehicle

```
@Component(value="personBean")
public class Person {
    private String name="Lucy";
    private final Vehicle vehicle;
    @Autowired
    public Person(Vehicle vehicle){
        this.vehicle = vehicle;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public Vehicle getVehicle() {
        return vehicle;
    }
}


@Component("vehicleBean")
public class Vehicle {
    private String name="Honda";
    private final VehicleServices vehicleServices;
    @Autowired
    public Vehicle(VehicleServices vehicleServices){
        this.vehicleServices = vehicleServices;
    }
    public String getName() {
        return name;
```

```
    }
    public void setName(String name) {
        this.name = name;
    }
    public VehicleServices getVehicleServices() {
        return vehicleServices;
    }
    public void printHello(){
        System.out.println(
            "Printing Hello from Component Vehicle Bean");
    }
    @Override
    public String toString(){
        return "Vehicle name is - " + name;
    }
}
```

Bước 5: Viết mã cho các lớp phục vụ (service). Lớp cung cấp các tính năng cho tầng giao diện hay tầng ứng dụng sử dụng.

```
@Component
@Scope(BeanDefinition.SCOPE_PROTOTYPE)
public class VehicleServices {
    @Autowired
    private Speakers speakers;
    private Tyres tyres;
    public VehicleServices(){
        System.out.println("VehicleServices object is created");
    }
    public void playMusic(){
        String music = speakers.makeSound();
        System.out.println(music);
    }
    public void moveVehicle(){
        String status = tyres.rotate();
        System.out.println(status);
    }
    public Speakers getSpeakers() {
        return speakers;
    }
    public void setSpeakers(Speakers speakers) {
        this.speakers = speakers;
    }
    public Tyres getTyres() {
        return tyres;
    }
    @Autowired
    public void setTyres(Tyres tyres) {
        this.tyres = tyres;
    }
}
```

Bước 6: Viết mã để cấu hình cho dự án Spring trong lớp ProjectConfig

```
@Configuration
@ComponentScan(
        basePackages = {"com.example.implementation",
                        "com.example.services"})
@ComponentScan(
        basePackageClasses =
                    {com.example.beans.Vehicle.class,
                    com.example.beans.Person.class})
public class ProjectConfig {

}
```

Bước 7: Viết mã cho ứng dụng hay viết mã tầng ứng dụng cho lớp Example16

```
public class Example16 {
    public static void main(String[] args) {
        var context = new
           AnnotationConfigApplicationContext(ProjectConfig.class);

        VehicleServices vehicleServices1 =
                context.getBean(VehicleServices.class);
        VehicleServices vehicleServices2 =
                context.getBean("vehicleServices",VehicleServices.class);
        System.out.println("Hashcode of the object vehicleServices1 : "
                +vehicleServices1.hashCode());
        System.out.println("Hashcode of the object vehicleServices2 : "
                +vehicleServices2.hashCode());

        if(vehicleServices1==vehicleServices2){
          System.out.println("VehicleServices bean
                                  is a singleton scoped bean");
        }else{
          System.out.println("VehicleServices bean
                                  is a prototype scoped bean");
        }
    }
}
```

Bước 8: Thực thi ứng dụng bằng cách chạy lớp Example16.

Bài 4: Cách dựng dự án từ dự án mẫu hay dự án có mã nguồn trên internet

Bước 1. Tạo dự án Spring Boot từ Spring Initializr

Bước 2. Nếu dự án mẫu có cơ sở dữ liệu thì chuẩn bị hệ quản trị cơ sở dữ liệu; tạo cơ sở dữ liệu; tạo các bảng dữ liệu có trong dự án; tạo dữ liệu cho các bạn đó; cấu hình driver tướng ứng trong tập tin pom.xml của dự án

Bước 3. Sao chép các lớp có trong dự án mẫu vào dự án nếu được thì tạo lần lượt các lớp tương tự như dự án mẫu để hiểu rõ hơn về dự án mẫu

Bước 4. Đồng bộ tập tin pom.xml từ dự án mẫu vào dự án của mình chú ý chỉ thêm các phụ thuộc hay thư viện mà trong dự án mình không có, cẩn thận thêm quá nhiều thư viện có version cũ

Bước 5. Kiểm tra một số lớp bị lỗi do thư viện có thay đổi tên hàm, tên đối số hoặc 1 cách nâng cấp khác.

Bước 6. Thực hiện kiểm tra các chức năng của chương trình

Bước 7. Nếu việc dựng quá phức tạp do không tương thích ở nhiều thư viện ta nên chọn 1 dự án mẫu khác để dựng

1.1.3. Bài tập làm thêm

Dự án 1.1. Tạo dự án để tìm hiểu về Spring Container với cấu hình bean bằng Java

a) Tạo 1 maven project (hoặc tử Spring Initializr)

b) Tạo 1 lớp GreetingService có 1 biến thành viên là message

c) Tạo 1 AppConf và định nghĩa Spring Bean bằng Java

d) Tạo 1 SpringApplication khởi tạo Spring Context và truy vấn Bean định nghĩa ở bước b) và bước c)

Gợi ý: tham khảo tại https://www.javaguides.net/2018/10/spring-ioc-container-overview.html

Dự án 1.2. Tạo dự án có trong bài viết *https://www.javaguides.net/2018/10/spring-scope-annotation-with-prototype.html* để hiểu thêm về khái niệm Scope trong Spring

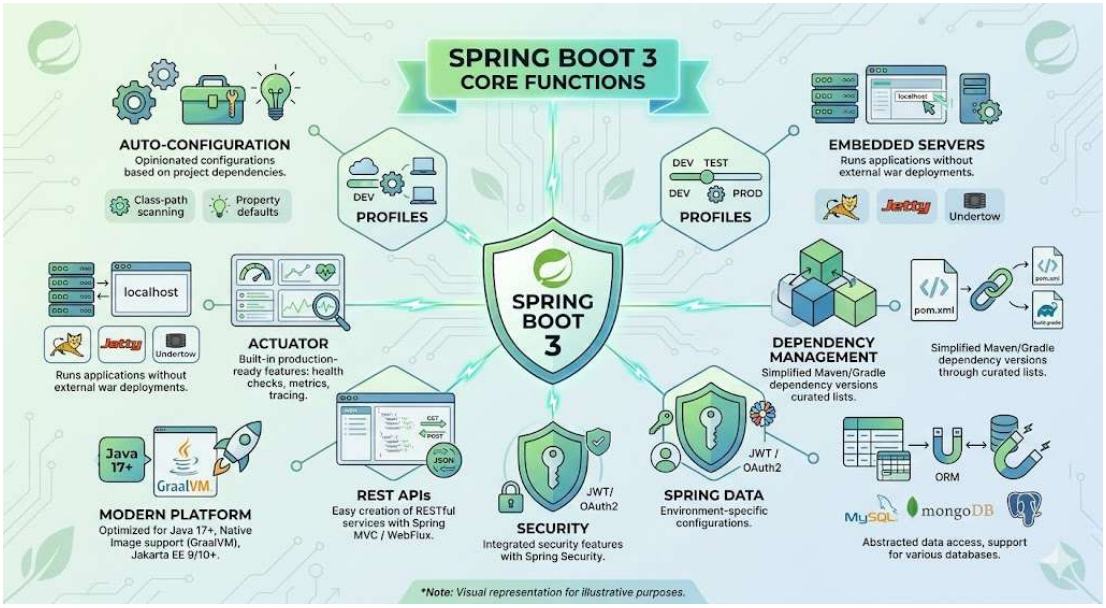
Dự án 1.3. Tạo dự án có trong bài viết *https://www.javaguides.net/2018/10/spring-scope-annotation-with-singleton-scope-example.html* để biết cách sử dụng @Scope + SCOPE_PROTOTYPE, SCOPE_SINGLETON

Dự án 1.4. Tạo dự án có trong bài viết *https://www.javaguides.net/2018/09/spring-propertysource-annotation-with-example.html* để biết cách đọc tập tin cấu hình từ Java tránh viết "Hard code" trong chương trình với @Value và @PropertySource

Dự án 1.5. Tạo dự án có trong bài viết *https://www.javaguides.net/2018/09/spring-autowired-annotation-with-example.html* để biết cách sử dụng chức năng DI thông qua @Autowired và @Qualifier

Dự án 1.6. Tạo dự án có trong bài viết https://www.geeksforgeeks.org/springboot/custom-bean-scope-in-spring để biết cách xây dựng một Scope mới

1.2. PHẦN 2: SPRING BOOT



Hình Tổng quan về Spring Boot 3.x

1.2.1. Ôn tập lý thuyết với các PROMPT

+ Trình bày sự khác biệt cốt lõi giữa việc triển khai một dự án bằng Spring Boot so với việc chỉ sử dụng Spring Framework truyền thống.

+ Giải thích cơ chế 'Auto-configuration' trong Spring Boot hoạt động như thế nào và cách nó giúp lập trình viên giảm bớt khối lượng công việc thiết lập ban đầu.

+ Embedded Web Server trong Spring Boot là gì? Việc tích hợp sẵn Tomcat hoặc Jetty mang lại lợi ích gì cho quá trình triển khai (deployment) so với cách đóng gói file WAR truyền thống.

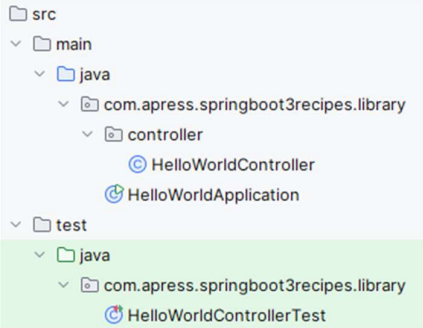
+ Mục đích của 'Spring Boot Starters' là gì? Hãy phân tích cách các Starter này giúp chuẩn hóa việc quản lý các dependencies trong một dự án Enterprise.

+ Hãy liệt kê các Spring Boot Starters thường sử dụng trong dự án Spring Boot. Với mỗi Spring Boot Starters liệt kê các tính năng của nó dưới dạng bảng có ví dụ minh họa.

1.2.2. Bài làm mẫu

Bài 1: Xây dựng ứng dụng RESTful API sử Spring Boot

Bước 1: Tạo dự án Spring Boot và tạo/cập nhật các lớp như hình bên dưới



Bước 2: Viết mã cho lớp HelloWorldController

```
@RestController
public class HelloWorldController {

    @GetMapping("/")
    public String hello() {
       var version = SpringBootVersion.getVersion();
       return String.format("Hello World, from Spring Boot %s!", version);
    }
}
```

Bước 3: Viết mã cho lớp HelloWorldApplication

```
@SpringBootApplication
public class HelloWorldApplication {

    public static void main(String[] args) {
       SpringApplication.run(HelloWorldApplication.class, args);
    }
}
```

Bước 4: Viết mã cho lớp kiểm tra HelloWorldControllerTest

```
@WebMvcTest(HelloWorldController.class)
class HelloWorldControllerTest {

    @Autowired
    private MockMvc mockMvc;

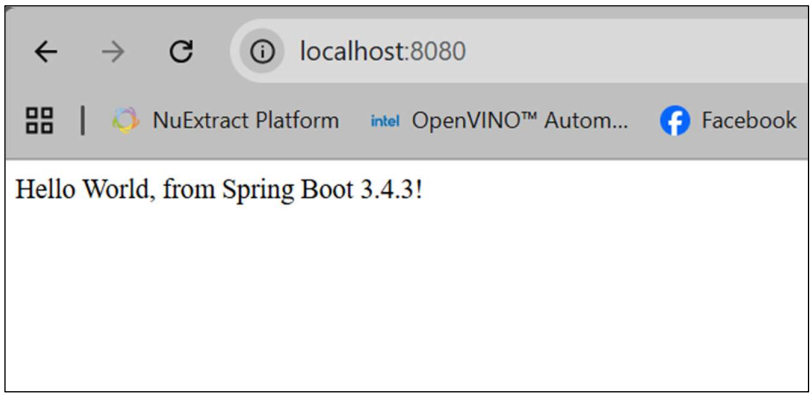
    @Test
    void testHelloWorldController() throws Exception {
       var expected = "Hello World, from Spring Boot ";
       mockMvc.perform(MockMvcRequestBuilders.get("/"))
          .andExpect(status().isOk())
          .andExpect(content().string(Matchers.containsString(expected)))
          .andExpect(content().contentTypeCompatibleWith(MediaType.TEXT_PLAIN));
    }
}
```

Bước 5: Chạy ứng dụng bằng cách vào lớp HelloWorldApplication và nhấn Run, Ứng dụng sẽ chạy như hình bên dưới



Hình Dự án Spring Boot chạy trên IntelliJ Community

Mở trình duyệt lên và đánh vào URL như hình để thấy RESTful API hoạt động

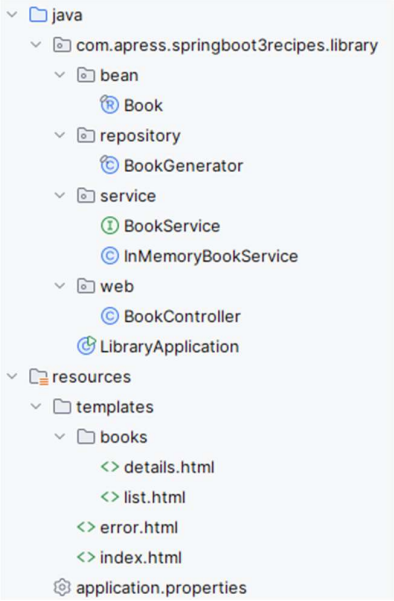


Hình Kiểm tra kết quả thực hiện các endpoint trên trình duyệt

Bước 6: Kiểm tra hoạt động mức đơn vị của ứng dụng bằng cách vào lớp HelloWorldControllerTest để chạy.

Bài 2: Xây dựng ứng dụng Website sử dụng Spring Boot và Thymeleaf

Bước 1: Tạo dự án với phụ thuộc là Thymeleaf và tổ chức mã nguồn như hình sau



Bước 2: Viết mã cho tầng dữ liệu - bean dữ liệu (Book) và repository (BookGenerator). Tìm hiểu cách sử dụng record trong Java.

```
public record Book(String isbn, String title, List<String> authors) {

    @Override
    public boolean equals(Object o) {
       if (o instanceof Book other) {
          return this.isbn != null && Objects.equals(this.isbn, other.isbn);
       }
       return false;
    }

    @Override
    public int hashCode() {
       return Objects.hash(this.isbn);
    }
}


public final class BookGenerator {

    public static List<Book> all() {
       try {
          var books = new ClassPathResource("books.csv").getInputStream();
          try (var lines = new BufferedReader(new
           InputStreamReader(books)).lines()) {
             return lines
                .skip(1)
                .map((line) -> line.split("\\|"))
                .map((row) -> new Book(row[0], row[1],
                       List.of(row[2].split(","))))
                .toList();
          }
       } catch (IOException ex) {
          throw new IllegalStateException("Error loading 'books.csv'!", ex);
       }
    }

    public static List<Book> random(int size) {
```

```
        var books = all();
        if (size >= books.size()) {
           return books;
        }
        return IntStream.range(0, size)
              .mapToObj( (x) -> randomBook(books)).toList();
    }

    public static Book random() {
        return randomBook(all());
    }

    private static Book randomBook(List<Book> books) {
        var idx = ThreadLocalRandom.current().nextInt(books.size());
        return books.get(idx);
    }
}
```

Bước 3: Viết mã cho tầng dịch vụ - interface BookService và lớp InMemoryBookService

```
public interface BookService {

    Iterable<Book> findAll();
    Book create(Book book);
    Optional<Book> find(String isbn);
}


@Service
class InMemoryBookService implements BookService {
    private final Map<String, Book> books = new ConcurrentHashMap<>();

    @Override
    public Iterable<Book> findAll() {
        return books.values();
    }
    @Override
    public Book create(Book book) {
        books.put(book.isbn(), book);
        return book;
    }
    @Override
    public Optional<Book> find(String isbn) {
        return Optional.ofNullable(books.get(isbn));
    }
}
```

Bước 4: Viết mã cho tầng giao diện

a. Phần mã viết bằng ngôn ngữ Java

```
@Controller
public class BookController {
    private final BookService bookService;

    public BookController(BookService bookService) {
        this.bookService = bookService;
    }

    //Trang chu + list.html
    @GetMapping("/books.html")
    public String all(Model model) {
        model.addAttribute("books", bookService.findAll());
        return "books/list";
    }
```

```
    //Trang xem thong tin chi tiet sach + details.html
    @GetMapping(value = "/books.html", params = "isbn")
    public String get(@RequestParam("isbn") String isbn, Model model) {

       bookService.find(isbn)
                  .ifPresent(book -> model.addAttribute("book", book));

       return "books/details";
    }


    @PostMapping("/books")
    public Book create(@ModelAttribute Book book) {
       return bookService.create(book);
    }
}
```

a. Phần mã viết bằng ngôn ngữ HTML + Thymeleaf

Trang chủ ứng dụng web index.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title th:text="#{main.title}">Spring Boot Recipes - Library</title>
</head>
<body>

<h3>Language</h3>
<div>
    <a href="?locale=nl" th:text="#{main.language.nl}">NL</a> |
    <a href="?locale=en" th:text="#{main.language.en}">EN</a>
</div>

<h1 th:text="#{index.title}">Library</h1>
<div>
    <a th:href="@{/books.html}" href="#" th:text="#{index.books.link}">List of
books</a>
</div>
</body>
</html>
```

Trang liệt kê danh sách sách list.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head lang="en">
    <meta charset="UTF-8">
    <title th:text="#{index.title}">Library - Available Books</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8"/>
</head>
<body>
    <h1 th:text="#{books.list.title}">Available Books</h1>
    <table>
        <thead>
            <tr>
                <th th:text="#{books.list.table.title}">Title</th>
                <th th:text="#{books.list.table.author}">Author</th>
                <th th:text="#{books.list.table.isbn}">ISBN</th>
            </tr>
        </thead>
        <tbody>
            <tr th:each="book : ${books}">
                <td th:text="${book.title}">Title</td>
                <td th:text="${book.authors}">Authors</td>
                <td><a th:href="@{/books.html(isbn=${book.isbn})}"
```

```
                        th:text="${book.isbn}" href="#">ISBN</a></td>
            </tr>
        </tbody>
    </table>
</body>
</html>
```

Trang xem thông tin chi tiết quyển sách

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head lang="en">
    <meta charset="UTF-8">
    <title>Library - Available Books</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8"/>
</head>
<body>
<div th:if="${book != null}">
    <div>
        <div th:text="${book.title}">Title</div>
        <div th:text="${book.authors}">authors</div>
        <div th:text="${book.isbn}">ISBN</div>
    </div>
</div>

<div th:if="${book} == null">
    <h1 th:text="'No book found with ISBN: ' + ${param.isbn}">Not Found</h1>
</div>

</body>
</html>
```

Trang thông báo lỗi của hệ thống

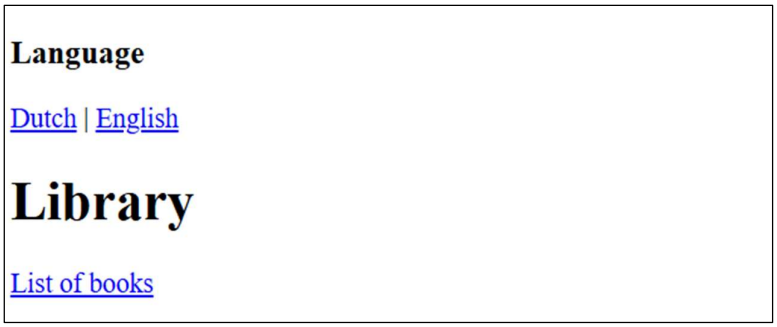
```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Spring Boot Recipes - Library</title>
</head>
<body>
<h1>Oops something went wrong, we don't know what but we are going to work on
it!</h1>

<div>
    <div>
        <span><strong>Status</strong></span>
        <span th:text="${status}"></span>
    </div>
    <div>
        <span><strong>Error</strong></span>
        <span th:text="${error}"></span>
    </div>
    <div>
        <span><strong>Message</strong></span>
        <span th:text="${message}"></span>
    </div>
    <div th:if="${exception != null}">
        <span><strong>Exception</strong></span>
        <span th:text="${exception}"></span>
    </div>
    <div th:if="${parameters != null}">
        <h3>Parameters</h3>
        <span th:each="parameter :${parameters}">
            <div th:text="${parameter.key} + ' : ' +
${#strings.arrayJoin(parameter.value, ',')}"></div>
        </span>
    </div>
```

```
    <div th:if="${trace != null}">
        <h3>Stacktrace</h3>
        <span th:text="${trace}"></span>
    </div>
</div>
</body>
</html>
```

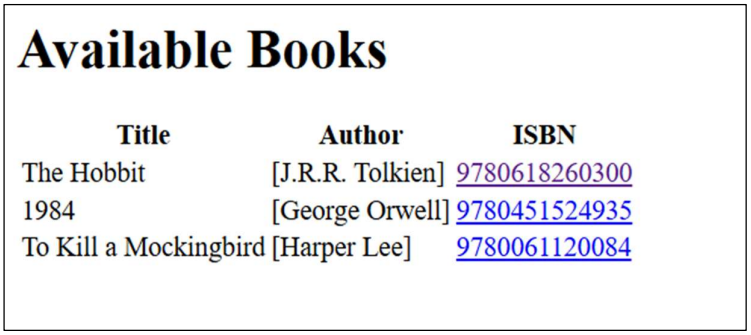
Bước 4: Chạy ứng dụng web và kiểm tra kết quả

- Mở trình duyệt và URL: http://localhost:8080/ để đến Trang chủ



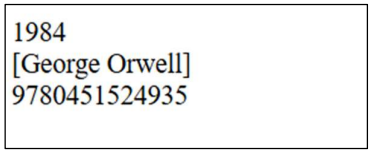
Hình Trang chủ của website

- Click vào "List of books" đến màn hình "Liệt kê danh sách sách"



Hình Liệt kê danh sách Sách

- Click vào ISBN của sách The Hobbit để vào trang thông tin chi tiết quyển sách



Hình Thông tin chi tiết quyển sách có ISBN là 9780451524935

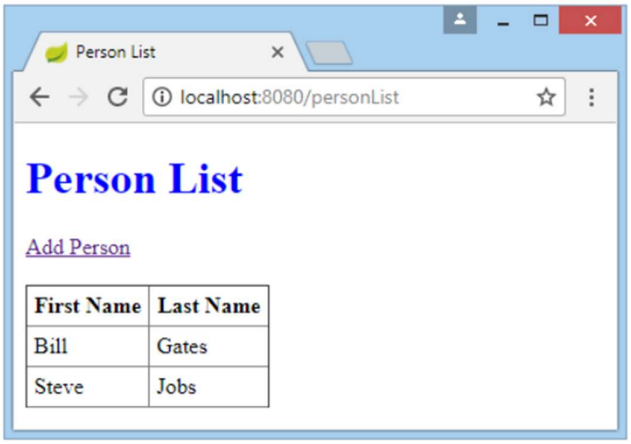
1.2.2. Bài tập làm thêm

Dự án 1.7. Tạo dự án có trong bài viết *https://spring.io/guides/gs/spring-boot* (một hướng dẫn từ chính hãng Spring) giúp các bạn làm quen với đọc tài liệu của hãng từ đó có thể dễ dàng đọc các tính năng mới của Spring Boot 4 và Spring Framework 7 nếu có. Dự án này giúp các bạn tạo một dự án RESTFul API đơn giản, cách kiểm tra 1 dự án Spring và cách triển khai theo dõi vận hành một ứng dụng Spring

Dự án 1.8. Tạo dự án có trong bài viết https://www.baeldung.com/spring-boot-start. Dự án này giúp các

bạn làm quen với một số tính năng cơ bản có trong hệ sinh thế Spring bao gồm a) Về spring-boot-starter-web, spring-boot-starter-data-jpa, spring-boot-starter-thymeleaf, spring-boot-starter-security b) Về cách làm việc với hệ quản trị cơ sở dữ liệu quan hệ c) Về cách bảo mật 1 ứng dụng web. Tại thời điểm này, các bạn có thể vẫn chưa hiểu hết nhưng có thể Prompt để khám phá các chức năng cơ bản này.

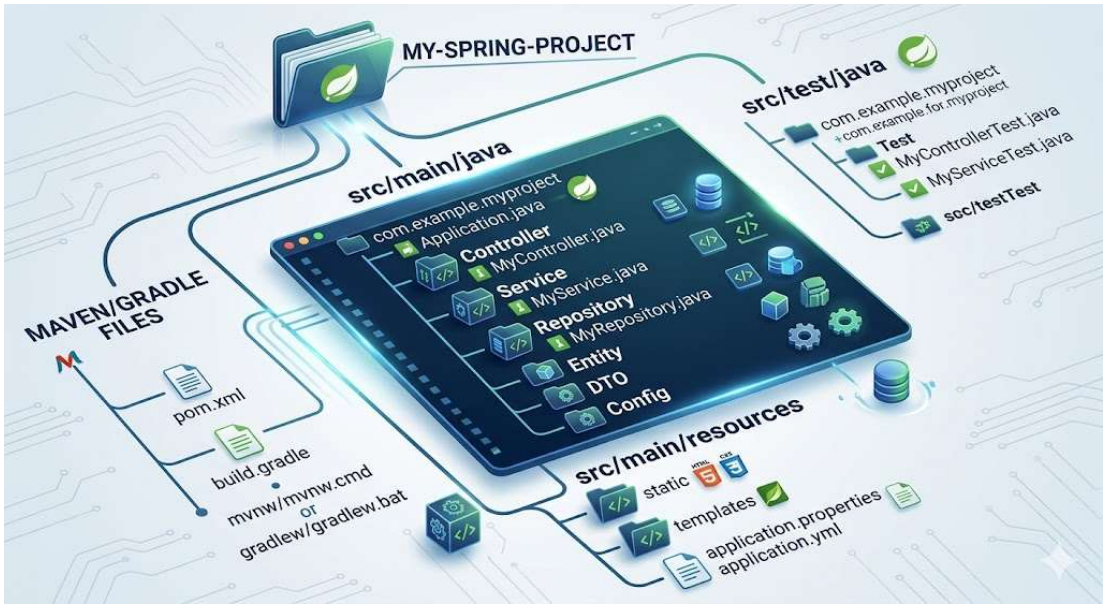
Dự án 1.9. Tạo dự án như trong bài viết *https://o7planning.org/11545/spring-boot-and-thymeleaf*



Hình Liệt kê danh sách các thành viên của công ty

Dự án 1.10. Tạo một dự án web cho phép hiển thị một hình captcha khác nhau cho mỗi thời gian truy cập vào URL của ứng dụng. Gợi ý thực hiện: a) Tạo dự án web như các bài trên b) Thu thập 20 hình captcha độc đáo c) Random chọn ngẫu nhiên 1 số từ 20 số d) Hiển thị hình đó trên ứng dụng web

1.3. PHẦN 3: SPRING PROJECT



Hình Cấu trúc dự án Spring Project với Spring Boot

1.3.1. Ôn tập lý thuyết với các PROMPT

+ Phân tích vai trò của Spring Boot trong kiến trúc Client-Server hiện đại: Làm thế nào một dự án Spring Boot có thể đồng thời phục vụ các tệp tin tĩnh (HTML/JS) cho Frontend và cung cấp dữ liệu qua API cho

Backend.

+ Trình bày cách sử dụng các Annotation @RestController và @RequestMapping để xây dựng một RESTful Web Service đơn giản. Giải thích cơ chế Spring Boot tự động chuyển đổi Java Object sang định dạng JSON để phản hồi cho Client.

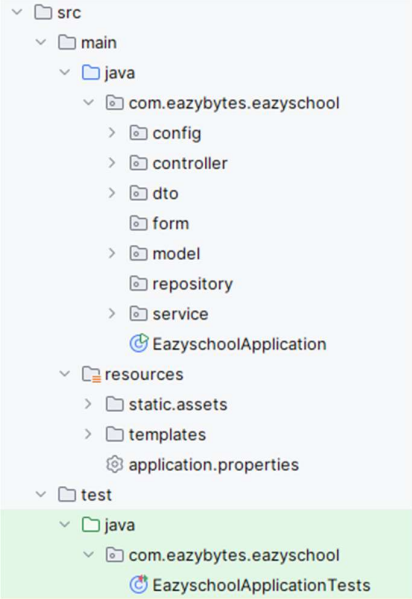
+ CORS (Cross-Origin Resource Sharing) là gì và tại sao nó quan trọng khi phát triển dự án tách biệt Frontend và Backend? Hãy hướng dẫn cách cấu hình CORS trong Spring Boot để cho phép một ứng dụng React hoặc Angular truy cập vào API.

+ Trong trường hợp không tách biệt hoàn toàn Frontend, hãy giải thích cách Spring Boot hỗ trợ các Template Engines như Thymeleaf để render giao diện trực tiếp từ phía Server. Ưu và nhược điểm của cách tiếp cận này so với việc dùng Single Page Application (SPA) là gì?

1.3.2. Bài làm mẫu

Bài 1: Tìm hiểu 1 dự án WEB mẫu sử dụng Spring Boot

Bước 1: Tải dự án 1_3_1_ex_01_blank.zip có cấu hình và một số tài nguyên cơ bản như sau



Dự án được tổ chức thành các package như sau:

- Tầng dùng chung:

  Gói form: chứa các lớp hỗ trợ xử lý phần giao diện tương tác với người dùng.

  Gói dto: chứa các lớp hỗ trợ chuyển dữ liệu giữa tầng ứng dụng và tầng dịch vụ.

- Tầng dữ liệu:

  Gói model**:** chứa các lớp bean dữ liệu

  Gói repository: chứa các lớp bean xử lý liên quan đến việc lưu trữ dữ liệu.

- Tầng dịch vụ:

  Gói service: chứa các lớp hỗ trợ tầng ứng dụng

- Tầng ứng dụng (giao diện)

  Gói config: chứa các cấu hình sử dụng cho ứng dụng ví dụ như cấu hình thông tin kết nối với cơ sở dữ liệu, thông tin kết nối với mail server, các bean mà bạn muốn khởi tạo.

  Gói controller: chứa các lớp xử lý tương tác với người dùng trên nền tảng WEB.

  Lớp EazyschoolApplication: ứng dụng spring boot

- Phần tài nguyên của ứng dụng

  static.assets: chứa các tài nguyên tĩnh của dự án như các tập tin css, image, …

  templates: chứa các trang thymeleaf, các trang giúp tạo ra giao diện cho ứng dụng web.

- Thông tin khác: dự án có sử dụng Lombok chi tiết cách dùng có thể đi nhanh đến chương 5.

Bước 2: Viết mã cho tầng dùng chung

```
//Đây là các Annotation của Lombok

@AllArgsConstructor
@Data
public class Product {
    private String name;
    private double price;
    private boolean inStock;

    public boolean getInStock() {
        return this.inStock;
    }
}
```

Bước 3: Viết mã cho tầng dữ liệu: tầng này có sử dụng validation chi tiết phần này được trình bày tại chương 3

```
@Data
public class Contact {
    @NotBlank(message="Name must not be blank")
    @Size(min=3, message="Name must be at least 3 characters long")
    private String name;

    @NotBlank(message="Mobile number must not be blank")
    @Pattern(regexp="(^$|[0-9]{10})",message = "Mobile number must be 10 digits")
    private String mobileNum;

    @NotBlank(message="Email must not be blank")
    @Email(message = "Please provide a valid email address" )
    private String email;

    @NotBlank(message="Subject must not be blank")
    @Size(min=5, message="Subject must be at least 5 characters long")
    private String subject;

    @NotBlank(message="Message must not be blank")
    @Size(min=10, message="Message must be at least 10 characters long")
    private String message;
}


@Data
public class Holiday {

    private final String day;
    private final String reason;
    private final Type type;

    public enum Type {
        FESTIVAL, FEDERAL
```

```
    }
}
```

Bước 4: Viết mã cho tầng dịch vụ

```
@Slf4j
@Service(value="contactService")
public class ContactService {

    public void listAllBeans() {
        String[] beanNames =
             EazyschoolApplication.applicationContext.getBeanDefinitionNames();

        java.util.Arrays.sort(beanNames);

        for (String beanName : beanNames) {
            System.out.println("Bean Name: " + beanName);
        }
    }
    private int counter = 0;

    public ContactService(){
        System.out.println("Contact Service Bean initialized");
    }

    public boolean saveMessageDetails(Contact contact){
        boolean isSaved = true;
        //TODO - Need to persist the data into the DB table
        log.info(contact.toString());

        listAllBeans();
        return isSaved;
    }

    public int getCounter() {
        return counter;
    }

    public void setCounter(int counter) {
        this.counter = counter;
    }
}
```

Bước 5: Viết mã cho tầng ứng dụng hay tầng giao diện

Phần mã bằng ngôn ngữ Java

```
@Slf4j
@Controller
public class ContactController {

    private final ContactService contactService;
    @Resource(name = "shoppingCart")
    private List<Product> products;

    @Autowired
    public ContactController(ContactService contactService) {
        this.contactService = contactService;
    }

    @RequestMapping("/contact")
    public String displayContactPage(Model model) {
        model.addAttribute("contact", new Contact());
        model.addAttribute("prods", products);

        return "contact.html";
    }

    @RequestMapping(value = "/saveMsg",method = POST)
```

```
    //@PostMapping(value = "/saveMsg") @Valid
    public String saveMessage(@ModelAttribute("contact") Contact contact,
                              Errors errors) {
        if(errors.hasErrors()){
            log.error("Contact form validation failed due to : " +
                            errors.toString());
            return "contact.html";
        }
        contactService.saveMessageDetails(contact);
        contactService.setCounter(contactService.getCounter()+1);

        log.info("Number of times the Contact form is submitted : "
                + contactService.getCounter());
        return "redirect:/contact";
    }
}


@Slf4j
@Controller
public class HolidaysController {


    @GetMapping("/holidays/{display}")
    public String displayHolidays(@PathVariable String display, Model model) {
        if(null != display && display.equals("all")){
            model.addAttribute("festival",true);
            model.addAttribute("federal",true);
        }else if(null != display && display.equals("federal")){
            model.addAttribute("federal",true);
        }else if(null != display && display.equals("festival")){
            model.addAttribute("festival",true);
        }
        List<Holiday> holidays = Arrays.asList(
                new Holiday(" Jan 1 ","New Year's Day", Holiday.Type.FESTIVAL),
                new Holiday(" Oct 31 ","Halloween", Holiday.Type.FESTIVAL),
                new Holiday(" Nov 24 ","Thanksgiving Day", Holiday.Type.FESTIVAL),
                new Holiday(" Dec 25 ","Christmas", Holiday.Type.FESTIVAL),
                new Holiday(" Jan 17 ","Martin Luther King Jr. Day",
Holiday.Type.FEDERAL),
                new Holiday(" July 4 ","Independence Day", Holiday.Type.FEDERAL),
                new Holiday(" Sep 5 ","Labor Day", Holiday.Type.FEDERAL),
                new Holiday(" Nov 11 ","Veterans Day", Holiday.Type.FEDERAL)
        );
        Holiday.Type[] types = Holiday.Type.values();
        for (Holiday.Type type : types) {
            model.addAttribute(type.toString(),
                    (holidays.stream().filter(holiday ->
holiday.getType().equals(type)).collect(Collectors.toList())));
        }

        return "holidays.html";
    }

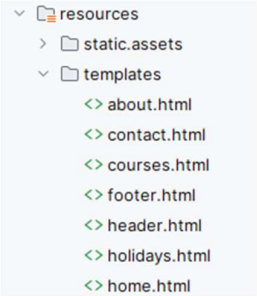
}


@Controller
public class HomeController {

    @RequestMapping(value={"", "/", "home"})
    public String displayHomePage() {
        return "home.html";
    }

}
```

Phần mã bằng Thymeleaf của tập tin xem thêm trong tập tin zip tải về ở trên tên các file đó như hình bên dưới



Bước 6: Viết code cho phần cấu hình cho ứng dụng

```
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addViewControllers(ViewControllerRegistry registry) {
        registry.addViewController("/courses").setViewName("courses");
        registry.addViewController("/about").setViewName("about");
    }
}


@Configuration
public class WebSessionScopeConfig {

    @Bean
    @SessionScope
    public List<Product> shoppingCart(){
        List<Product> list = new ArrayList<Product>();
        list.add(new Product("Iphone", 1000, true));
        list.add(new Product("Tablet", 2000, true));
        return list;
    }
}
```

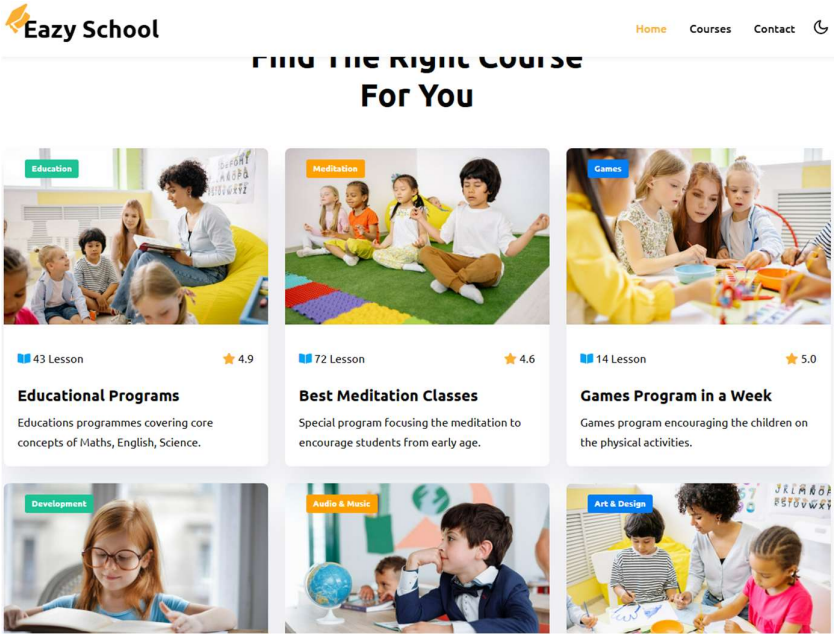
Bước 7: Thực thi ứng dụng các bạn sẽ có được kết quả như sau:

1. http://localhost:8080/home



Hình Trang chủ dự án

2. http://localhost:8080/courses



Hình Liệt kê danh sách các khóa học

1.3.3. Bài tập làm thêm

***Dự án 1.11**. Tạo ứng dụng web động giúp tra cứu điểm của học sinh thi tốt nghiệp phổ thông dựa vào tập tin excel chưa thông tin điểm thi gồm các cột MSSV, Họ và Tên, Điểm Toán, Điểm Văn và Điểm Anh Văn.

Gợi ý thực hiện: a) đọc trực tiếp từ tập tin excel hoặc chuyển dữ liệu lên hệ quản trị cơ sở dữ liệu b) sử dụng chức năng captcha để hạn chế việc DDoS ứng dụng c) truy vấn và hiển thị điểm của thí sinh

***Dự án 1.12.** Tạo ứng dụng RESTful API để hỗ trợ các ứng dụng bên thứ ba (như web application, mobile application) có thể tra cứu thông tin thông qua API

Gợi ý thực hiện: a) dựng API trên dự án ở dự án 10. b) Xây dựng Restful API với input là SBD, **User-Token** c) Kiểm tra User-Token hợp lệ mới trả về kết quả là JSON tương ứng với điểm số của học sinh

***Dự án 1.13.** Tạo ứng dụng web hiển thị sơ yếu lý lịch (CV) của các bạn được trình bày với CV template đẹp mắt và host trên Internet hoặc sử dụng github để host

***Dự án 1.14.** Tạo ứng dụng web để nhận thông tin phản hồi về việc sử dụng dịch vụ của khách hàng

Gợi ý thực hiện: a) Tìm 1 màn hình phản hồi về dịch vụ các bạn sử dụng có trên mạng b) thực hiện cài đặt màn hình đó cho dự án của mình. c) Sử dụng Prompts hỗ trợ sinh code HTML, CSS cho các bạn

D. TÓM TẮT CHƯƠNG 1

Chương này đã cung cấp một cái nhìn toàn diện về hệ sinh thái Spring, từ nền tảng vững chắc của Spring Framework cho đến sự tiện lợi vượt bậc của Spring Boot trong việc tối giản hóa cấu hình dự án. Thông qua việc tìm hiểu cách Spring Boot đóng vai trò làm cầu nối giữa Backend và Frontend, người học đã nắm bắt được quy trình phát triển một ứng dụng Web hiện đại. Những kiến thức lý thuyết này không còn nằm trên

trang giấy mà đã được cụ thể hóa thông qua các dự án nhỏ, giúp sinh viên làm quen với luồng công việc thực tế của một nhà phát triển phần mềm.

CHƯƠNG 2: RESTFUL API

A. MỤC TIÊU CHƯƠNG

Hỗ trợ việc luyện tập và nghiên cứu nội dung Chương 3 có trong đề cương học phần Chuyên đề J2EE (841468) bao gồm RESTful API là gì? Cách thiết kế và xây dựng RESTful API? Cách để tạo tài liệu cho RESTful API với Swagger? HATEOAS là gì? Khi nào thiết kế RESTful API sử dụng HATEOAS.

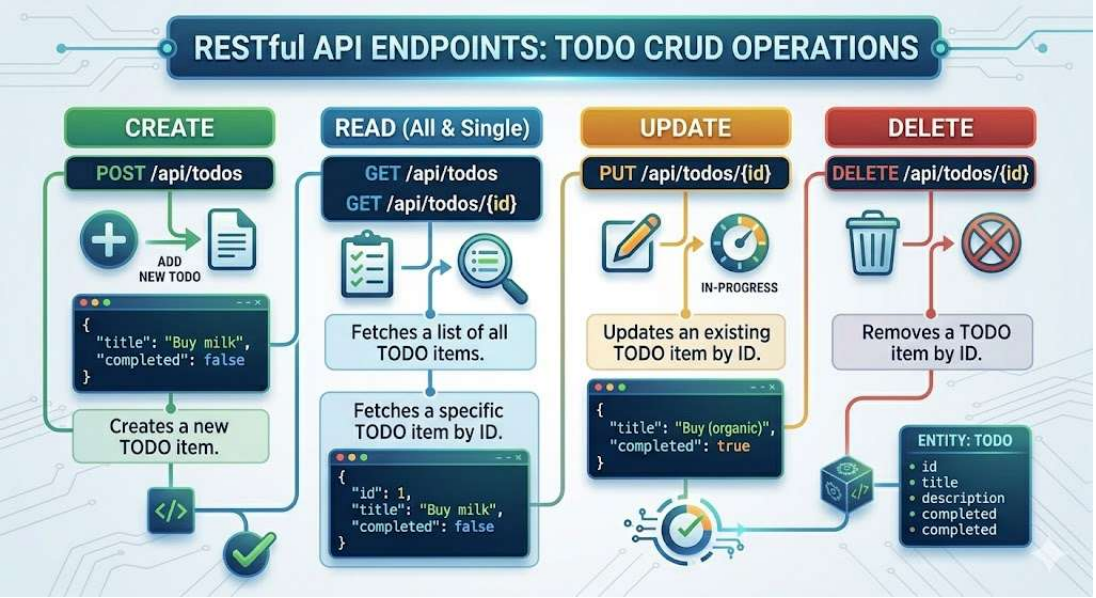
B. KẾT CẤU CHƯƠNG

Chương bao gồm 3 phần là

- Phần 1: RESTful API
- Phần 2: Swagger
- Phần 3: HATEOAS

C. NỘI DUNG CHƯƠNG

2.1. PHẦN 1: RESTFUL API



Hình Hiển thị danh sách Restful API trên thực thể TODO

2.1.1. Ôn tập lý thuyết với các PROMPT

+ RESTful API là gì? Hãy giải thích triết lý 'Resource-based' (dựa trên tài nguyên) và tại sao tính chất 'Stateless' (không lưu trạng thái) lại giúp hệ thống có khả năng mở rộng cực cao?

+ Trong thiết kế RESTful API, hãy phân biệt ý nghĩa và cách sử dụng của 5 phương thức: GET, POST, PUT, PATCH và DELETE. Khi nào chúng ta nên dùng PUT thay vì PATCH?

+ Giải thích vai trò và sự khác biệt của các Annotation sau trong việc tiếp nhận dữ liệu từ người dùng: @PathVariable, @RequestParam và @RequestBody. Cho ví dụ một tình huống cụ thể sử dụng cả ba.

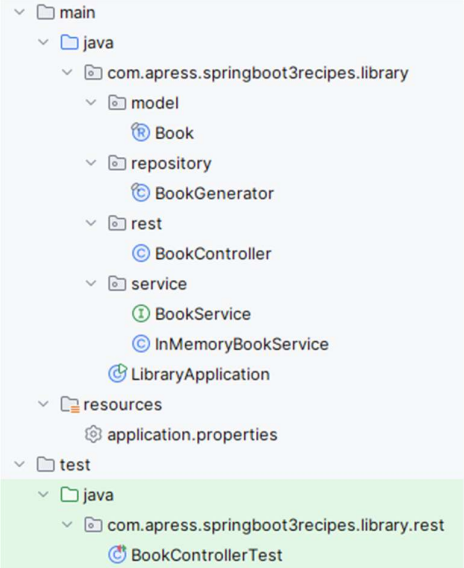
+ "Phân tích cách @RestController hoạt động trong Spring Boot. Làm thế nào mà một Java Object lại có thể tự động biến đổi thành chuỗi JSON để gửi về cho phía Frontend thông qua thư viện Jackson tích hợp sẵn?

+ Để xây dựng một RESTful API chuyên nghiệp, tại sao chúng ta nên chia code thành các lớp Controller, Service và Repository? Hãy giải thích luồng đi của một Request từ khi Client gửi đi cho đến khi nhận được kết quả.

2.1.2. Bài làm mẫu

Bài 1: Xây dựng RESTful API cho phép liệt kê, tìm kiếm và thêm 1 quyển sách mới

Bước 1**:** Tạo dự án Spring Boot và tổ chức dự án như hình sau:



Bước 2: Viết mã cho tầng dữ liệu (chứa gói model và gói repository)

```
public record Book(String isbn, String title, List<String> authors) {

    @Override
    public boolean equals(Object o) {
       if (o instanceof Book other) {
          return this.isbn != null && Objects.equals(this.isbn, other.isbn);
       }
       return false;
    }

    @Override
    public int hashCode() {
       return Objects.hash(this.isbn);
    }
}

public final class BookGenerator {

    public static List<Book> all() {
       try {
          var books = new ClassPathResource("books.csv").getInputStream();
          try (var lines = new BufferedReader(new InputStreamReader(books)).lines())
{
             return lines
                .skip(1)
                .map((line) -> line.split("\\|"))
```

```
                .map((row) -> new Book(row[0], row[1], List.of(row[2].split(","))))
                .toList();
          }
       } catch (IOException ex) {
          throw new IllegalStateException("Error loading 'books.csv'!", ex);
       }
    }

    public static List<Book> random(int size) {
       var books = all();
       if (size >= books.size()) {
          return books;
       }
       return IntStream.range(0, size)
             .mapToObj( (x) -> randomBook(books)).toList();
    }

    public static Book random() {
       return randomBook(all());
    }

    private static Book randomBook(List<Book> books) {
       var idx = ThreadLocalRandom.current().nextInt(books.size());
       return books.get(idx);
    }
}
```

Bước 3: Viết mã cho tầng dịch vụ (chứa gói service)

```
public interface BookService {
    Iterable<Book> findAll();
    Book create(Book book);
    Optional<Book> find(String isbn);
}


@Service
class InMemoryBookService implements BookService {

    private final Map<String, Book> books = new ConcurrentHashMap<>();

    @Override
    public Iterable<Book> findAll() {
       return books.values();
    }

    @Override
    public Book create(Book book) {
       books.put(book.isbn(), book);
       return book;
    }

    @Override
    public Optional<Book> find(String isbn) {
       return Optional.ofNullable(books.get(isbn));
    }
}
```

Bước 4: Viết mã cho tầng ứng dụng (chứa gói rest)

```
@RestController
@RequestMapping("/books")
public class BookController {

    private final BookService bookService;

    public BookController(BookService bookService) {
       this.bookService = bookService;
    }
```

```
    @GetMapping
    public Iterable<Book> all() {
       return bookService.findAll();
    }

    @GetMapping("/{isbn}")
    public ResponseEntity<Book> get(@PathVariable("isbn") String isbn) {
       return ResponseEntity.of(bookService.find(isbn));
    }

    @PostMapping
    public ResponseEntity<Book> create(@RequestBody Book book,
                                       UriComponentsBuilder uriBuilder) {
    var created = bookService.create(book);
    var newBookUri = uriBuilder.path("/books/{isbn}").build(created.isbn());
    return ResponseEntity.created(newBookUri).body(created);
    }
}


@SpringBootApplication
public class LibraryApplication {

    public static void main(String[] args) {
       SpringApplication.run(LibraryApplication.class, args);
    }

    @Bean
    public ApplicationRunner booksInitializer(BookService bookService) {
       return args -> {
          bookService.create(
                      new Book("9780061120084", "To Kill a Mockingbird",
                      List.of("Harper Lee")));
          bookService.create(
                      new Book("9780451524935", "1984",
                                  List.of("George Orwell")));
          bookService.create(
                      new Book("9780618260300", "The Hobbit",
                      List.of("J.R.R. Tolkien")));
       };
    }
}
```

Bước 5: Viết mã để kiểm tra tính năng của ứng dụng

```
@WebMvcTest(BookController.class)
public class BookControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private BookService bookService;

    @Test
    public void shouldReturnListOfBooks() throws Exception {

       when(bookService.findAll()).thenReturn(Arrays.asList(
             new Book("123", "Spring 5 Recipes", List.of("Marten Deinum", "Josh
Long")),
             new Book("321", "Pro Spring MVC", List.of("Marten Deinum", "Colin
Yates"))));

       mockMvc.perform(get("/books"))
          .andExpect(status().isOk())
          .andExpect(MockMvcResultMatchers.jsonPath("$", Matchers.hasSize(2)))
          .andExpect(MockMvcResultMatchers.jsonPath("$[*].isbn",
Matchers.containsInAnyOrder("123", "321")))
```

```
          .andExpect(MockMvcResultMatchers.jsonPath("$[*].title",
Matchers.containsInAnyOrder("Spring 5 Recipes", "Pro Spring MVC")));
    }

    @Test
    public void shouldReturn404WhenBookNotFound() throws Exception {

       when(bookService.find(anyString())).thenReturn(Optional.empty());

       mockMvc.perform(get("/books/123"))
             .andExpect(status().isNotFound());
    }

    @Test
    public void shouldReturnBookWhenFound() throws Exception {

       when(bookService.find(anyString())).thenReturn(
                   Optional.of(new Book("123", "Spring 6 Recipes", List.of("Marten
Deinum", "Josh Long"))));

       mockMvc.perform(get("/books/123"))
             .andExpect(status().isOk())
             .andExpect(MockMvcResultMatchers.jsonPath("$.isbn",
Matchers.equalTo("123")))
             .andExpect(MockMvcResultMatchers.jsonPath("$.title",
Matchers.equalTo("Spring 6 Recipes")));
    }

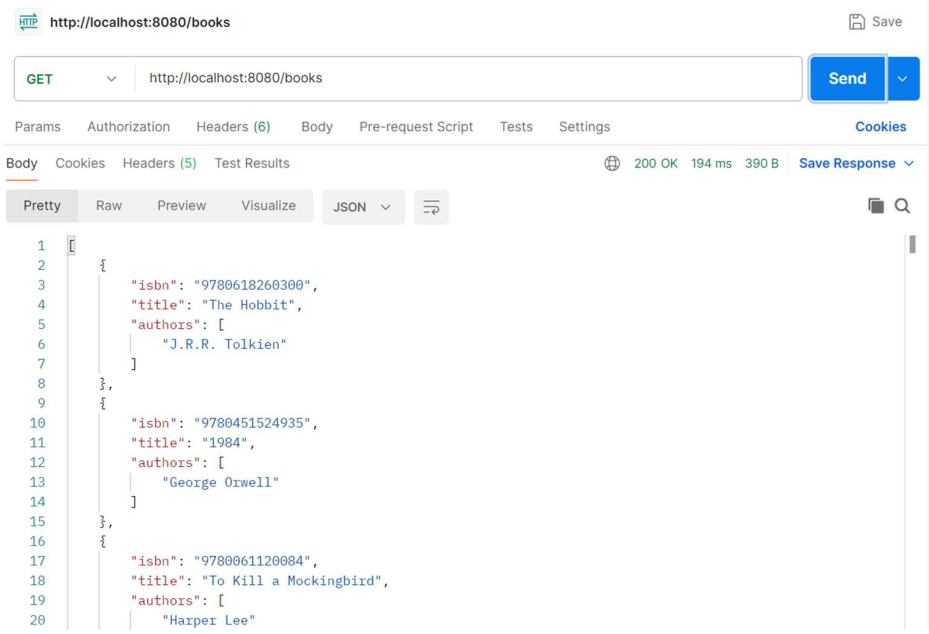
    @Test
    public void shouldAddBook() throws Exception {
        when(bookService.create(any(Book.class))).thenReturn(new Book("123456789",
"Test Book Stored",
               List.of("T. Author")));

        mockMvc.perform(post("/books")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{ \"isbn\" : \"123456789\", \"title\" : \"Test Book\",
\"authors\" : [\"T. Author\"]}"))
            .andExpect(status().isCreated())
            .andExpect(header().string("Location",
"http://localhost/books/123456789"));
  }

}
```

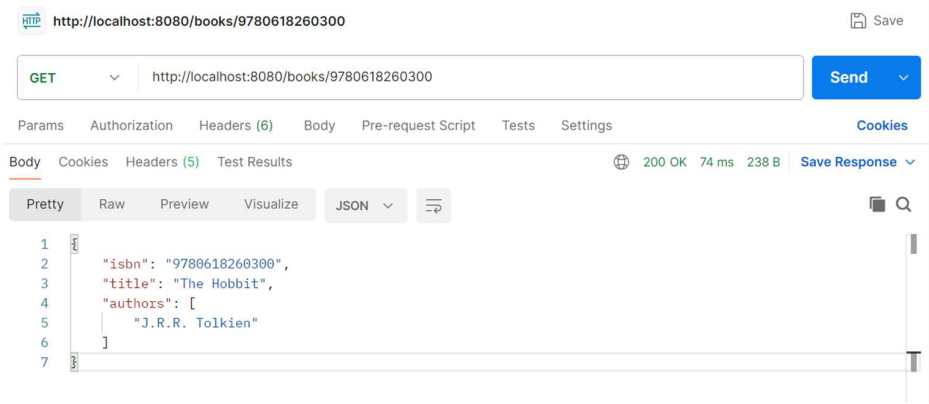
Bước 6: Thực hiện ứng dụng với ứng dụng POSTMAN (xem hướng dẫn và tải về cài đặt tại https://www.postman.com/

- Liệt kê danh sách các quyển sách



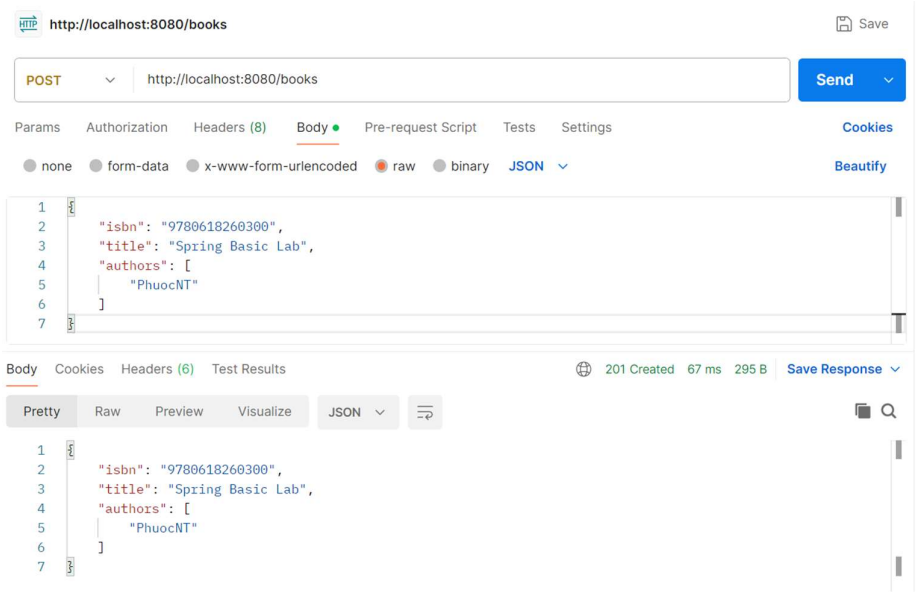
Hình Liệt kê danh sách quyển sách khi sử dụng POSTMAN để gọi endpoint

- Lấy thông tin của quyển sách khi biết isbn của quyển sách



Hình Thông tin của 1 quyển sách

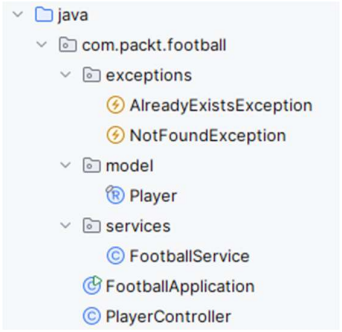
- Tạo quyển sách mới



Hình Thêm một quyển sách bằng ứng dụng POSTMAN

Bài 2: **Xây dựng RESTful API để thực hiện CRUD trên thực thể Player.**

Bước 1: Tạo dự án Spring Boot với cấu trúc dự án như hình bên dưới



Bước 2: Viết mã cho tầng dữ liệu

```
public record Player(String id, int jerseyNumber, String name,
                     String position, LocalDate dateOfBirth) {
}
```

Bước 3: Viết mã cho tầng dịch vụ

```
@Service
public class FootballService {
    private final Map<String, Player> players = new HashMap<String,
Player>(Map.ofEntries(
            Map.entry("1884823", new Player("1884823", 5, "Ivana ANDRES",
"Defender", LocalDate.of(1994, 07, 13))),
            Map.entry("325636", new Player("325636", 11, "Alexia PUTELLAS",
"Midfielder", LocalDate.of(1994, 02, 04))),
            Map.entry("396930", new Player("396930", 2, "Ona BATLLE", "Defender",
LocalDate.of(1999, 06, 10))),
            Map.entry("377762", new Player("377762", 6, "Aitana BONMATI",
"Midfielder", LocalDate.of(1998, 01, 18))),
            Map.entry("387133", new Player("387133", 4, "Irene PAREDES", "Defender",
LocalDate.of(1998, 01, 18))),
            Map.entry("387134", new Player("387134", 8, "Vicky LOSADA",
"Midfielder", LocalDate.of(1991, 01, 01))),
```

```
            Map.entry("396911", new Player("396911", 15, "Eva NAVARRO", "Forward",
LocalDate.of(2001, 01, 27))),
            Map.entry("420276", new Player("420276", 9, "Esther GONZALEZ",
"Forward", LocalDate.of(1992, 12, 8))),
            Map.entry("377723", new Player("377723", 20, "Rocio GALVEZ", "Defender",
LocalDate.of(1997, 04, 15))),
            Map.entry("396914", new Player("396914", 14, "Laia CODINA", "Defender",
LocalDate.of(2000, 01, 22))),
            Map.entry("420284", new Player("420284", 21, "Claudia ZORNOZA",
"Midfielder", LocalDate.of(1990, 10, 20))),
            Map.entry("415394", new Player("415394", 18, "Salma PARALLUELO",
"Forward", LocalDate.of(2003, 11, 13))),
            Map.entry("387138", new Player("387138", 10, "Jennifer HERMOSO",
"Forward", LocalDate.of(1990, 05, 9))),
            Map.entry("467287", new Player("467287", 16, "Maria PEREZ",
"Midfielder", LocalDate.of(2001, 12, 24))),
            Map.entry("420277", new Player("420277", 7, "Irene GUERRERO",
"Midfielder", LocalDate.of(1996, 12, 12))),
            Map.entry("396929", new Player("396929", 12, "Oihane HERNANDEZ",
"Defender", LocalDate.of(2000, 05, 04))),
            Map.entry("396907", new Player("396907", 23, "Cata COLL", "Goalkeeper",
LocalDate.of(2001, 04, 23))),
            Map.entry("467297",
                    new Player("467297", 22, "Athenea DEL CASTILLO", "Forward",
LocalDate.of(2000, 10, 24))),
            Map.entry("398097", new Player("398097", 8, "Mariona CALDENTEY",
"Forward", LocalDate.of(1996, 03, 19))),
            Map.entry("413022", new Player("413022", 3, "Teresa ABELLEIRA",
"Midfielder", LocalDate.of(2000, 01, 9))),
            Map.entry("415396", new Player("415396", 13, "Enith SALON",
"Goalkeeper", LocalDate.of(2001, 9, 24)))));

    public List<Player> listPlayers() {
        return players.values().stream().collect(Collectors.toList());
    }

    public Player getPlayer(String id) {
        Player player = players.get(id);
        if (player == null)
            throw new NotFoundException("Player not found");
        return player;
    }

    public Player addPlayer(Player player) {
        if (players.containsKey(player.id())) {
            throw new AlreadyExistsException("The player already exists");
        } else {
            players.put(player.id(), player);
            return player;
        }
    }

    public Player updatePlayer(Player player) {
        if (!players.containsKey(player.id())) {
            throw new NotFoundException("The player does not exist");
        } else {
            players.put(player.id(), player);
            return player;
        }
    }

    public void deletePlayer(String id) {
        if (players.containsKey(id)) {
            players.remove(id);
        }
    }
}
```

Bước 4: Viết mã cho tầng ứng dụng (hay RESTful)

```
@RequestMapping("/players")
@RestController
public class PlayerController {
    private FootballService footballService;
    public PlayerController(FootballService footballService) {
        this.footballService = footballService;
    }
    @GetMapping
    public List<Player> listPlayers() {
        return footballService.listPlayers();
    }
    @GetMapping("/{id}")
    public Player readPlayer(@PathVariable String id) {
        return footballService.getPlayer(id);
    }
    @PostMapping
    public void createPlayer(@RequestBody Player player) {
        footballService.addPlayer(player);
    }
    @PutMapping("/{id}")
    public void updatePlayer(@PathVariable String id, @RequestBody Player player) {
        footballService.updatePlayer(player);
    }
    @DeleteMapping("/{id}")
    public void deletePlayer(@PathVariable String id) {
        footballService.deletePlayer(id);
    }
    @ResponseStatus(value = HttpStatus.NOT_FOUND, reason = "Not found")
    @ExceptionHandler(NotFoundException.class)
    public void notFoundHandler() {
    }
    @ResponseStatus(value = HttpStatus.BAD_REQUEST, reason = "Already exists")
    @ExceptionHandler(AlreadyExistsException.class)
    public void alreadyExistsHandler() {
    }
}
```

Bước 5: Viết mã cho phần xử lý ngoại lệ (mã này được giải thích chi tiết ở chương 4)

```
public class AlreadyExistsException extends RuntimeException {
    public AlreadyExistsException(String message) {
        super(message);
    }
}


public class NotFoundException extends RuntimeException {
    public NotFoundException(String message) {
        super(message);
    }
}
```

Bước 6: Thực hiện kiểm tra với POSTMAN

- Endpoint http://localhost:8080/players với method GET: liệt kê danh sách các player

- Endpoint http://localhost:8080/players/{id} với method GET: liệt kê thông tin chi tiết của player với id = {id}

- Endpoint http://localhost:8080/players với method POST: thêm một player mới với thông tin có trong body hoặc JSON data.

- Endpoint http://localhost:8080/players/{id} với method PUT: cập nhật thông tin của player có id = {id} và dữ liệu mới có trong body hoặc JSON data.

- Endpoint http://localhost:8080/players/{id} với method DELETE: xóa player với id = {id}

2.1.3. Bài tập làm thêm

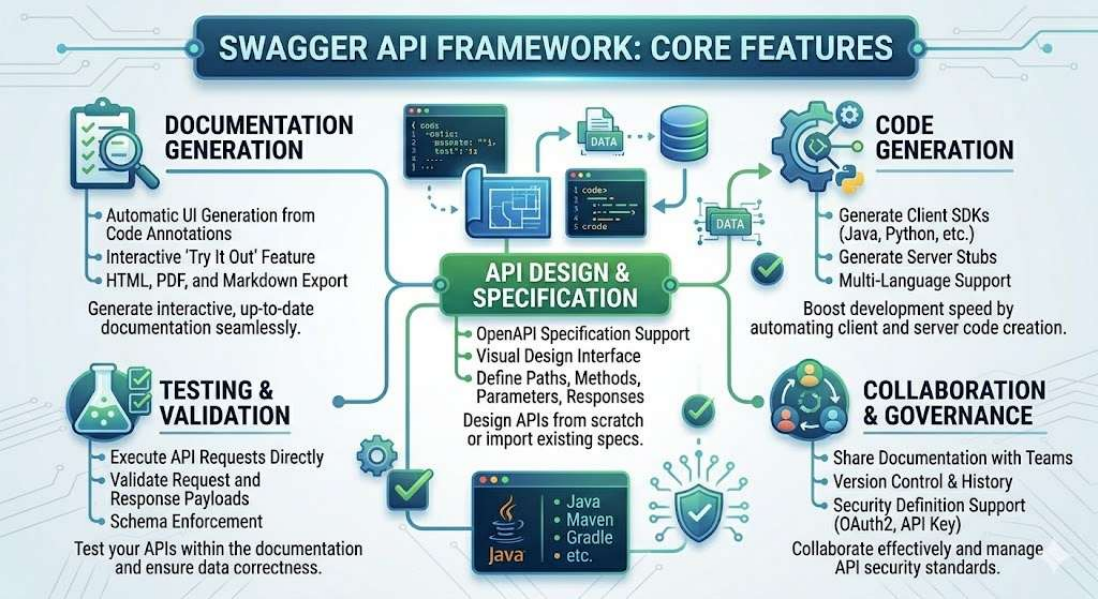
Dự án 2.1. Tạo dự án có trong bài viết https://www.javaguides.net/2022/12/spring-boot-3-crud-restful-webservices.html. Dự án tạo các RESTful API cho phép CRUD trên thực thể bài báo nghiên cứu khoa học (sinh viên tự tìm hiểu xem thông tin 1 bài báo khoa học mình cần lưu là gì từ google scholar)

Dự án 2.2. Tạo dự án có trong bài viết *https://o7planning.org/10865/simple-crud-example-with-spring-mvc-restful-web-service* hoặc bài viết *https://o7planning.org/11645/crud-restful-web-service-example-with-spring-boot*. Dự án tạo các RESTful API cho phép CRUD trên thực thể nhân viên

Dự án 2.3. Tạo dự án RESTful cho phép thực hiện CRUD trên thực thể sơ yếu lý lịch (CV) bao gồm các thông tin thông tin cá nhân ứng viên bao gồm CCCD, Họ và tên, Ngày sinh, Địa chỉ; thông tin trình độ văn hóa; thông tin về kinh nghiệm làm việc.

Chú ý: CRUD là từ viết tắt của: Create – Tạo thực thể; Retrieve – Liệt kê thông tin các thực thể, thông tin chi tiết một thực thể, tìm kiếm thông tin một thực thể dựa vào id và các thông tin khác, hỗ trợ phân trang nếu có; Update – Cập nhật thông tin một thực thể; Delete – Xóa thông tin 1 thực thể.

2.2. PHẦN 2: SWAGGER & OPEN API



Hình Các tính năng quan trọng của SWAGGER

2.2.1. Ôn tập lý thuyết với các PROMPT

+ Swagger và OpenAPI Specification (OAS) là gì? Tại sao chúng lại được coi là tiêu chuẩn 'vàng' trong việc viết tài liệu cho RESTful API hiện nay?

+ Làm thế nào để tích hợp Swagger vào dự án Spring Boot 4 bằng thư viện springdoc-openapi-starter-webmvc-ui? Hãy liệt kê các bước từ thêm dependency vào pom.xml đến việc truy cập giao diện Swagger UI.

+ Giải thích cơ chế tự động quét (scan) của SpringDoc. Làm thế nào mà Swagger có thể liệt kê chính xác các phương thức (GET, POST...), tham số và kiểu dữ liệu trả về mà không cần chúng ta phải viết tài liệu thủ công?

+ Cách sử dụng các Annotation như @Tag, @Operation, và @ApiResponse để mô tả chi tiết cho từng API là gì? Hãy cho một ví dụ code minh họa cách đặt tên nhóm API và mô tả các mã lỗi (404, 500) trong Swagger UI.

+ Tính năng 'Try it out' trong Swagger UI hoạt động như thế nào? Làm thế nào một lập trình viên Frontend có thể sử dụng giao diện này để kiểm tra dữ liệu trả về mà không cần dùng đến các công cụ bên ngoài như Postman?

+ SpringDoc OpenAPI đã thay thế hoàn toàn Springfox cũ để triển khai Swagger? Có nên sử dụng Springfox cho việc triển khai Swagger trên các dự án Spring Boot 3&4 không?

2.2.2. Bài làm mẫu

Bài 1: Viết tài liệu cho các REST API có trong dự án bằng Spring Open API

Bước 1: Mở dự án đã thực hiện ở Chương 2 – Phần 1 – Bài 2.

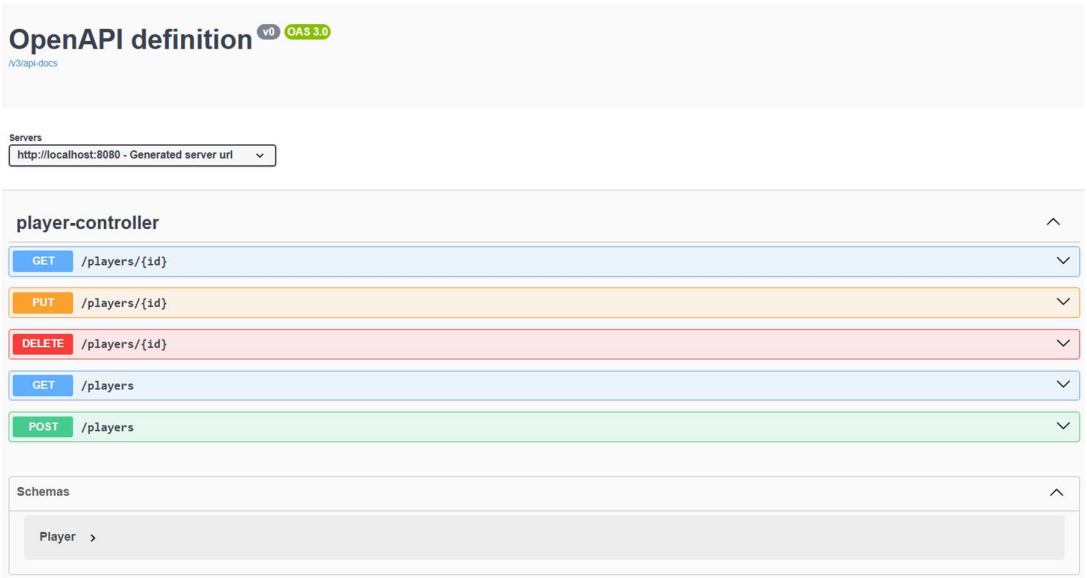
Bước 2: Cập nhật tập tin pom.xml của dự án bằng cách thêm 1 dependence là org.springdoc:springdoc-openapi-starter-webmvc-ui giống như hình bên dưới

```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.6.0</version>
</dependency>
```

Bước 3: Chạy dự án tại lớp PlayerController. Bây giờ bạn có thể sử dụng các RESTful API đã xây dựng và có thể xem phần tài liệu hướng dẫn sử dụng các RESTful API này

Bước 4: Xem tài liệu hướng dẫn RESTful API có trong dự án này bằng cách

- Mở trình duyệt với URL: http://localhost:8080/swagger-ui/index.html xuất hiện màn hình như bên dưới liệt kê tất cả RESTful API có trong dự án và các tham số của từng RESTful API



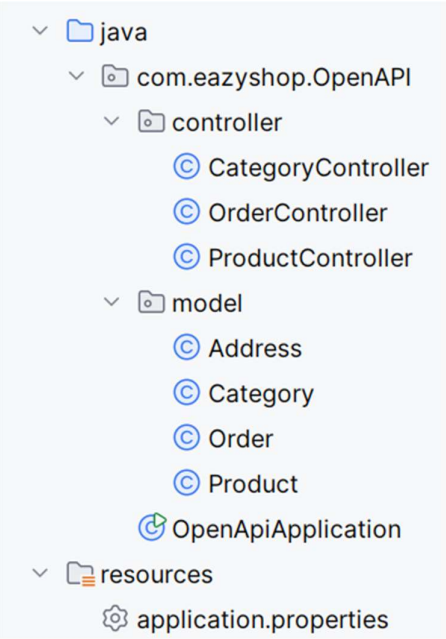
Hình Liệt kê danh sách các RESTful API có trong ứng dụng sử dụng Swagger-UI

Chú ý: Hiện tại không có thông tin hướng dẫn từng endpoint bởi vì chúng ta không viết phần hướng dẫn đó trong các API có trong dự án. Viết hướng dẫn thế nào các bạn tra cứu thêm thông tin trên

internet.

Bài 2: Viết tài liệu mô tả chi tiết các REST API có trong dự án bằng Spring Open API

Bước 1: Tải dự án 2_2_2_ex_02_blank.zip có cấu hình và một số tài nguyên cơ bản như sau



Bước 2: Viết mô tả về dự án RESTful API

```
@SpringBootApplication
public class OpenApiApplication {

    public static void main(String[] args) {
       SpringApplication.run(OpenApiApplication.class, args);
    }

    @Bean
    public OpenAPI eazyShopOpenAPI() {
       List<Tag> tags = new ArrayList<>();
       Tag tag = new Tag();
       tag.setName("Category");
       tags.add(tag);
       return new OpenAPI()
             .info(new Info().title("Eazy Shop API")
                   .description("Eazy shop sample application")
                   .version("v0.0.1")
                   .contact(new Contact()
                       .name("Eazy Shop")
                       .email("support@eazyshop.com")
                       .url("www.eazyshop.com"))
                   .license(new License()
                       .name("Apache 2.0")
                       .url("http://springdoc.org")))
             .externalDocs(new ExternalDocumentation()
                   .description("EazyShop Wiki Documentation")
                   .url("https://eazyshop.wiki.github.org/docs"))
             .tags(tags);

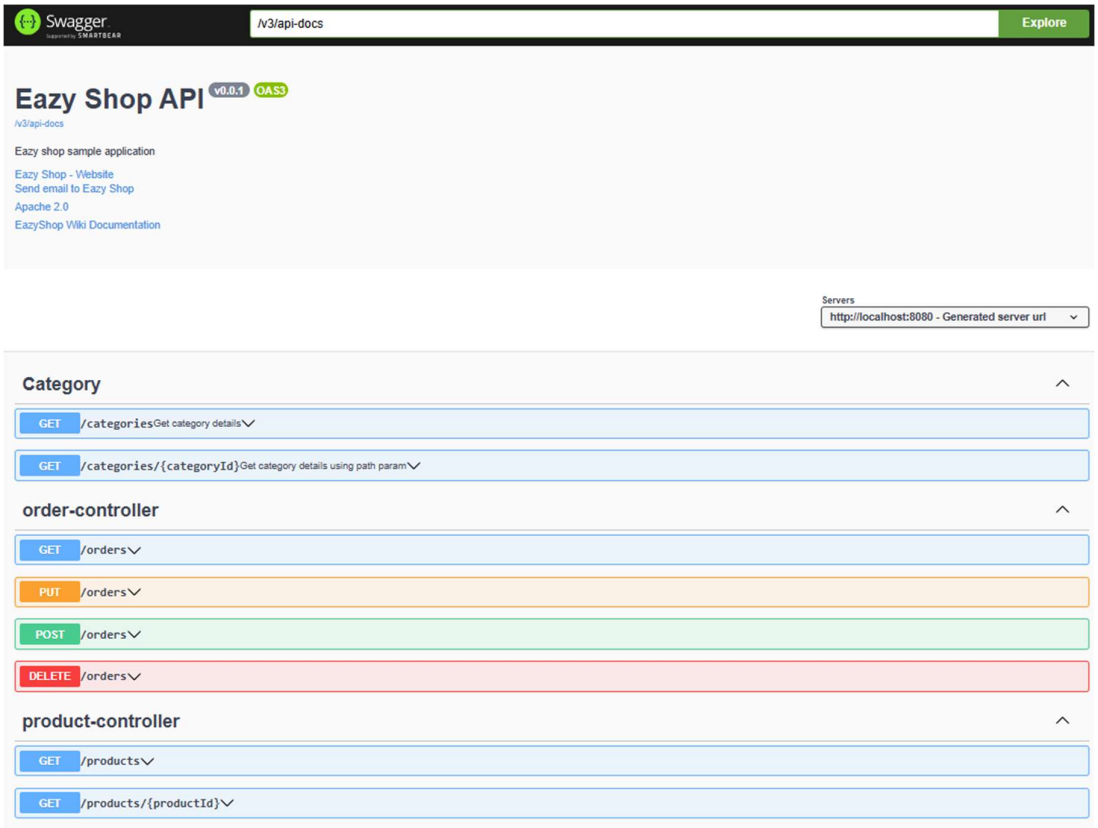

    }
}
```

Bước 3: Viết mô tả các RESTful API cho CategoryController

```
@RestController
public class CategoryController {
    @Tag(name = "Category")
    @Operation(summary = "Get category details")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Found the Category",
              content = { @Content(mediaType = "application/json",
              schema = @Schema(implementation = Category.class)) }),
        @ApiResponse(responseCode = "400", description = "Invalid id supplied",
              content = @Content),
        @ApiResponse(responseCode = "404", description = "Category not found",
                    content = @Content) })
    @GetMapping("/categories")
    public List<Category> getCategories(
              @RequestParam(required = false) int categoryId) {
        List<Category> categories = new ArrayList<>();
        return categories;
    }

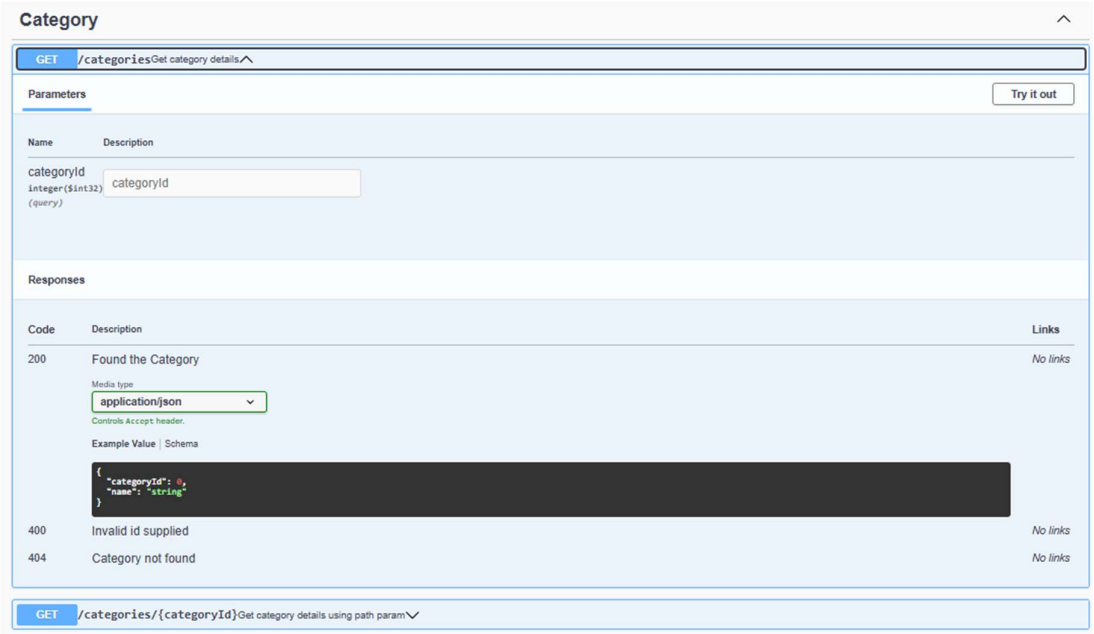
    @Tag(name = "Category")
    @Operation(summary = "Get category details using path param")
    @GetMapping("/categories/{categoryId}")
    public Category getCategoryDetails(@PathVariable int categoryId) {
        Category category = new Category();
        return category;
    }
}
```

Bước 4: Chạy ứng dụng. Vào trình duyệt với URL là http://localhost:8080/swagger-ui/index.html sẽ nhận được kết quả như sau:



Hình Liệt kê danh sách các Endpoint có trong ứng dụng bằng Swagger UI

Click vào các mục dưới Category chúng ta sẽ thấy các thông tin mô tả về các RESTful API mà ta đã viết ở trên



Hình Hiển thị thông tin mô tả một Endpoint

2.2.3. Bài tập làm thêm

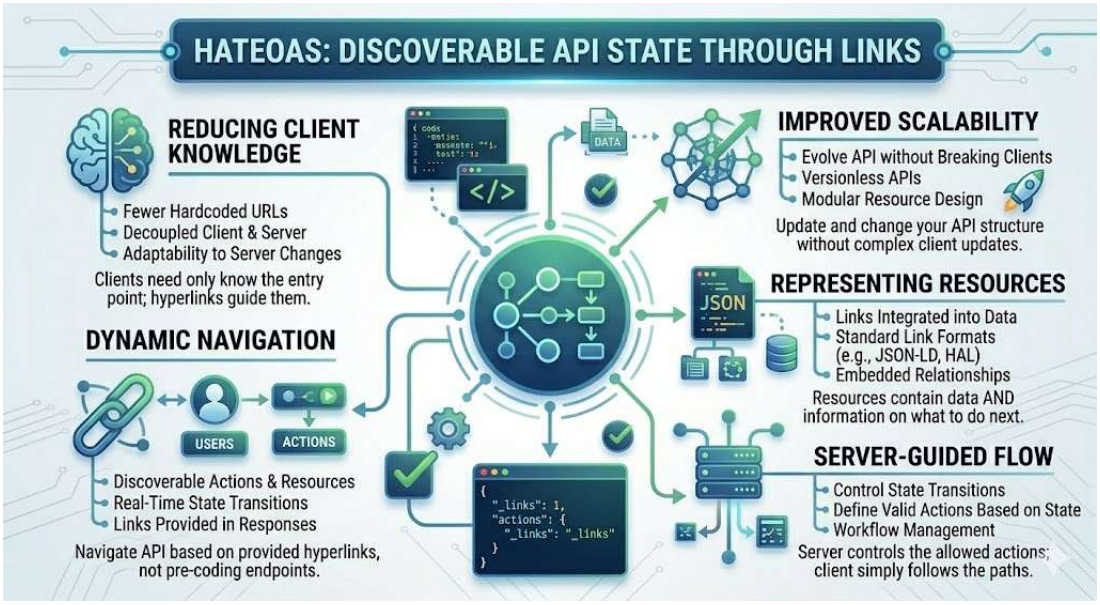
Dự án 2.4. Nâng cấp các dự án RESTful API đã làm bằng cách cấu hình cho phép tích hợp với Swagger UI/Open API. Ví dụ các dự án sau:

- Dự án CRUD với RESTful trên thực thể là bài báo khoa học
- Dự án CRUD với RESTful trên thực thể là nhân viên
- Dự án CRUD với RESTful trên thực thể là sơ yếu lý lịch (CV)

Dự án 2.5. Tạo dự án như trong bài viết https://github.com/essentialprogramming/spring-boot-openapi

Dự án 2.6. Tạo dự án RESTful API cho hệ thống bầu chọn hay bình bầu như trong bài viết https://github.com/JolaPsh/voting-system, sau đó tích hợp với Swagger để viết phần hướng dẫn sử dụng các endpoint của trong ứng dụng

2.3. PHẦN 3: HATEOAS (HYPERMEDIA AS THE ENGINE OF APPLICATION STATE)



Hình Các tính năng chính HATEOAS

2.3.1. Ôn tập lý thuyết với các PROMPT

+ HATEOAS là gì trong kiến trúc RESTful API? Hãy giải thích ý nghĩa của cụm từ 'Hypermedia là động cơ điều khiển trạng thái ứng dụng' bằng một ví dụ thực tế như quy trình đặt hàng online.

+ Hãy so sánh một phản hồi (Response) JSON thông thường với một phản hồi có áp dụng HATEOAS. Tại sao việc thêm các đường dẫn liên kết (_links) lại giúp giảm sự phụ thuộc (decoupling) giữa Frontend và Backend?

+ Trong hệ sinh thái Spring Boot, thư viện Spring HATEOAS hỗ trợ chúng ta như thế nào? Hãy giải thích vai trò của các lớp như EntityModel, CollectionModel và cách sử dụng WebMvcLinkBuilder để tạo liên kết tự động.

+ Định dạng HAL (Hypermedia Application Language) là gì và tại sao nó thường được sử dụng mặc định cùng với HATEOAS trong các dự án Spring? Hãy trình bày cấu trúc của một file JSON chuẩn HAL.

+ Mặc dù HATEOAS rất mạnh mẽ, nhưng tại sao không phải dự án nào cũng sử dụng nó? Hãy phân tích các ưu điểm (khả năng khám phá, tính linh hoạt) và nhược điểm (độ phức tạp, kích thước dữ liệu) để xác định khi nào một sinh viên nên áp dụng HATEOAS vào dự án của mình.

2.3.2. Bài làm mẫu

Bài 1: Thêm chức năng HATEOAS vào RESTFul API

Bước 1: Sử dụng mã nguồn từ dự án thực hiện ở Chương 2 – Phần 2 – Bài 1

Bước 2: Cập nhật tập tin pom.xml bằng cách thêm phụ thuộc vào dự án như hình bên dưới

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-hateoas</artifactId>
</dependency>
```

Chú ý: nếu các thư viện ko tương thích sẽ gặp lỗi khi chạy chức năng này

Bước 3: Cập nhật mã cho các lớp model / dto liên quan đến RESTful API muốn trả về dữ liệu đó. Trong dự án này thì chúng ta cập nhật lớp Player

```
public class Player extends RepresentationModel<Player> {
    private String id;
    private int jerseyNumber;
    private String name;
    private String position;
    private LocalDate dateOfBirth;

    public Player(String id, int jerseyNumber,
             String name, String position, LocalDate dateOfBirth) {
        this.id = id;
        this.jerseyNumber = jerseyNumber;
        this.name = name;
        this.position = position;
        this.dateOfBirth = dateOfBirth;
    }

    //getter/setter
}
```

Bước 4: Cập nhật mã ở các lớp tầng endpoints

```
@RequestMapping("/players")
@RestController
public class PlayerController {

    private FootballService footballService;

    public PlayerController(FootballService footballService) {
        this.footballService = footballService;
    }

    // 1. Single Resource Representation (EntityModel)
    @GetMapping("/{id}")
    public EntityModel<Player> readPlayer(@PathVariable String id) {
        Player player = footballService.getPlayer(id);

        // Build self link: /players/{id}
        player.add(linkTo(methodOn(PlayerController.class)
                    .readPlayer(id)).withSelfRel());

        // Build an optional link to see all players: /players
        player.add(linkTo(methodOn(PlayerController.class)
                    .listPlayers()).withRel("all-players"));

        return EntityModel.of(player);
    }

    // 2. Collection Resource Representation (CollectionModel)
    @GetMapping
    public CollectionModel<EntityModel<Player>> listPlayers() {
        List<Player> players = footballService.listPlayers();

        // Map individual list items to add self-links
        List<EntityModel<Player>> playerModels = players.stream()
                .map(product -> EntityModel.of(player,
                        linkTo(methodOn(PlayerController.class)
                             .readPlayer(player.getId())).withSelfRel()))
                .collect(Collectors.toList());

        return CollectionModel.of(playerModels,
                linkTo(methodOn(PlayerController.class)
```

```
                        .listPlayers()).withSelfRel());
    }

    // Các hàm còn lại không thay đổi
    // …
}
```

Bước 5: Chạy ứng dụng và thử gọi các Endpoints sau:

- Với endpoint: http://localhost:8080/players Method: GET. Kết quả nhận được như hình sau



- Với endpoint http://localhost:8080/players/325636 Method: GET. Kết quả nhận được như hình sau



2.3.3. Bài tập làm thêm

Dự án 2.7. Tạo dự án RESTful API có hỗ trợ tự khám phá các API thông qua HATEOAS như trong bài viết https://www.baeldung.com/rest-api-discoverability-with-spring

Dự án 2.8. Tạo dự án RESTful API có hỗ trợ HATEOAS như trong bài viết https://pradeepl.com/blog/rest/hateoas

***Dự án 2.9.** Nâng cấp các dự án RESTful API đã phát triển bằng cách tích hợp HATEOAS vào dự án. Một số dự án như

- Dự án các bài báo khoa học

- Dự án các nhân viên

- Dự án các sơ yếu lý lịch

Chú ý: Đọc thêm các mức độ trưởng thành của việc phát triển RESTful API tại bài viết *https://blog.restcase.com/4-maturity-levels-of-rest-api-design/*



Hình Bốn mức độ trưởng thành của RESTful API

***Dự án 2.10**. Tạo dự án có trong bài viết *https://spring.io/guides/tutorials/rest*. Trong dự án này có rất nhiều kỹ thuật sẽ được trình bày ở các chương tiếp theo.

D. TÓM TẮT CHƯƠNG

Chương 2 đã dẫn dắt người học đi sâu vào RESTful API, chuẩn kiến trúc cốt lõi trong phát triển ứng dụng phân tán hiện đại. Sinh viên được làm quen với việc sử dụng các phương thức HTTP để quản lý tài nguyên, cũng như cách Spring Boot chuyển hóa dữ liệu linh hoạt giữa Java Object và JSON. Để chuyên nghiệp hóa sản phẩm, nội dung về Swagger (SpringDoc OpenAPI) đã được tích hợp nhằm tự động hóa việc xây dựng tài liệu tương tác, giúp việc kết nối giữa Backend và Frontend trở nên minh bạch và chuẩn xác. Đặc biệt, khái niệm HATEOAS đã mở ra tư duy về một API tự điều hướng, nâng cấp ứng dụng lên mức độ trưởng thành cao nhất theo mô hình Richardson. Với công cụ này trong tay, sinh viên hoàn toàn có thể tự tin xây dựng những dịch vụ web đạt chuẩn.

CHƯƠNG 3 VALIDATE DỮ LIỆU - ENUM VÀ RESPONSE

A. MỤC TIÊU CHƯƠNG

Hỗ trợ việc luyện tập và nghiên cứu nội dung Chương 4 có trong đề cương học phần Chuyên đề J2EE (841468). Chương 3 tập trung trang bị cho sinh viên kỹ năng kiểm soát dữ liệu đầu vào và linh hoạt phương thức phản hồi thông tin trong các ứng dụng Spring Boot. Sinh viên sẽ nắm vững cách sử dụng Bean Validation để thiết lập "hàng rào" bảo mật, đảm bảo dữ liệu đầu vào luôn chính xác và an toàn trước khi xử lý. Song song đó, chương này hướng dẫn cách trả về kết quả dưới dạng JSON hoặc render giao diện với Thymeleaf Template Engine. Việc kết hợp giữa kiểm soát dữ liệu đầu vào và đa dạng đầu ra sẽ giúp sinh viên xây dựng được những ứng dụng chuyên nghiệp.

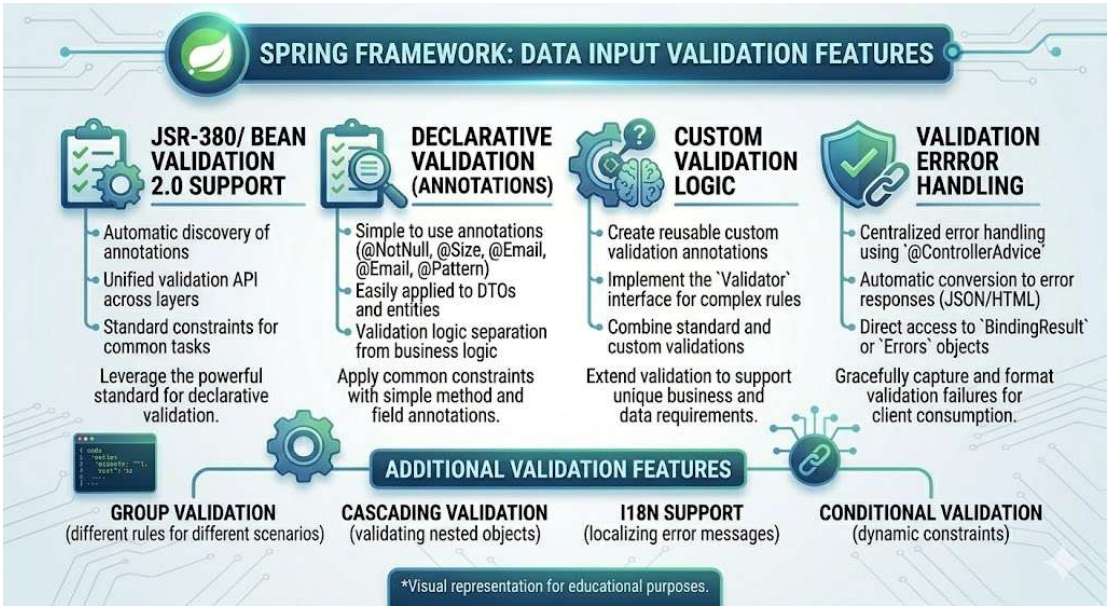
B. KẾT CẤU CHƯƠNG

Chương bao gồm 2 phần là

- Phần 1: Validate dữ liệu - Enum
- Phần 2: Response

C. NỘI DUNG CHƯƠNG

3.1. PHẦN 1: VALIDATE DỮ LIỆU



Hình Các tính năng kiểm tra dữ liệu có trong Spring Boot

3.1.1. Ôn tập lý thuyết với các PROMPT

+ Bean Validation (JSR 380) là gì và tại sao chúng ta nên sử dụng nó thay vì kiểm tra dữ liệu bằng các câu lệnh if-else truyền thống? Hãy liệt kê các Annotation phổ biến như @NotNull, @Size, @Min, @Max và @Email kèm theo ví dụ áp dụng trên một class DTO.

+ Làm thế nào để kích hoạt việc kiểm tra dữ liệu trong Spring Boot Controller bằng cách sử dụng Annotation @Valid hoặc @Validated? Khi dữ liệu đầu vào không hợp lệ, Spring Boot sẽ ném ra ngoại lệ nào và làm sao để chúng ta bắt được thông báo lỗi đó?

+ Enum trong Java là gì và tại sao nên sử dụng Enum thay vì dùng String hay Integer để lưu trữ các trạng thái (như OrderStatus, UserRole) trong dự án Spring Boot? Hãy phân tích lợi ích về mặt an toàn kiểu dữ liệu (Type-safety) và dễ bảo trì mã nguồn.

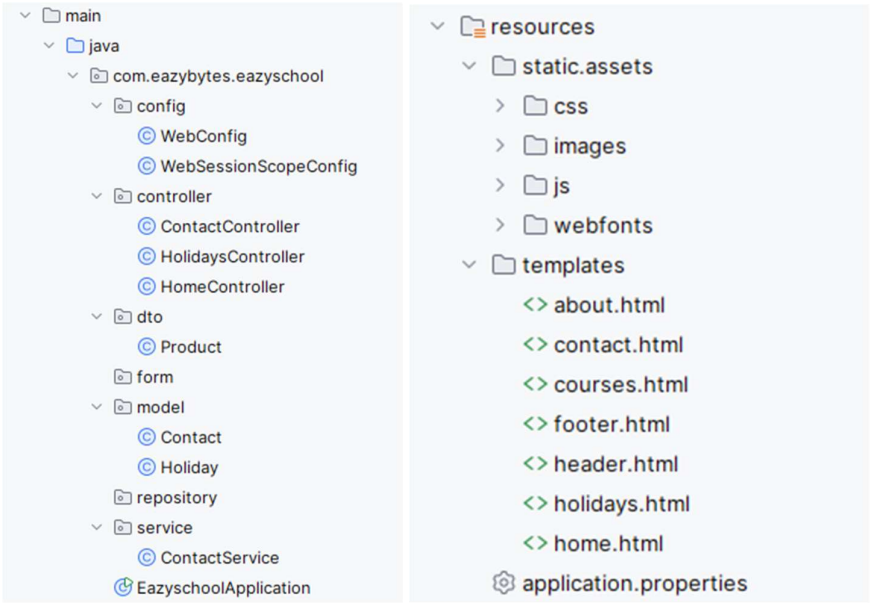
+ Làm thế nào để viết một Custom Validator trong Spring Boot để kiểm tra xem một giá trị String từ người dùng gửi lên có khớp với các giá trị được định nghĩa trong một Enum hay không? Hãy hướng dẫn cách tạo Annotation @EnumValidator tùy chỉnh.

+ Khi làm việc với RESTful API, làm thế nào để cấu hình Jackson để tự động chuyển đổi giữa String từ JSON và kiểu Enum trong Java? Hãy giải thích cách sử dụng @JsonValue và @JsonCreator để tùy biến quá trình chuyển đổi này.

3.1.2. Bài tập mẫu

Bài 1: **Dự án hướng dẫn cách kiểm tra tính hợp lệ của dữ liệu người dùng bằng Spring**

Bước 1: Tải dự án 3_1_1_ex_01_blank.zip có cấu hình và một số tài nguyên cơ bản như sau



Bước 2: Thêm thư viện spring-boot-starter-validation hỗ trợ kiểm tra dữ liệu, thư viện lombok hỗ trợ xây dựng các bean dữ liệu nhanh vào dự án bằng cách cập nhật phụ thuộc có trong tập tin pom.xml

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.36</version>
</dependency>
```

Bước 3: Viết mã khai báo các ràng buộc trên dữ liệu cần kiểm tra ở tầng dữ liệu. Trong dự án này ta muốn kiểm tra các ràng buộc trong thực thể Contact.

```
@Data
public class Contact {
    @NotBlank(message="Name must not be blank")
    @Size(min=3, message="Name must be at least 3 characters long")
    private String name;

    @NotBlank(message="Mobile number must not be blank")
    @Pattern(regexp="(^$|[0-9]{10})",message = "Mobile number must be 10 digits")
    private String mobileNum;

    @NotBlank(message="Email must not be blank")
    @Email(message = "Please provide a valid email address" )
    private String email;

    @NotBlank(message="Subject must not be blank")
    @Size(min=5, message="Subject must be at least 5 characters long")
    private String subject;

    @NotBlank(message="Message must not be blank")
    @Size(min=10, message="Message must be at least 10 characters long")
    private String message;
}
```

Chú ý: đoạn mã trên có sử dụng Lombok được hướng dẫn chi tiết ở Chương 5.

Bước 4: Viết mã thực hiện việc kiểm tra dữ liệu trong Contact có tuân thủ các ràng buộc hay không. Nếu không tuân thủ thì trả về danh sách các ràng buộc bị vi phạm (hay lỗi dữ liệu). Việc kiểm tra này thường được thực hiện ở tầng ứng dụng để đảm bảo dữ liệu chính xác trước khi chuyển đến tầng dịch vụ. Trong ứng dụng này ta sẽ thực hiện việc kiểm tra đó tại tầng ứng dụng cụ thể là lớp ContactController.

```
@Slf4j
@Controller
public class ContactController {

    private final ContactService contactService;
    @Resource(name = "shoppingCart")
    private List<Product> products;

    @Autowired
    public ContactController(ContactService contactService) {
        this.contactService = contactService;
    }

    @RequestMapping("/contact")
    public String displayContactPage(Model model) {
        model.addAttribute("contact", new Contact());
        model.addAttribute("prods", products);

        return "contact.html";
    }

    @RequestMapping(value = "/saveMsg",method = POST) //Cách 1
    //@PostMapping(value = "/saveMsg") @Valid //Cách 2
    public String saveMessage(@ModelAttribute("contact") Contact contact,
                              Errors errors) {
        if(errors.hasErrors()){
            log.error("Contact form validation failed due to : " +
                                    errors.toString());
            return "contact.html";
        }

        contactService.saveMessageDetails(contact);
```

```
        contactService.setCounter(contactService.getCounter()+1);

        log.info("Number of times the Contact form is submitted : "
                + contactService.getCounter());

        return "redirect:/contact";
    }
}
```

Chú ý: trong đoạn mã trên có sử dụng Logging. Kỹ thuật này được trình bày chi tiết ở 4.

Bước 5: Các lỗi (vi phạm ràng buộc) được tìm thấy có thể gửi thẳng đến view cụ thể ở đây gửi thẳng đến các trang viết bằng Thymeleaf. Từ trang này ta có thể trình bày lỗi với nhiều cách khác nhau tùy vào phần thiết kế thông báo lỗi của ứng dụng. Sinh viên có thể tham khảo phần thông báo lỗi này tại trang contact.html có nội dung như sau:

```
…
<ul>
    <li class="alert alert-danger" role="alert"
        th:each="error : ${#fields.errors('contact.*')}" th:text="${error}"
    />
</ul>
…
```

Bước 6: Chạy dự án, mở trình duyệt, đến trang contact, nhấn thêm 1 contact mới và thử nhập contact với một số chỗ vi phạm ràng buộc xem thế nào.

3.1.3. Bài tập làm thêm

Dự án 3.1. Tạo dự án như trong bài viết https://dev.to/gianfcop98/spring-boot-and-validation-a-complete-guide-with-valid-and-validated-471p

Dự án 3.2. Tạo dự án như trong bài viết https://www.abhijits.com/spring%20boot/spring-boot-validation

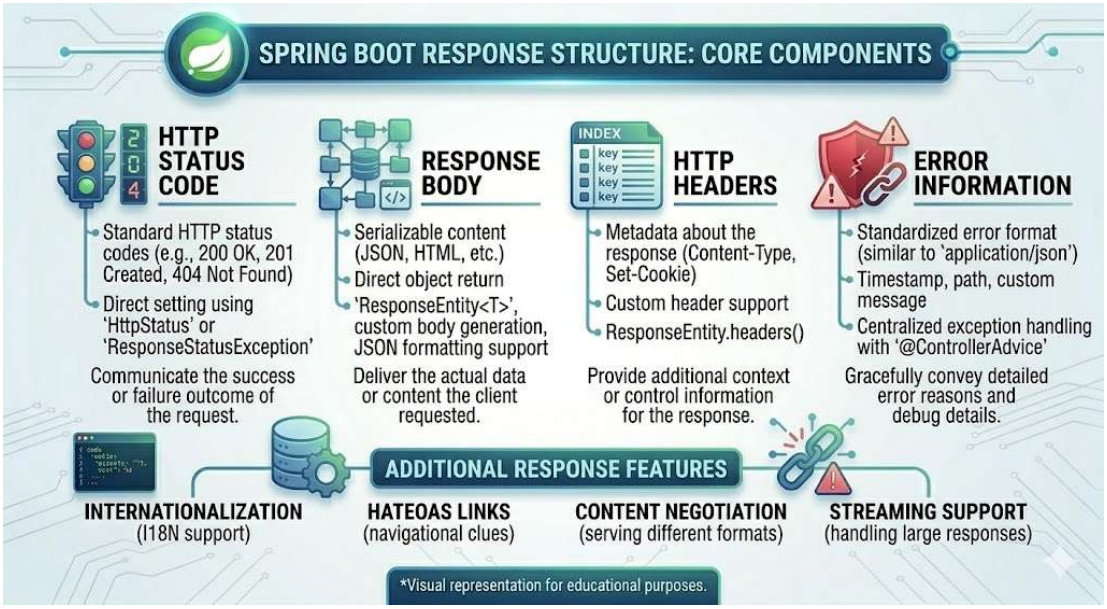
Dự án 3.3. Nâng cấp các dự án RESTful API đã phát triển bằng cách thêm các chức năng kiểm tra dữ liệu đầu vào. Một số dự án như

- Dự án các bài báo khoa học
- Dự án các nhân viên
- Dự án các sơ yếu lý lịch

Dự án 3.4. Tạo dự án cho đồ án cuối kỳ với các hỗ trợ sau

- Hỗ trợ phát triển RESTful API
- Hỗ trợ SWAGGER / OpenAPI để đặc tả chức năng của các endpoint
- Thiết kế các thực thể dữ liệu cho dự án (chưa tích hợp phần xử lý JPA)
- Cấu hình phần kiểm tra tính hợp lệ của dữ liệu có trên các thực thể
- Tầng dữ liệu tạm thời lưu vào bộ nhớ thông qua các lớp Collection
- Hỗ trợ HATEOAS cho các endpoint quan trọng

3.2. PHẦN 2: RESPONSE



Hình Hỗ trợ các Response có trong Spring Boot

3.2.1. Ôn tập lý thuyết với các PROMPT

+ Trong Spring Boot, ResponseEntity là gì và tại sao nó lại linh hoạt hơn việc chỉ trả về một Object thông thường? Hãy hướng dẫn cách sử dụng nó để tùy chỉnh mã trạng thái (HTTP Status Code), Header và Body của một phản hồi API.

+ Tại sao các dự án thực tế thường xây dựng một lớp 'BaseResponse' dùng chung thay vì trả về dữ liệu thô? Hãy gợi ý một cấu trúc JSON chuẩn bao gồm các trường: timestamp, status, message và data, kèm theo cách triển khai trong Spring.

+ Khi nào chúng ta nên sử dụng Thymeleaf thay vì trả về JSON? Hãy giải thích cách @Controller (không phải @RestController) phối hợp với Thymeleaf để tìm tệp tin .html trong thư mục resources/templates và truyền dữ liệu qua lớp Model.

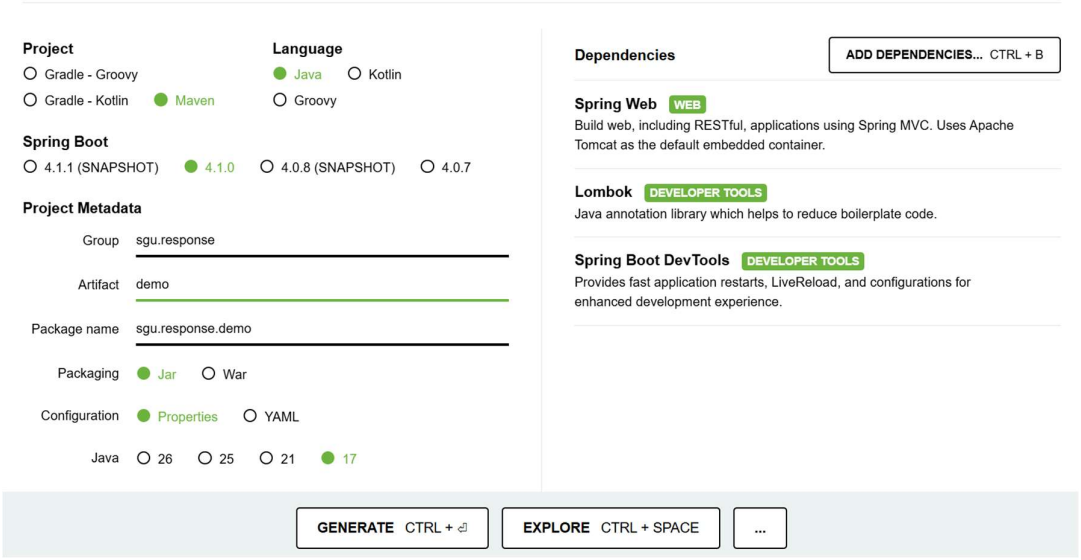
+ Trong Thymeleaf, các cú pháp th:text, th:if, th:each và th:object được sử dụng như thế nào để hiển thị dữ liệu từ Backend lên giao diện người dùng? Hãy cho một ví dụ hiển thị danh sách sinh viên cùng với thông báo lỗi nếu dữ liệu không hợp lệ.

+ Làm thế nào để kiểm soát các trường dữ liệu trả về trong JSON (ví dụ: đổi tên trường bằng @JsonProperty, ẩn trường nhạy cảm như mật khẩu bằng @JsonIgnore, hoặc định dạng ngày tháng với @JsonFormat)?

3.2.2. Bài làm mẫu

Bài 1: Ứng dụng làm quen với ResponseEntity

Bước 1: Tạo dự án từ Spring Boot từ trang https://start.spring.io/ với các cấu hình như sau



Bước 2: Tạo Controller đặt tên là ExampleController

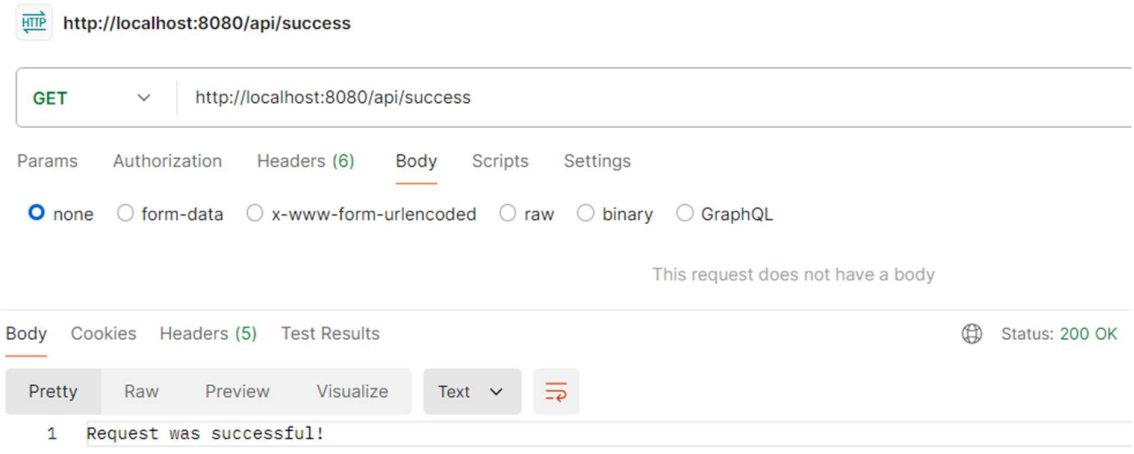
```java
@RestController
@RequestMapping("/api")
public class ExampleController {
//    HTTP Status Code
//    200 OK: Request processed successfully
//    201 Created: A new resource has been created
//    204 No Content: Request successful, no response body
//    400 Bad Request: Invalid or malformed request
//    404 Not Found: Requested resource not found
//    500 Internal Server Error: Server failed to process the request

    @GetMapping("/success")
    public ResponseEntity<String> successResponse() {
        String responseBody = "Request was successful!";
        return new ResponseEntity<>(responseBody, HttpStatus.OK);
    }

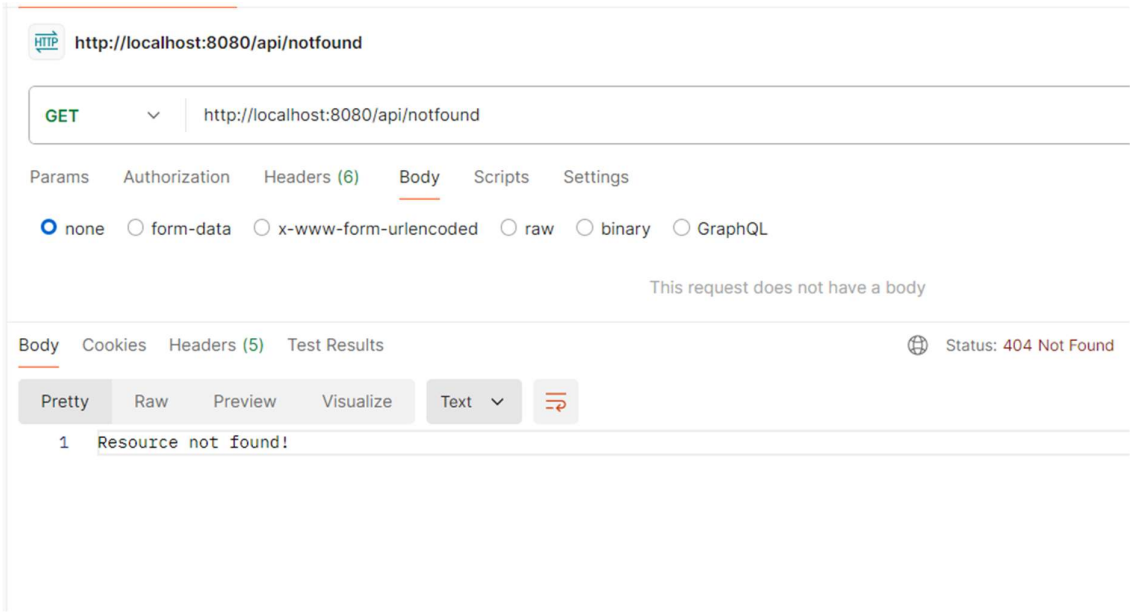
    @GetMapping("/notfound")
    public ResponseEntity<String> notFoundResponse() {
        String responseBody = "Resource not found!";
        return new ResponseEntity<>(responseBody, HttpStatus.NOT_FOUND);
    }

    @GetMapping("/customheader")
    public ResponseEntity<String> customHeaderResponse() {
        String responseBody = "Response with custom header!";
        return ResponseEntity
                .status(HttpStatus.OK)
                .header("Custom-Header", "CustomValue")
                .body(responseBody);
    }
}
```

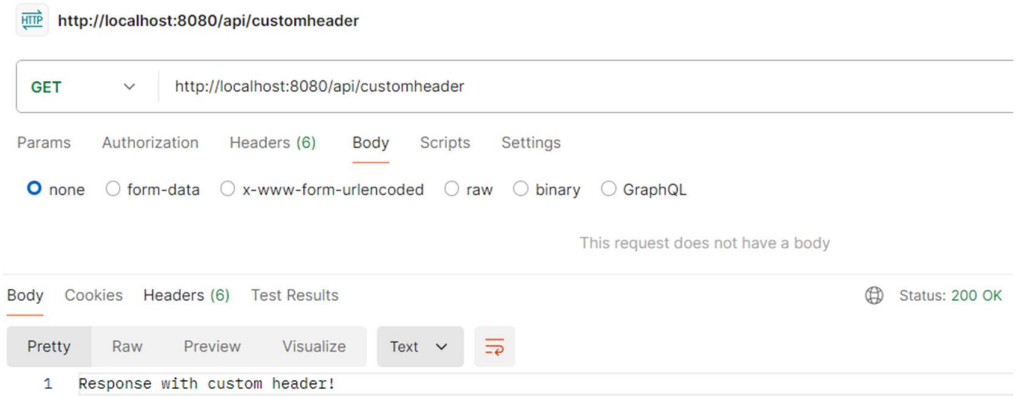
Bước 3: Kiểm tra endpoint /api/success bằng ứng dụng Postman



Bước 4: Kiểm tra endpoint /api/notfound bằng ứng dụng Postman

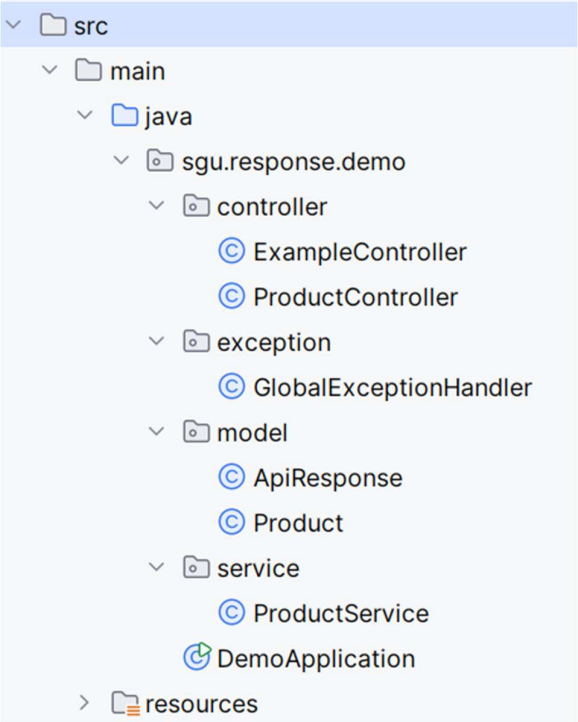


Bước 5: Kiểm tra endpoint /api/customheader bằng ứng dụng Postman



Bài 2: https://www.javacodegeeks.com/2024/12/handling-api-responses-in-spring-boot-best-practices-and-real-life-examples.html

Bước 1. Phát triển tiếp Bài 1 và cập nhật cấu trúc dự án như hình bên dưới



Bước 2. Viết mã cho lớp ApiResponse

```
@Data
@AllArgsConstructor
public class ApiResponse<T> {
    private boolean ok;
    private String message;
    private T data;
}
```

Bước 3. Viết mã cho endpoint liên quan đến thực thể Product.

```
@Data
@AllArgsConstructor
public class Product {
    private long id;
    private String name;
}

@Service //Dummy code
public class ProductService {
    public Product findById(Long id) {
        if (id > 10 || id < 0)
            return null;
        else
            return new Product(id, "Name " + id);
    }
}

@RestController
@RequestMapping("/api")
public class ProductController {
    @Autowired
    private ProductService productService;
```

```
    @GetMapping("/products/{id}")
    public ResponseEntity<ApiResponse<Product>> getProduct
                                    (@PathVariable Long id) {
        Product product = productService.findById(id);
        if (product == null) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND)
                    .body(new ApiResponse<Product>(false,
                            "Product not found", null));
        }
        return ResponseEntity.ok(new ApiResponse<Product>(true,
                "Product retrieved successfully", product));
    }

    @GetMapping("/products/delete/{id}")
    public ResponseEntity<ApiResponse<Product>> deleteProduct
                                    (@PathVariable Long id)
    {
        throw new RuntimeException("NOT SUPPORT");
    }
}
```

Bước 4. Viết mã cho xử lý ngoại lệ toàn cục trong dự án

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiResponse<Void>> handleGenericException(Exception ex) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                .body(new ApiResponse<>(false,
                        "An unexpected error occurred", null));
    }
}
```

Bước 5. Kiểm tra endpoint products/1 – trả về kết quả hợp lệ

Bước 6. Kiểm tra endpoint products/20 – trả về kết quả không hợp lệ

Bước 7. Kiểm tra endpoint products/delete/1 – trả về ngoại lệ.

3.2.3. Bài tập làm thêm

Dự án 3.5. Hoàn thiện phần xử lý lỗi và xử lý khi truy cập 1 endpoint không hợp lệ bằng cách sử dụng ApiResponse

D. TÓM TẮT CHƯƠNG

Chương này đã trang bị cho người học cách xây dựng một lớp giao tiếp dữ liệu an toàn và linh hoạt trong Spring Boot. Thông qua việc làm chủ Bean Validation, sinh viên đã biết cách thiết lập hàng rào bảo mật ngay tại cửa ngõ đầu vào, kết hợp cùng Enum để chuẩn hóa các kiểu dữ liệu đặc thù và loại bỏ các sai sót logic không đáng có. Phần tiếp theo của chương đã mở ra cái nhìn toàn diện về kỹ thuật phản hồi dữ liệu, từ việc sử dụng ResponseEntity để cấu trúc hóa các JSON Response chuyên nghiệp cho đến việc vận dụng Thymeleaf để render các giao diện HTML động. Sự kết hợp giữa kiểm soát đầu vào chặt chẽ và đa dạng hóa phương thức đầu ra chính là chìa khóa giúp ứng dụng trở nên tin cậy và thân thiện hơn với người dùng cuối.

CHƯƠNG 4: XỬ LÝ NGOẠI LỆ

A. MỤC TIÊU CHƯƠNG

Hỗ trợ việc luyện tập và nghiên cứu nội dung Chương 5 có trong đề cương học phần Chuyên đề J2EE (841468). Chương này giúp xây dựng một hệ thống phần mềm có khả năng phục hồi và ổn định cao thông qua cơ chế quản lý sai sót tập trung. Mục đích của chương này là giúp sinh viên thoát khỏi cách xử lý lỗi rời rạc bằng try-catch để tiến tới thiết lập một "bộ lọc ngoại lệ toàn cục" (Global Exception Handling) chuyên nghiệp. Bên cạnh đó, chương này còn chú trọng vào kỹ năng sử dụng Logging (như Logback hoặc SLF4J) để ghi lại dấu vết hệ thống, giúp nhà phát triển nhanh chóng khoanh vùng và khắc phục lỗi trong môi trường thực tế.

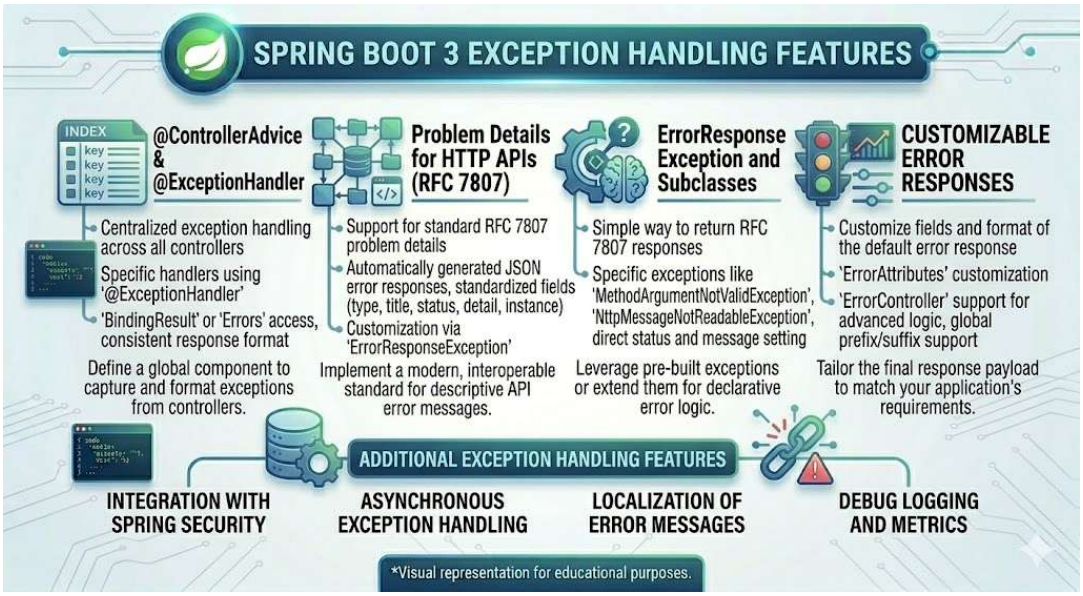
B. KẾT CẤU CHƯƠNG

Chương bao gồm 2 phần

- Phần 1: Xử lý ngoại lệ với @ControllerAdviced và @ExceptionHandler
- Phần 2: Xử lý Logging

C. NỘI DUNG CHƯƠNG

4.1. PHẦN 1: XỬ LÝ NGOẠI LỆ VỚI @CONTROLLERADVICED VÀ @EXCEPTIONHANDLER



Hình Tổng quan về xử lý ngoại lệ trong Spring

4.1.1. Ôn tập lý thuyết với các PROMPT

+ Tại sao trong các dự án chuyên nghiệp, người ta lại ưu tiên sử dụng cơ chế xử lý ngoại lệ tập trung (Global Exception Handling) thay vì dùng các khối try-catch phân tán ở khắp mọi nơi? Hãy giải thích lợi ích về mặt bảo trì và tính nhất quán của mã nguồn.

+ @ControllerAdvice và @RestControllerAdvice là gì? Hãy giải thích cách chúng hoạt động như một

'Interceptors' (bộ chặn) để bắt các ngoại lệ được ném ra từ bất kỳ Controller nào trong hệ thống.

+ Làm thế nào để sử dụng @ExceptionHandler bên trong một class @ControllerAdvice để xử lý một ngoại lệ cụ thể (ví dụ: ResourceNotFoundException)? Hãy viết một đoạn code mẫu trả về một đối tượng ErrorResponse kèm theo mã lỗi HTTP 404.

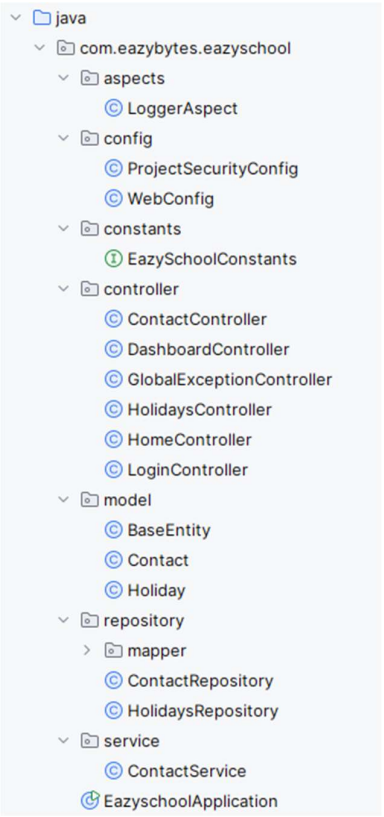
+ Làm thế nào để cấu hình xử lý ngoại lệ sao cho hệ thống tự động trả về JSON cho các yêu cầu API nhưng lại chuyển hướng người dùng đến trang error.html (Thymeleaf) nếu đó là một yêu cầu từ trình duyệt? Hãy hướng dẫn cách sử dụng ModelAndView trong xử lý ngoại lệ.

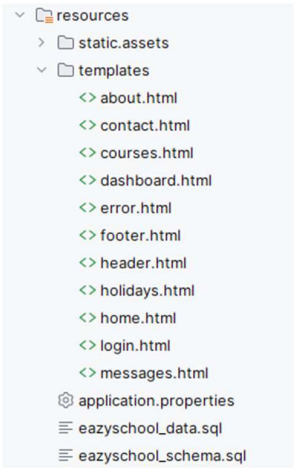
+ Khi người dùng gửi dữ liệu không hợp lệ (vi phạm các ràng buộc ở Chương 3), Spring Boot sẽ ném ra MethodArgumentNotValidException. Hãy hướng dẫn cách viết một phương thức xử lý lỗi này để trả về một danh sách các thông báo lỗi chi tiết cho từng trường dữ liệu (field errors).

4.1.2. Bài làm mẫu

Bài 1: Xử lý ngoại lệ trong dự án Spring

Bước 1: Tải dự án 4_1_1_ex_01_blank.zip có cấu hình và một số tài nguyên cơ bản như sau





Bước 2: Chuẩn bị cơ sở dữ liệu cho dự án

- Hệ quản trị sử dụng cho dự án là MySQL. Hướng dẫn tải và cài đặt tại https://www.mysql.com/products/community/

- Cài đặt ứng dụng kết nối làm việc với MySQL là MySQL Workbench. Hướng dẫn tải và cài đặt tại https://www.mysql.com/products/workbench/

- Tạo cơ sở dữ liệu mang tên là javaee5

- Tạo cấu trúc và dữ liệu ban đầu cho dự án

#schema.sql

```
CREATE TABLE IF NOT EXISTS `contact msg` (
  `contact_id` int AUTO_INCREMENT  PRIMARY KEY,
  `name` varchar(100) NOT NULL,
  `mobile_num` varchar(10) NOT NULL,
  `email` varchar(100) NOT NULL,
  `subject` varchar(100) NOT NULL,
  `message` varchar(500) NOT NULL,
  `status` varchar(10) NOT NULL,
  `created_at` TIMESTAMP NOT NULL,
  `created_by` varchar(50) NOT NULL,
  `updated_at` TIMESTAMP DEFAULT NULL,
  `updated_by` varchar(50) DEFAULT NULL
);

CREATE TABLE IF NOT EXISTS `holidays` (
  `day` varchar(20) NOT NULL,
  `reason` varchar(100) NOT NULL,
  `type` varchar(20) NOT NULL,
  `created_at` TIMESTAMP NOT NULL,
  `created_by` varchar(50) NOT NULL,
  `updated_at` TIMESTAMP DEFAULT NULL,
  `updated_by` varchar(50) DEFAULT NULL
);
```

#data.sql

```
INSERT INTO `holidays` (`day`,`reason`,`type`,`created_at`, `created_by`)
 VALUES (' Jan 1 ','New Year''s Day','FESTIVAL',CURDATE(),'DBA');

INSERT INTO `holidays` (`day`,`reason`,`type`,`created_at`, `created_by`)
 VALUES (' Oct 31 ','Halloween','FESTIVAL',CURDATE(),'DBA');

INSERT INTO `holidays` (`day`,`reason`,`type`,`created_at`, `created_by`)
 VALUES (' Nov 24 ','Thanksgiving Day','FESTIVAL',CURDATE(),'DBA');

INSERT INTO `holidays` (`day`,`reason`,`type`,`created_at`, `created_by`)
 VALUES (' Dec 25 ','Christmas','FESTIVAL',CURDATE(),'DBA');

INSERT INTO `holidays` (`day`,`reason`,`type`,`created_at`, `created_by`)
 VALUES (' Jan 17 ','Martin Luther King Jr. Day','FEDERAL',CURDATE(),'DBA');

INSERT INTO `holidays` (`day`,`reason`,`type`,`created_at`, `created_by`)
 VALUES (' July 4 ','Independence Day','FEDERAL',CURDATE(),'DBA');

INSERT INTO `holidays` (`day`,`reason`,`type`,`created_at`, `created_by`)
 VALUES (' Sep 5 ','Labor Day','FEDERAL',CURDATE(),'DBA');

INSERT INTO `holidays` (`day`,`reason`,`type`,`created_at`, `created_by`)
  VALUES (' Nov 11 ','Veterans Day','FEDERAL',CURDATE(),'DBA');
```

Bước 3: Thêm các thư viện vào dự án bằng cách cập nhật tập tin pom.xml như sau

#pom.xml

```
…
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
  </dependency>
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
  </dependency>

…
```

Bước 4: Khai báo thông tin về cơ sở dữ liệu bằng cách cập nhật tập tin application.properties

```
#application.properties

spring.datasource.url=jdbc:mysql://localhost/javaee5
spring.datasource.username=root
spring.datasource.password=root

server.error.whitelabel.enabled=true
server.error.path=/error
```

Bước 5: Viết mã cho lớp GlobalExceptionController để khai báo các xử lý toàn cục cho toàn bộ ứng dụng. Đoạn mã này sử dụng chức năng AOP của Spring.

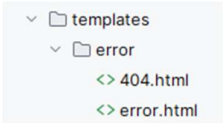
```
@Slf4j
@ControllerAdvice
public class GlobalExceptionController {

    @ExceptionHandler(Exception.class)
    public ModelAndView exceptionHandler(Exception exception){
        ModelAndView errorPage = new ModelAndView();
        errorPage.setViewName("error");
        errorPage.addObject("errormsg", exception.getMessage());
        return errorPage;
    }
}
```

Bước 6: Viết code cho màn hình thông báo lỗi tại trang error.html

```
…
<!-- error block -->
<section class="page_404">
    <div class="container">
        <div class="row">
            <div class="col-sm-12 ">
                <div class="col-sm-10 col-sm-offset-1  text-center">
                    <div class="four_zero_four_bg">
                        <h1 class="text-center">oops...</h1>
                    </div>
                    <div class="contant_box_404">
                        <h3 class="h2">
                            Look like you're lost
                        </h3>
                        <p th:text="${'Error Message - ' + errormsg}"></p>
                        <a th:href="@{/home}" class="link_404 btn btn-style btn-
style-3">Go to Home</a>
                    </div>
                </div>
            </div>
        </div>
    </div>
</section>
<!-- //error block -->
…
```

Bước 6: Viết code cho các màn hình lỗi như 404.html



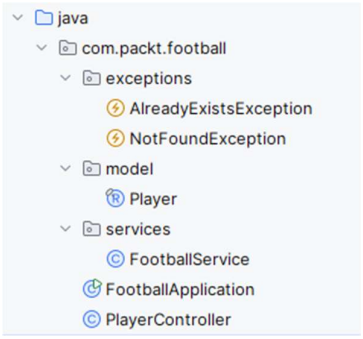

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
```

```
    <title>Spring Boot Recipes - Library - Resource Not Found</title>
</head>
<body>
    <h1>Oops, the page couldn't be located.</h1>
</body>
</html>
```

Bước 7: Chạy dự án và kiểm tra

Bài 2: Xử lý ngoại lệ trên dự án RESTful API

Bước 1: Tạo dự án Spring Boot có cấu trúc như hình bên dưới



Bước 2: Tạo riêng gói exceptions để lưu trữ tất các xử lý ngoại lệ có trong dự án. Viết mã cho các lớp xử lý ngoại lệ như sau:

```
public class AlreadyExistsException extends RuntimeException {
    public AlreadyExistsException(String message) {
        super(message);
    }
}


public class NotFoundException extends RuntimeException {
    public NotFoundException(String message) {
        super(message);
    }
}
```

Bước 3: Viết mã cho các lớp ở tầng dữ liệu và tầng dịch vụ như sau:

```
public record Player(String id, int jerseyNumber, String name, String
                     position, LocalDate dateOfBirth) {
}


@Service
public class FootballService {
    private final Map<String, Player> players = new HashMap<String,
Player>(Map.ofEntries(
            Map.entry("1884823", new Player("1884823", 5, "Ivana ANDRES",
"Defender", LocalDate.of(1994, 07, 13))),
            Map.entry("325636", new Player("325636", 11, "Alexia PUTELLAS",
"Midfielder", LocalDate.of(1994, 02, 04))),
            Map.entry("396930", new Player("396930", 2, "Ona BATLLE", "Defender",
LocalDate.of(1999, 06, 10))),
            Map.entry("377762", new Player("377762", 6, "Aitana BONMATI",
"Midfielder", LocalDate.of(1998, 01, 18))),
            Map.entry("387133", new Player("387133", 4, "Irene PAREDES", "Defender",
LocalDate.of(1998, 01, 18))),
            Map.entry("387134", new Player("387134", 8, "Vicky LOSADA",
"Midfielder", LocalDate.of(1991, 01, 01))),
```

```
            Map.entry("396911", new Player("396911", 15, "Eva NAVARRO", "Forward",
LocalDate.of(2001, 01, 27))),
            Map.entry("415396", new Player("415396", 13, "Enith SALON",
"Goalkeeper", LocalDate.of(2001, 9, 24)))));

    public List<Player> listPlayers() {
        return players.values().stream().collect(Collectors.toList());
    }

    public Player getPlayer(String id) {
        Player player = players.get(id);
        if (player == null)
            throw new NotFoundException("Player not found");
        return player;
    }

    public Player addPlayer(Player player) {
        if (players.containsKey(player.id())) {
            throw new AlreadyExistsException("The player already exists");
        } else {
            players.put(player.id(), player);
            return player;
        }
    }

    public Player updatePlayer(Player player) {
        if (!players.containsKey(player.id())) {
            throw new NotFoundException("The player does not exist");
        } else {
            players.put(player.id(), player);
            return player;
        }
    }

    public void deletePlayer(String id) {
        if (players.containsKey(id)) {
            players.remove(id);
        }
    }
}
```

Bước 4: Viết mã RESTful API có phần xử lý ngoại lệ

```
@RequestMapping("/players")
@RestController
public class PlayerController {

    private FootballService footballService;

    public PlayerController(FootballService footballService) {
        this.footballService = footballService;
    }

    @GetMapping
    public List<Player> listPlayers() {
        return footballService.listPlayers();
    }

    @GetMapping("/{id}")
    public Player readPlayer(@PathVariable String id) {
        return footballService.getPlayer(id);
    }

    @PostMapping
    public void createPlayer(@RequestBody Player player) {
        footballService.addPlayer(player);
    }

    @PutMapping("/{id}")
    public void updatePlayer(@PathVariable String id, @RequestBody Player player) {
```

```
        footballService.updatePlayer(player);
    }

    @DeleteMapping("/{id}")
    public void deletePlayer(@PathVariable String id) {
        footballService.deletePlayer(id);
    }

    @ResponseStatus(value = HttpStatus.NOT_FOUND, reason = "Not found")
    @ExceptionHandler(NotFoundException.class)
    public void notFoundHandler() {
    }

    @ResponseStatus(value = HttpStatus.BAD_REQUEST, reason = "Already exists")
    @ExceptionHandler(AlreadyExistsException.class)
    public void alreadyExistsHandler() {
    }
}
```

4.1.3. Bài tập làm thêm

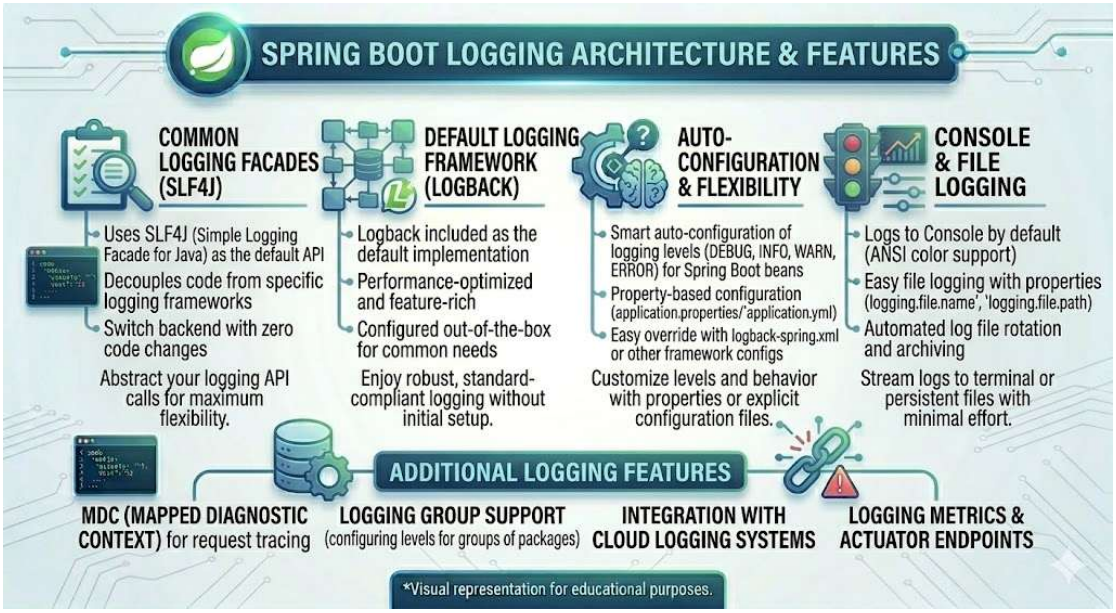
Dự án 4.1. Tạo dự án xử lý ngoại lệ như trong bài viết https://www.javaguides.net/2021/10/spring-boot-exception-handling-example.html

Dự án 4.2. Tạo dự án xử lý ngoại lệ như trong bài viết https://github.com/bezkoder/spring-boot-exception-handling

Dự án 4.3. Tạo dự án xử lý ngoại lệ như trong bài viết https://www.toptal.com/developers/java/spring-boot-rest-api-error-handling

Dự án 4.4. Tạo dự án xử lý ngoại lệ như trong bài viết https://www.baeldung.com/rest-api-error-handling-best-practices

4.2. PHẦN 2: XỬ LÝ LOGGING



Hình Tổng quan xử lý Logging mà Spring hỗ trợ

4.2.1. Ôn tập lý thuyết với các PROMPT

+ Logging là gì và tại sao nó lại quan trọng hơn việc sử dụng System.out.println() trong phát triển ứng dụng chuyên nghiệp? Hãy giải thích ý nghĩa và kịch bản sử dụng cho 5 cấp độ log: TRACE, DEBUG, INFO,

WARN, và ERROR.

+ Làm thế nào để sử dụng Annotation @Slf4j của Lombok để ghi log trong một class Spring Boot? Hãy viết một ví dụ minh họa cách ghi log kèm theo các tham số (placeholder {}) để tối ưu hiệu suất thay vì cộng chuỗi thủ công.

+ Spring Boot sử dụng Logback làm mặc định, vậy làm thế nào để cấu hình tệp logback-spring.xml? Hãy hướng dẫn cách thiết lập để log vừa hiển thị trên Console, vừa được lưu vào tệp tin (File Appender) với cơ chế tự động xoay vòng (Rolling Policy) theo ngày.

+ Khi ứng dụng chạy trên nhiều máy chủ, làm thế nào để thu thập log tập trung? Hãy giải thích mô hình tích hợp Spring Boot với ELK Stack (Elasticsearch, Logstash, Kibana) và cách gửi log trực tiếp từ ứng dụng qua Logstash. (khám phá thêm)

+ Làm thế nào để tích hợp Spring Boot với các hệ thống theo dõi log và chỉ số hiện đại như Prometheus, Grafana hoặc Loki? Hãy giải thích vai trò của thư viện Spring Boot Actuator trong việc cung cấp thông tin vận hành cho các công cụ này.

4.2.2. Bài làm mẫu

Bài 1: Xây dựng chính sách log tập trung cho dự án

Bước 1: Tải dự án 4_2_1_ex_01_blank.zip. Dự án này có sử dụng cơ sở dữ liệu từ hệ quản trị cơ sở dữ liệu MySQL do đó các bạn tiến hành cài đặt như các bước có trong bài 1 ở chương 4.1

Bước 2: Thêm các dòng sau vào tập tin cấu hình tên là application.properties

```
logging.level.org.springframework=INFO
logging.level.com.eazybytes.eazyschool=DEBUG
#logging.path=/var/logs/app.log
logging.file.name=myapp.log
```

Bước 3: Cấu hình log cho dự án tại resources/logback.xml

```
<configuration>

    <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
       <!-- encoders are assigned the type
ch.qos.logback.classic.encoder.PatternLayoutEncoder
          by default -->
       <encoder>
          <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n
          </pattern>
       </encoder>
    </appender>

    <appender name="FILE" class="ch.qos.logback.core.FileAppender">
       <file>app.log</file>

       <encoder>
          <pattern>%date %level [%thread] %logger{10} [%file:%line] %msg%n
          </pattern>
       </encoder>
    </appender>

    <logger name="com.apress" level="DEBUG" additivity="false">
       <appender-ref ref="STDOUT" />
       <appender-ref ref="FILE" />
    </logger>
```

```
    <root level="INFO">
       <appender-ref ref="STDOUT" />
       <appender-ref ref="FILE" />
    </root>
</configuration>
```

Bước 4: Viết mã gắn chức năng log vào các lớp muốn log lại khi có sự cố trong xử lý hoặc để trace hoặc debug. Ví dụ gắn 1 đoạn mã log vào lớp ContactController như bên dưới

```
@Slf4j
@Controller
public class ContactController {
   …
   @RequestMapping(value = "/log4test",method = GET)
   public ModelAndView saveMessage(@RequestParam String name,
                                   @RequestParam String mobileNum,
                                   @RequestParam String email,
                                   @RequestParam String subject,
                                   @RequestParam String message) {
         log.info("Name : " + name);
         log.info("Mobile Number : " + mobileNum);
         log.info("Email Address : " + email);
         log.info("Subject : " + subject);
         log.info("Message : " + message);
         return new ModelAndView("redirect:/contact");
   }

    @RequestMapping(value = "/saveMsg",method = POST)
   public String saveMessage(@Valid @ModelAttribute("contact")
                             Contact contact, Errors errors) {
      if(errors.hasErrors()){
        log.error("Contact form validation failed due to : " + errors.toString());
        return "contact.html";
      }
      contactService.saveMessageDetails(contact);
      return "redirect:/contact";
   }

    …
}
```

Bước 5: Chạy dự án và kiểm tra tập tin log được tạo ra có tên như đã cấu hình là app.log

4.2.3. Bài tập làm thêm

Dự án 4.5. Tạo dự án như trong bài viết https://o7planning.org/11145/spring-email và thêm phần xử lý Logging phù hợp vào trong dự án.

Dự án 4.6. Tạo dự án như trong bài viết https://o7planning.org/11753/use-logging-in-spring-boot

Dự án 4.7. Nâng cấp các dự án RESTful API đã phát triển bằng cách thêm các chức năng hỗ trợ xử lý logging. Một số dự án như

- Dự án các bài báo khoa học

- Dự án các nhân viên

- Dự án các sơ yếu lý lịch

Chú ý: sinh viên có thể gắn phần log vào phần xử lý ngoại lệ, log vào phần cần giám sát khi vận hành hệ thống, log vào phần hỗ trợ debug, …

Dự án 4.8. Nâng cấp dự án đồ án cuối kỳ bằng cách thực hiện các bước sau

- Tích hợp và cấu hình phần logging vào trong dự án

- Viết mã xử lý logging cho phần dữ liệu người dùng nhập không đúng
- Viết mã xử lý logging cho phép theo dõi hệ thống khi vận hành và hỗ trợ debug

D. TÓM TẮT CHƯƠNG

Chương 4 đã giúp sinh viên hoàn thiện "mảnh ghép" cuối cùng về tính bền vững của ứng dụng thông qua việc làm chủ cơ chế xử lý ngoại lệ toàn cục và hệ thống Logging chuyên nghiệp. Thay vì để phần mềm dừng hoạt động đột ngột, người học đã biết cách sử dụng @ControllerAdvice để điều hướng lỗi một cách thông minh, giúp trải nghiệm người dùng luôn mượt mà dù hệ thống gặp sự cố. Việc hiểu rõ các cấp độ Log và cách vận hành Logback cũng trang bị cho các bạn kỹ năng "thám tử" để nhanh chóng truy vết và khắc phục lỗi trong thực tế. Khép lại tư duy về vận hành, chúng ta sẽ bước sang Chương 5 để tối ưu hóa hiệu suất viết mã với Lombok và MapStruct. Đây là chương học giúp các bạn loại bỏ hoàn toàn "mã nguồn lặp lại" (boilerplate code), tự động hóa việc tạo Getter/Setter và chuyển đổi dữ liệu giữa các lớp (DTO mapping) một cách tinh tế và hiệu quả nhất.

CHƯƠNG 5: LOMBOK VÀ MAPSTRUCT

A. MỤC TIÊU CHƯƠNG

Hỗ trợ việc luyện tập và nghiên cứu nội dung Chương 6 có trong đề cương học phần Chuyên đề J2EE (841468). Chương 5 được thiết kế như một "cuộc cách mạng về năng suất," tập trung vào việc tinh gọn mã nguồn và tối ưu hóa quy trình phát triển ứng dụng Spring Boot thông qua hai thư viện bổ trợ mạnh mẽ là Lombok và MapStruct. Mục đích cốt lõi của chương này là giúp sinh viên loại bỏ hoàn toàn các boilerplate code như Getter, Setter, Constructors hay toString bằng các Annotation thông minh của Lombok, giúp lớp Model trở nên trong sáng và dễ bảo trì hơn. Bên cạnh đó, chương này trình bày kỹ thuật chuyển đổi dữ liệu chuyên nghiệp với MapStruct, thay thế việc sao chép thủ công các trường dữ liệu giữa Entity và DTO bằng cơ chế tự động hóa mạnh mẽ, chính xác và có hiệu suất cao.

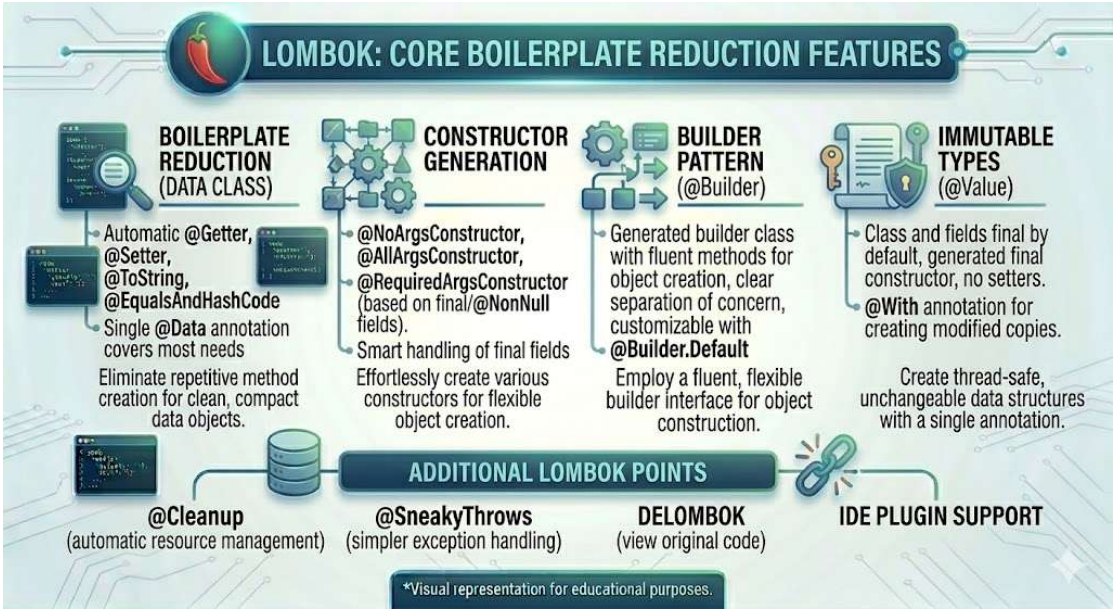
B. KẾT CẤU CHƯƠNG

Chương bao gồm 2 phần

- Phần 1: Sử dụng Lombok
- Phần 2: Sử dụng MapStruct

C. NỘI DUNG CHƯƠNG

5.1. PHẦN 1: SỬ DỤNG LOMBOK



Hình Tổng quan các tính năng của Lombok

5.1.1. Ôn tập lý thuyết với các PROMPT

+ Lombok là gì và nó hoạt động như thế nào trong quá trình biên dịch (compile-time) của Java? Tại sao chúng ta cần cài đặt thêm Plugin Lombok trên IDE (IntelliJ/Eclipse) thì mã nguồn mới không báo lỗi đỏ?

+ Hãy giải thích công dụng và sự khác biệt giữa các Annotation: @Getter, @Setter, @ToString, @EqualsAndHashCode và @RequiredArgsConstructor. Tại sao Annotation @Data lại được coi là 'tất cả trong một' cho một lớp POJO?

+ Design Pattern Builder là gì và Annotation @Builder của Lombok giúp chúng ta tạo ra các đối tượng phức tạp một cách linh hoạt như thế nào? Hãy cho một ví dụ so sánh giữa việc dùng Constructor thông thường và dùng Builder.

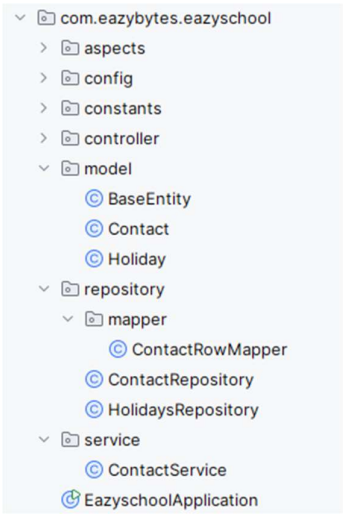
+ Phân biệt cách sử dụng @NoArgsConstructor, @AllArgsConstructor và @RequiredArgsConstructor. Trong một dự án Spring Boot, tại sao @RequiredArgsConstructor thường được dùng để thay thế @Autowired cho việc Dependency Injection?

+ Tôi có một lớp Entity Java truyền thống với đầy đủ Getter, Setter, Constructors (khoảng 100 dòng code). Hãy hướng dẫn tôi cách thay thế toàn bộ chúng bằng các Annotation của Lombok để rút gọn xuống còn dưới 20 dòng mà vẫn giữ nguyên tính năng.

5.1.2. Bài làm mẫu

Bài 1: Áp dụng lombok vào trong dự án Spring.

Bước 1: Tải dự án 5_1_1_ex_01_blank.zip. Dự án này có sử dụng cơ sở dữ liệu từ hệ quản trị cơ sở dữ liệu MySQL do đó các bạn tiến hành cài đặt như các bước có trong bài 1 ở chương 4.1. Cấu trúc của dự án như sau:



Bước 2: Thêm lombok vào trong dự án bằng cách thêm dependency lombok vào trong dự án

```
...
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.36</version>
</dependency>

//làm việc với spring-jdbc
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
...
```

Bước 3: Lombok thường hỗ trợ tạo các bean dữ liệu nhanh. Trong dự án này nó giúp tạo các bean dữ liệu như lớp BaseEntity, lớp Holiday và lớp Contact

```
//Annotation của lOmbok, tham khảo thêm một số annotation của lombok tại
https://projectlombok.org/
@Data
public class BaseEntity {

    private LocalDateTime createdAt;
    private String createdBy;
    private LocalDateTime updatedAt;
    private String updatedBy;
}


@Data
public class Holiday extends BaseEntity {

    private String day;
    private String reason;
    private Type type;

    public enum Type {
        FESTIVAL, FEDERAL
    }
}


@Data
public class Contact extends BaseEntity{

    private int contactId;

    @NotBlank(message="Name must not be blank")
    @Size(min=3, message="Name must be at least 3 characters long")
    private String name;

    @NotBlank(message="Mobile number must not be blank")
    @Pattern(regexp="(^$|[0-9]{10})",message = "Mobile number must be 10 digits")
    private String mobileNum;

    @NotBlank(message="Email must not be blank")
    @Email(message = "Please provide a valid email address" )
    private String email;

    @NotBlank(message="Subject must not be blank")
    @Size(min=5, message="Subject must be at least 5 characters long")
    private String subject;

    @NotBlank(message="Message must not be blank")
    @Size(min=10, message="Message must be at least 10 characters long")
    private String message;

    private String status;
}
```

Bước 4: Xây dựng lớp ContactRepository để thực hiện các truy vấn vào cơ sở dữ liệu sử dụng spring-jdbc

```
public class ContactRowMapper implements RowMapper<Contact> {

    @Override
    public Contact mapRow(ResultSet rs, int rowNum) throws SQLException {
        Contact contact = new Contact();
        contact.setContactId(rs.getInt("CONTACT_ID"));
        contact.setName(rs.getString("NAME"));
        contact.setMobileNum(rs.getString("MOBILE_NUM"));
        contact.setEmail(rs.getString("EMAIL"));
        contact.setSubject(rs.getString("SUBJECT"));
```

```
        contact.setMessage(rs.getString("MESSAGE"));
        contact.setStatus(rs.getString("STATUS"));
        contact.setCreatedAt(rs.getTimestamp("CREATED_AT").toLocalDateTime());
        contact.setCreatedBy(rs.getString("CREATED_BY"));

        if(null!=rs.getTimestamp("UPDATED_AT")){
            contact.setUpdatedAt(rs.getTimestamp("UPDATED_AT").toLocalDateTime());
        }
        contact.setUpdatedBy(rs.getString("UPDATED_BY"));
        return contact;
    }
}


@Repository
public class ContactRepository {

    private final JdbcTemplate jdbcTemplate;

    @Autowired
    public ContactRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public int saveContactMsg(Contact contact){
        String sql = "insert into contact_msg
                (name,mobile_num,email,subject,message,status," +
                "created_at,created_by) values (?,?,?,?,?,?,?,?)";
        return jdbcTemplate.update(sql,contact.getName(),contact.getMobileNum(),
                contact.getEmail(),contact.getSubject(),contact.getMessage(),
                contact.getStatus(),contact.getCreatedAt(),contact.getCreatedBy());
    }

    public List<Contact> findMsgsWithStatus(String status) {
        String sql = "select * from contact_msg where status = ?";
        return jdbcTemplate.query(sql,new PreparedStatementSetter() {
            public void setValues(PreparedStatement preparedStatement)
                                        throws SQLException {
                preparedStatement.setString(1, status);
            }
        },new ContactRowMapper());
    }

    public int updateMsgStatus(int contactId, String status,String updatedBy) {
        String sql = "update contact_msg set status = ?,
                            updated_by = ?,updated_at =? where contact_id = ?";
        return jdbcTemplate.update(sql,new PreparedStatementSetter() {
            public void setValues(PreparedStatement preparedStatement)
                                    throws SQLException {
                preparedStatement.setString(1, status);
                preparedStatement.setString(2, updatedBy);
                preparedStatement.setTimestamp(3,
                                            Timestamp.valueOf(LocalDateTime.now()));
                preparedStatement.setInt(4, contactId);
            }
        });
    }
}
```

Bước 5: Xây dựng lớp HolidaysRepository để thực hiện các truy vấn vào cơ sở dữ liệu sử dụng spring-jdbc

```
@Repository
public class HolidaysRepository {

    private final JdbcTemplate jdbcTemplate;

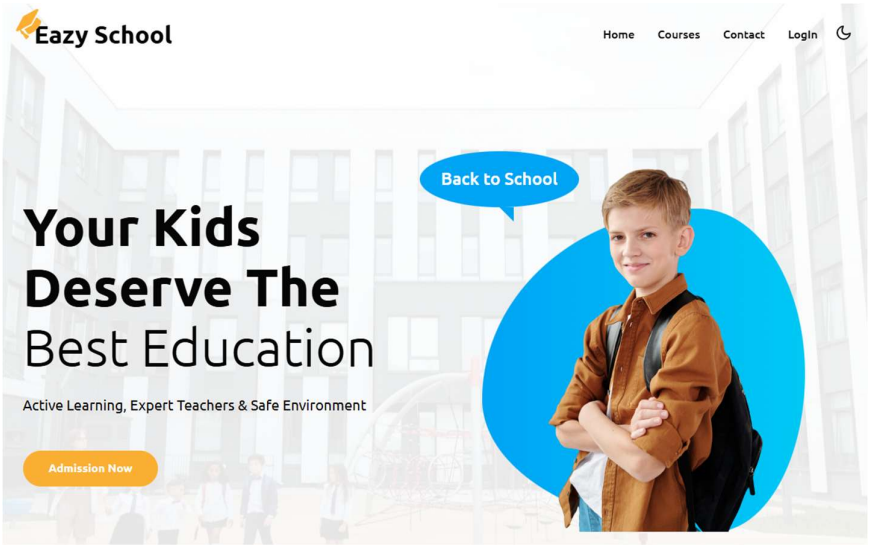
    @Autowired
    public HolidaysRepository(JdbcTemplate jdbcTemplate) {
```

```
        this.jdbcTemplate = jdbcTemplate;
    }

    public List<Holiday> findAllHolidays() {
        String sql = "SELECT * FROM holidays";
        var rowMapper = BeanPropertyRowMapper.newInstance(Holiday.class);
        return jdbcTemplate.query(sql, rowMapper);
    }
}
```

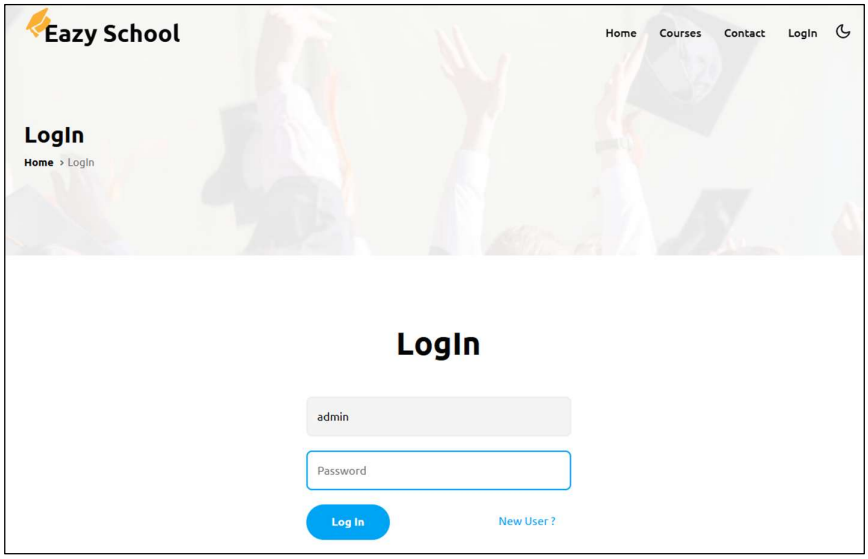
Bước 6: Chạy ứng dụng web này ta được kết quả như sau:

- http://localhost:8080



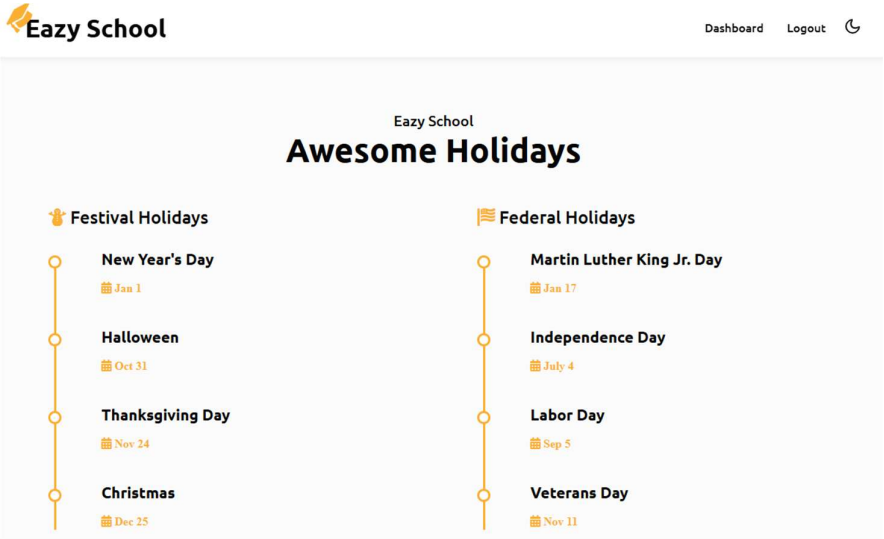
Hình Trang chủ của ứng dụng

- Click vào Login: username: admin; password: 54321



Hình Trang đăng nhập của ứng dụng

- Vào http://localhost:8080/holidays/all



Hình Trang holidays của website

5.1.3. Bài tập làm thêm

Dự án 5.1. Tạo dự án như trong bài viết https://www.geeksforgeeks.org/advance-java/using-lombok-to-reduce-boilerplate-code-in-spring-boot

Dự án 5.2. Nâng cấp các dự án RESTful API đã phát triển bằng cách thêm các chức năng hỗ trợ xử lý bằng Lombok. Một số dự án như

- Dự án các bài báo khoa học
- Dự án các nhân viên
- Dự án các sơ yếu lý lịch

Dự án 5.3. Nâng cấp dự án đồ án cuối kỳ bằng cách tích hợp với Lombok và sử dụng các Annotation của Lombok

5.2. PHẦN 2: XỬ LÝ MAPSTRUCT

5.2.1. Ôn tập lý thuyết với các PROMPT

+ Tại sao chúng ta nên sử dụng MapStruct để chuyển đổi giữa Entity và DTO thay vì viết các phương thức 'convert' thủ công hoặc dùng BeanUtils.copyProperties? Hãy phân tích ưu điểm về mặt hiệu suất (Performance) và tính an toàn lúc biên dịch (Compile-time safety).

+ Hãy hướng dẫn cách tạo một Interface Mapper đơn giản với Annotation @Mapper. Làm thế nào để MapStruct tự động tạo ra mã triển khai (Implementation) và cách chúng ta nhúng (Inject) Mapper này vào Spring Service bằng @Autowired?

+ Trong trường hợp tên thuộc tính của Entity và DTO không giống nhau (ví dụ: userName và name), làm thế nào để sử dụng @Mapping với tham số source và target để kết nối chúng? Hãy cho ví dụ minh họa.

+ Làm thế nào để MapStruct xử lý các đối tượng lồng nhau (Nested Objects) hoặc chuyển đổi danh sách (List<Entity> sang List<DTO>)? Tôi có cần phải viết mã xử lý vòng lặp không hay MapStruct tự động làm điều đó?

+ Nếu tôi muốn thực hiện một số tính toán hoặc định dạng dữ liệu trong quá trình mapping (ví dụ: chuyển kiểu Date sang String theo định dạng cụ thể), tôi có thể sử dụng @Named, expression hoặc các phương thức default trong Interface Mapper như thế nào?

5.2.2. Bài làm mẫu

Bài 1: Ứng dụng MapStruct khi map giữa dto và model

Bước 1: Tạo dự án Spring Boot mới, bổ sung các dependency vào tập tin pom.xml của dự án

```
<dependency>
    <groupId>org.modelmapper</groupId>
    <artifactId>modelmapper</artifactId>
    <version>3.1.0</version>
</dependency>

<dependency>
    <groupId>org.mapstruct</groupId>
    <artifactId>mapstruct</artifactId>
    <version>1.5.3.Final</version>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
          <version>8.0.33</version>
    <scope>runtime</scope>
</dependency>

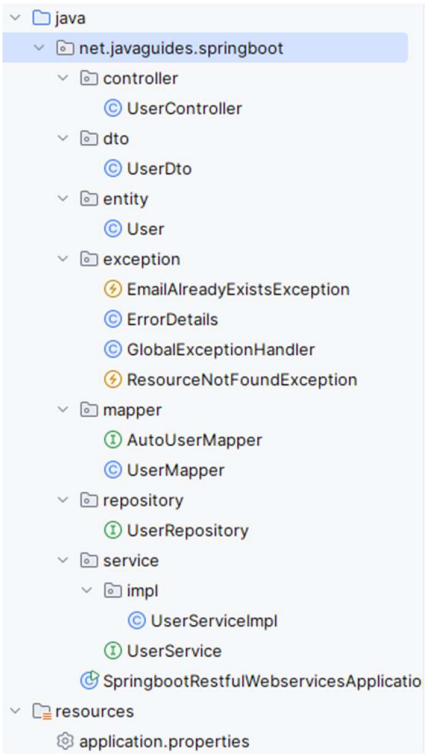
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
          <version>1.18.36</version>
    <optional>true</optional>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.8.17</version>
</dependency>
```

- Dự án có sử dụng JPA (phần này được đề cập chi tiết ở Chương 6)
- Dự án có sử dụng MapStruct
- Dự án có sử dụng Lombok
- Dự án có sử dụng Validation
- Dự án có sử dụng Xử lý ngoại lệ

Bước 2: Tạo các lớp và interface cho dự án như hình bên dưới



Bước 3: Viết mã cho tầng dữ liệu

```
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false)
    private String firstName;
    @Column(nullable = false)
    private String lastName;
    @Column(nullable = false, unique = true)
    private String email;
}

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
}
```

Bước 4: Viết mã cho tầng dịch vụ

```
@Schema(
        description = "UserDto Model Information"
)
@Setter
@Getter
@NoArgsConstructor
@AllArgsConstructor
public class UserDto {

    private Long id;
```

```
    @Schema(
            description = "User First Name"
    )

    @NotEmpty(message = "User first name should not be null or empty")
    private String firstName;

    @Schema(
            description = "User Last Name"
    )

    @NotEmpty(message = "User last name should not be null or empty")
    private String lastName;

    @Schema(
            description = "User Email Address"
    )

    @NotEmpty(message = "User email should not be null or empty")
    @Email(message = "Email address should be valid")
    private String email;
}


public interface UserService {
    UserDto createUser(UserDto user);
    UserDto getUserById(Long userId);
    List<UserDto> getAllUsers();
    UserDto updateUser(UserDto user);
    void deleteUser(Long userId);
}


public interface AutoUserMapper {
    AutoUserMapper MAPPER = Mappers.getMapper(AutoUserMapper.class);
    UserDto mapToUserDto(User user);
    User mapToUser(UserDto userDto);
}

public class UserMapper {

    // Convert User JPA Entity into UserDto
    public static UserDto mapToUserDto(User user){
        UserDto userDto = new UserDto(
                user.getId(),
                user.getFirstName(),
                user.getLastName(),
                user.getEmail()
        );
        return userDto;
    }

    // Convert UserDto into User JPA Entity
    public static User mapToUser(UserDto userDto){
        User user = new User(
                userDto.getId(),
                userDto.getFirstName(),
                userDto.getLastName(),
                userDto.getEmail()
        );
        return user;
    }
}


@Service
@AllArgsConstructor
public class UserServiceImpl implements UserService {

    private UserRepository userRepository;
```

```
    private ModelMapper modelMapper;

    @Override
    public UserDto createUser(UserDto userDto) {

        // Convert UserDto into User JPA Entity
        // User user = UserMapper.mapToUser(userDto);
        // User user = modelMapper.map(userDto, User.class);
        Optional<User> optionalUser =
                    userRepository.findByEmail(userDto.getEmail());

        if(optionalUser.isPresent()){
            throw new EmailAlreadyExistsException("Email Already Exists for User");
        }

        User user = AutoUserMapper.MAPPER.mapToUser(userDto);
        User savedUser = userRepository.save(user);

        // Convert User JPA entity to UserDto
        // UserDto savedUserDto = UserMapper.mapToUserDto(savedUser);
        // UserDto savedUserDto = modelMapper.map(savedUser, UserDto.class);

        UserDto savedUserDto = AutoUserMapper.MAPPER.mapToUserDto(savedUser);

        return savedUserDto;
    }

    @Override
    public UserDto getUserById(Long userId) {
        User user = userRepository.findById(userId).orElseThrow(
                () -> new ResourceNotFoundException("User", "id", userId)
        );
        //return UserMapper.mapToUserDto(user);
        //return modelMapper.map(user, UserDto.class);
        return AutoUserMapper.MAPPER.mapToUserDto(user);
    }

    @Override
    public List<UserDto> getAllUsers() {
        List<User> users = userRepository.findAll();
        return users.stream().map(UserMapper::mapToUserDto)
                .collect(Collectors.toList());
    }

    @Override
    public UserDto updateUser(UserDto user) {

        User existingUser = userRepository.findById(user.getId()).orElseThrow(
                () -> new ResourceNotFoundException("User", "id", user.getId())
        );

        existingUser.setFirstName(user.getFirstName());
        existingUser.setLastName(user.getLastName());
        existingUser.setEmail(user.getEmail());
        User updatedUser = userRepository.save(existingUser);
        //return UserMapper.mapToUserDto(updatedUser);
        //return modelMapper.map(updatedUser, UserDto.class);
        return AutoUserMapper.MAPPER.mapToUserDto(updatedUser);
    }

    @Override
    public void deleteUser(Long userId) {

        User existingUser = userRepository.findById(userId).orElseThrow(
                () -> new ResourceNotFoundException("User", "id", userId)
        );

        userRepository.deleteById(userId);
    }
}
```

Bước 5: Viết mã cho tầng ứng dụng (RESTful API) có phần xử lý Validation

```
@Tag(
        name = "CRUD REST APIs for User Resource",
        description = "CRUD REST APIs - Create User, Update User, Get User, Get All
                       Users, Delete User"
)
@RestController
@AllArgsConstructor
@RequestMapping("api/users")
public class UserController {

    private UserService userService;

    @Operation(
       summary = "Create User REST API",
       description = "Create User REST API is used to save user in a database"
    )
    @ApiResponse(
       responseCode = "201",
       description = "HTTP Status 201 CREATED"
    )
    // build create User REST API
    @PostMapping
    public ResponseEntity<UserDto> createUser(@Valid @RequestBody UserDto user){
        UserDto savedUser = userService.createUser(user);
        return new ResponseEntity<>(savedUser, HttpStatus.CREATED);
    }

    @Operation(
        summary = "Get User By ID REST API",
        description = "Get User By ID REST API is used to get a single user from the
                                database"
    )
    @ApiResponse(
            responseCode = "200",
            description = "HTTP Status 200 SUCCESS"
    )
    // build get user by id REST API
    // http://localhost:8080/api/users/1
    @GetMapping("{id}")
    public ResponseEntity<UserDto> getUserById(@PathVariable("id") Long userId){
        UserDto user = userService.getUserById(userId);
        return new ResponseEntity<>(user, HttpStatus.OK);
    }

    @Operation(
            summary = "Get All Users REST API",
            description = "Get All Users REST API is used to get a all the users
                           from the database"
    )
    @ApiResponse(
            responseCode = "200",
            description = "HTTP Status 200 SUCCESS"
    )
    // Build Get All Users REST API
    // http://localhost:8080/api/users
    @GetMapping
    public ResponseEntity<List<UserDto>> getAllUsers(){
        List<UserDto> users = userService.getAllUsers();
        return new ResponseEntity<>(users, HttpStatus.OK);
    }

    @Operation(
            summary = "Update User REST API",
            description = "Update User REST API is used to update a particular user
                             in the database"
    )
    @ApiResponse(
            responseCode = "200",
            description = "HTTP Status 200 SUCCESS"
```

```
    )
    // Build Update User REST API
    @PutMapping("{id}")
    // http://localhost:8080/api/users/1
    public ResponseEntity<UserDto> updateUser(@PathVariable("id") Long userId,
                                              @RequestBody @Valid UserDto user){
        user.setId(userId);
        UserDto updatedUser = userService.updateUser(user);
        return new ResponseEntity<>(updatedUser, HttpStatus.OK);
    }

    @Operation(
            summary = "Delete User REST API",
            description = "Delete User REST API is used to delete a particular user
                           from the database"
    )
    @ApiResponse(
            responseCode = "200",
            description = "HTTP Status 200 SUCCESS"
    )
    // Build Delete User REST API
    @DeleteMapping("{id}")
    public ResponseEntity<String> deleteUser(@PathVariable("id") Long userId){
        userService.deleteUser(userId);
        return new ResponseEntity<>("User successfully deleted!", HttpStatus.OK);
    }
}
```

Bước 6: Viết mã xử lý ngoại lệ

```
@ResponseStatus(value = HttpStatus.BAD_REQUEST)
public class EmailAlreadyExistsException extends RuntimeException{
    private String message;

    public EmailAlreadyExistsException(String message){
        super(message);
    }
}


@ResponseStatus(value = HttpStatus.NOT_FOUND)
public class ResourceNotFoundException extends RuntimeException{

    private String resourceName;
    private String fieldName;
    private Long fieldValue;

    public ResourceNotFoundException(String resourceName, String fieldName,
                                     Long fieldValue){
        super(String.format("%s not found with %s : '%s'",
                            resourceName, fieldName, fieldValue));
        this.resourceName = resourceName;
        this.fieldName = fieldName;
        this.fieldValue = fieldValue;
    }
}


@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
public class ErrorDetails {
    private LocalDateTime timestamp;
    private String message;
    private String path;
    private String errorCode;
}
```

```
@ControllerAdvice
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorDetails> handleResourceNotFoundException
            (ResourceNotFoundException exception, WebRequest webRequest){
        ErrorDetails errorDetails = new ErrorDetails(
            LocalDateTime.now(),
            exception.getMessage(),
            webRequest.getDescription(false),
            "USER_NOT_FOUND"
        );

        return new ResponseEntity<>(errorDetails, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(EmailAlreadyExistsException.class)
    public ResponseEntity<ErrorDetails> handleEmailAlreadyExistsException
                (EmailAlreadyExistsException exception,
                                                WebRequest webRequest){
        ErrorDetails errorDetails = new ErrorDetails(
            LocalDateTime.now(),
            exception.getMessage(),
            webRequest.getDescription(false),
            "USER_EMAIL_ALREADY_EXISTS"
        );

        return new ResponseEntity<>(errorDetails, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorDetails> handleGlobalException(
                        Exception exception, WebRequest webRequest){
        ErrorDetails errorDetails = new ErrorDetails(
                LocalDateTime.now(),
                exception.getMessage(),
                webRequest.getDescription(false),
                "INTERNAL SERVER ERROR"
        );

        return new ResponseEntity<>(errorDetails, HttpStatus.INTERNAL_SERVER_ERROR);
    }

    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
                    MethodArgumentNotValidException ex,
                    HttpHeaders headers, HttpStatusCode status,WebRequest request) {
        Map<String, String> errors = new HashMap<>();
        List<ObjectError> errorList = ex.getBindingResult().getAllErrors();

        errorList.forEach((error) ->{
            String fieldName = ((FieldError) error).getField();
            String message = error.getDefaultMessage();
            errors.put(fieldName, message);
        });

        return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST);
    }
}
```

Bước 7: Viết mã nguồn cho lớp SpringbootRestfulWebservicesApplication

```
@SpringBootApplication
@OpenAPIDefinition(
       info = @Info(
             title = "Spring Boot REST API Documentation",
             description = "Spring Boot REST API Documentation",
             version = "v1.0",
             contact = @Contact(
```

```
                    name = "Ramesh",
                    email = "javaguides.net@gmail.com",
                    url = "https://www.javaguides.net"
              ),
              license = @License(
                    name = "Apache 2.0",
                    url = "https://www.javaguides.net/license"
              )
        ),
        externalDocs = @ExternalDocumentation(
              description = "Spring Boot User Management Documentation",
              url = "https://www.javaguides.net/user_management.html"
        )
)
public class SpringbootRestfulWebservicesApplication {

    @Bean
    public ModelMapper modelMapper(){
       return new ModelMapper();
    }

    public static void main(String[] args) {
       SpringApplication.run(SpringbootRestfulWebservicesApplication.class,
                 args);
    }

}
```

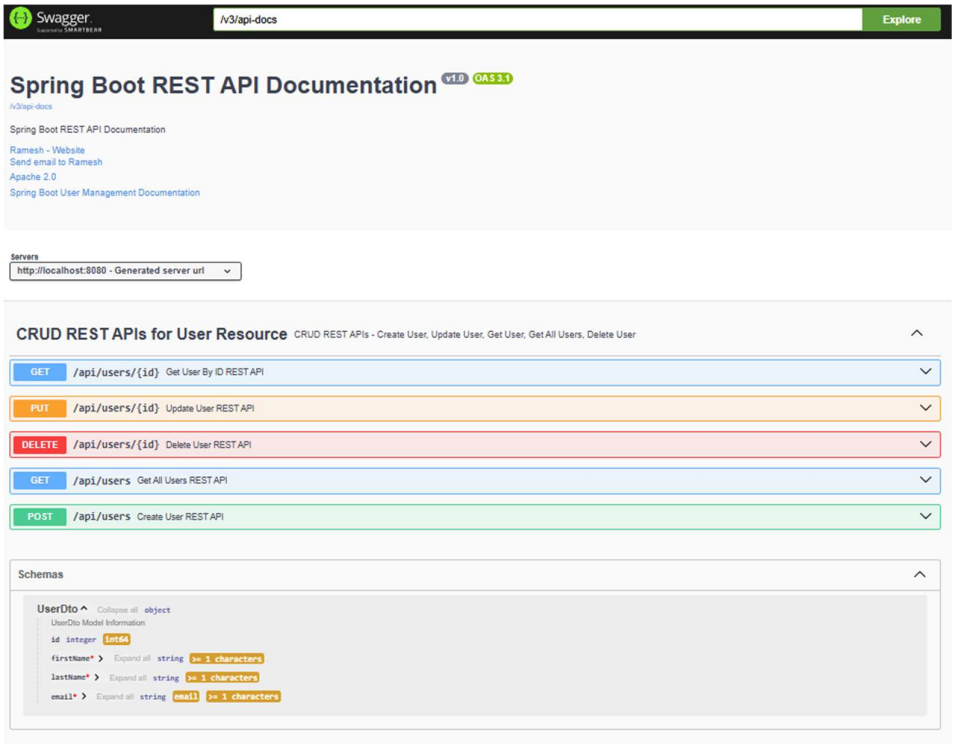
Bước 8: Chạy dự án

- Vào cơ sở dữ liệu user_management để thêm thông tin vào users.

- Duyệt URL: http://localhost:8080/api/users


```
[
  {
    "id": 1,
    "firstName": "Phuoc",
    "lastName": "Nguyen Thanh",
    "email": "phuocnt@gmail.com"
  },
  {
    "id": 2,
    "firstName": "Phuc",
    "lastName": "Nguyen Thanh",
    "email": "phuocnt@sgu.edu.vn"
  }
]
```

- Duyệt URL: http://localhost:8080/swagger-ui/index.html



5.2.3. Bài tập làm thêm

Dự án 5.4. Tạo dự án như trong bài viết https://www.baeldung.com/mapstruct

Dự án 5.5. Tạo dự án như trong bài viết https://www.baeldung.com/java-mapstruct-lombok

Dự án 5.6. Tạo dự án như trong bài viết https://bootify.io/spring-data/mapstruct-with-maven-and-lombok.html

D. TÓM TẮT CHƯƠNG

Chương 5 đã giúp sinh viên thực hiện một bước tiến lớn trong việc "sạch hóa" mã nguồn và tối ưu hóa năng suất lập trình. Bằng cách kết hợp Lombok để loại bỏ những đoạn code lặp lại nhàm chán và MapStruct để tự động hóa quy trình chuyển đổi dữ liệu giữa các lớp, các bạn đã có thể xây dựng những Project có cấu trúc cực kỳ tinh gọn, chuyên nghiệp và dễ bảo trì. Việc làm chủ hai thư viện này không chỉ giúp giảm thiểu sai sót chủ quan mà còn giúp lập trình viên tập trung hoàn toàn vào tư duy nghiệp vụ thay vì các thao tác thủ công. Khép lại phần tối ưu hóa mã nguồn, chúng ta sẽ bước sang Chương 6, một trong những nội dung quan trọng nhất của khóa học là JPA (Java Persistence API) và Hibernate.

CHƯƠNG 6: JPA (JAVA PERSISTENCE API) VÀ HIBERNATE

A. MỤC TIÊU CHƯƠNG

Hỗ trợ việc luyện tập và nghiên cứu nội dung Chương 7 có trong đề cương học phần Chuyên đề J2EE (841468). Chương 6 mở ra cánh cửa kết nối giữa thế giới đối tượng trong Java và hệ quản trị cơ sở dữ liệu quan hệ, giúp ứng dụng của sinh viên có khả năng lưu trữ dữ liệu bền vững. Mục tiêu chính của chương này là giúp người học làm chủ JPA (Java Persistence API) và Hibernate để thực hiện kỹ thuật ánh xạ thực thể (Entity Mapping), biến các lớp Java thành các bảng dữ liệu một cách tự động và nhất quán. Sinh viên sẽ được hướng dẫn cách thao tác với dữ liệu thông qua các cơ chế truy vấn đa dạng, từ việc sử dụng các phương thức có sẵn của Spring Data JPA đến việc viết các câu lệnh JPQL linh hoạt. Việc hiểu rõ vai trò của Hibernate như một "cầu nối" ORM (Object-Relational Mapping) mạnh mẽ sẽ giúp sinh viên tối giản hóa các thao tác SQL thủ công, từ đó tập trung vào logic nghiệp vụ và tối ưu hóa hiệu suất truy xuất dữ liệu cho hệ thống.

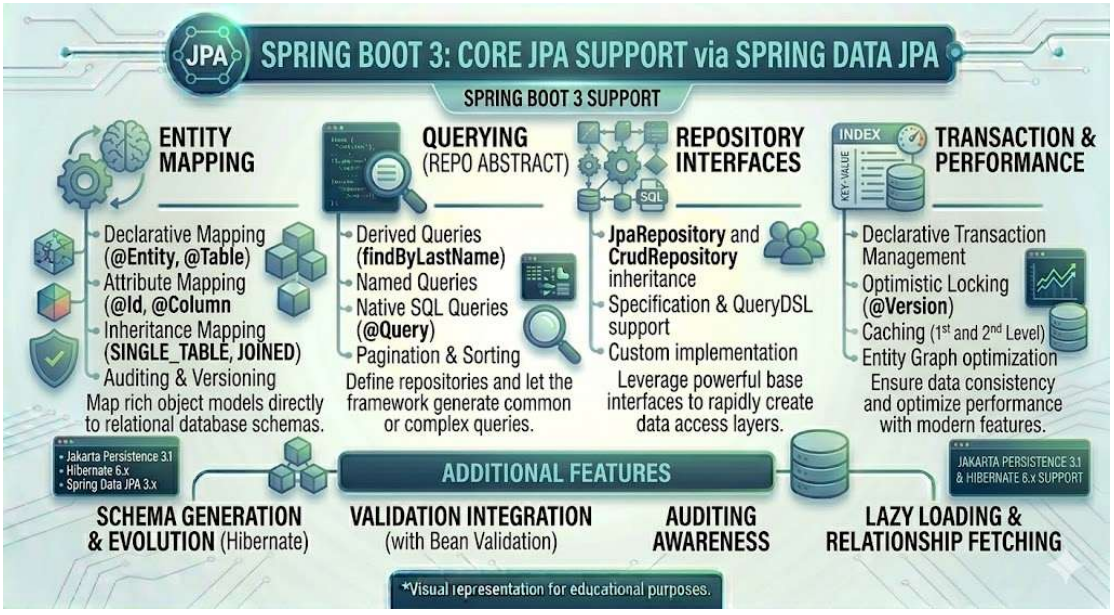
B. KẾT CẤU CHƯƠNG

Chương bao gồm 2 phần

- Phần 1: Java Persistence API
- Phần 2: Hibernate & Spring Data JPA

C. NỘI DUNG CHƯƠNG

6.1. PHẦN 1: JPA (JAVA PERSISTENCE API)



Hình Tổng quan các tính năng được hỗ trợ bởi Spring Data JPA

6.1.1. Ôn tập lý thuyết với các PROMPT

+ JPA là gì và nó khác gì so với Hibernate? Hãy giải thích khái niệm ORM (Object-Relational Mapping) và tại sao nó lại giúp lập trình viên Java tiết kiệm thời gian hơn so với việc viết JDBC thuần túy.

+ Làm thế nào để ánh xạ một Java Class thành một Table trong Database? Hãy giải thích ý nghĩa của các Annotation: @Entity, @Table, @Id, @GeneratedValue, và @Column. Làm thế nào để định nghĩa kiểu dữ liệu và độ dài cho một cột?

+ Spring Data JPA Repository là gì? Hãy giải thích cơ chế Query Methods (truy vấn theo tên phương thức) của nó. Ví dụ: Làm thế nào để tạo một phương thức tìm kiếm người dùng theo email và tên mà không cần viết bất kỳ dòng SQL nào?

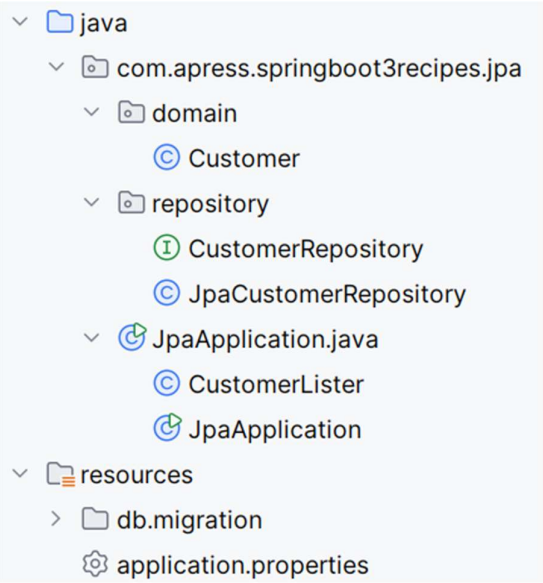
+ Phân biệt sự khác nhau giữa JPQL (Java Persistence Query Language) và Native SQL Query. Khi nào chúng ta nên dùng @Query để viết câu lệnh tùy chỉnh thay vì dùng các phương thức có sẵn của Repository?

+ Làm thế nào để ánh xạ các mối quan hệ giữa các bảng trong JPA (One-to-One, One-to-Many, Many-to-Many)? Hãy giải thích ý nghĩa của tham số mappedBy và tại sao việc hiểu rõ 'ai là chủ sở hữu quan hệ' lại cực kỳ quan trọng để tránh lỗi lặp vô tận?

6.1.2. Bài làm mẫu

Bài 1: Xây dựng dự án sử dụng JPA

Bước 1: Tạo dự án Spring Boot sử dụng Spring Data JPA, và tạo các lớp như hình sau:



Bước 2: Cấu hình cơ sở dữ liệu cho dự án

```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/customers
spring.datasource.username=root
spring.datasource.password=root

spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=create #none | create-update | …
```

Chú ý: cập nhật tập tin pom.xml để tải driver mysql, cài đặt mysql, cài đặt mysql-workbench và tạo cơ sở dữ liệu đặt tên là customers

Bước 3: Viết mã tạo thực thể (entity) theo đặc tả của JPA

```
@Entity
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private final String name;

    @Column(nullable = false)
    private final String email;

    public Customer() {
       this(null,null);
    }

    public Customer(String name, String email) {
       this.name = name;
       this.email = email;
    }

    // Getter, Setter, equals, hashCode
}
```

Bước 4: Viết mã cho Repository sử dụng JPA để thao tác với cơ sở dữ liệu

```
public interface CustomerRepository
{
    List<Customer> findAll();
    Optional<Customer> findById(long id);
    Customer save(Customer customer);
}


@Repository
class JpaCustomerRepository implements CustomerRepository {

    @PersistenceContext
    private EntityManager em;

  @Override
    public List<Customer> findAll() {
       return em.createQuery("SELECT c FROM Customer c",
                             Customer.class).getResultList();
    }

    @Override
    public Optional<Customer> findById(long id) {
       var customer = em.find(Customer.class, id);
       return Optional.ofNullable(customer);
    }

    @Override
    public Customer save(Customer customer) {
       em.persist(customer);
       return customer;
    }
}
```

Bước 5: Viết mã để kiểm tra

```
@SpringBootApplication
public class JpaApplication {

    public static void main(String[] args) {
       SpringApplication.run(JpaApplication.class, args);
    }
}

@Component
class CustomerLister implements ApplicationRunner {

    private final Logger logger = LoggerFactory.getLogger(getClass());
    private final CustomerRepository customers;

    CustomerLister(CustomerRepository customers) {
       this.customers = customers;
    }

    @Override
    public void run(ApplicationArguments args) {
       customers.findAll()
                .forEach( (customer) -> logger.info("{}", customer));
    }
}
```

Bước 6: Chạy dự án nó sẽ in ra màn hình danh sách khách hàng có trong bảng dữ liệu customers. Bạn có thể nhập trực tiếp MySQL Workbench và chạy lại sẽ thấy các khách hàng vừa nhập ở màn hình Console.

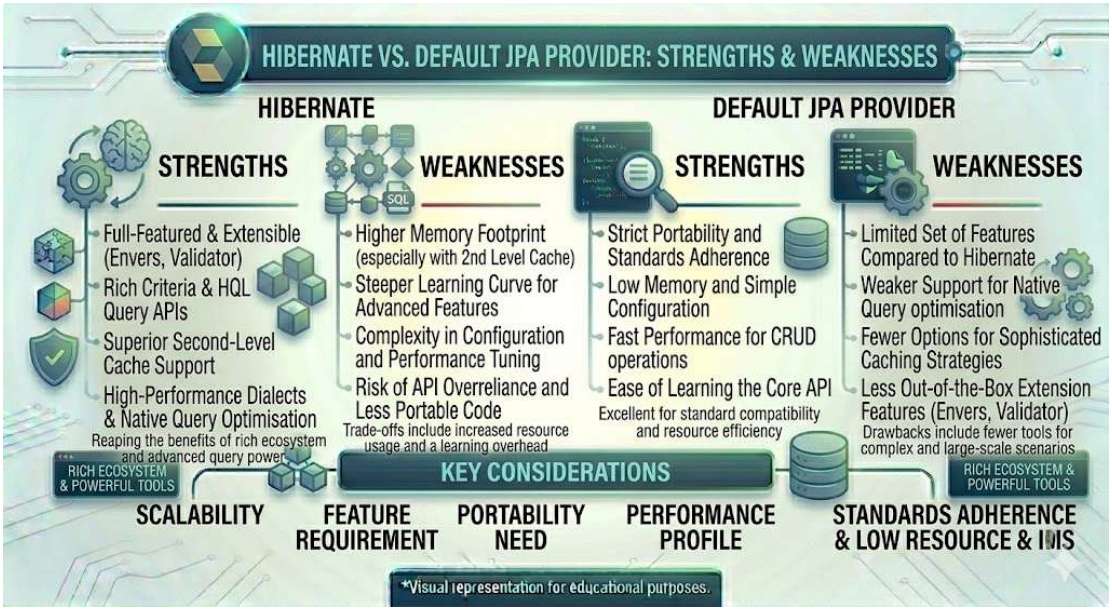
6.1.3. Bài tập làm thêm

Dự án 6.1. Tạo dự án như trong bài viết https://o7planning.org/14311/spring-data-jpa-dto-projections

Dự án 6.2. Nâng cấp các dự án RESTful API đã phát triển bằng cách thêm các chức năng hỗ trợ xử lý phần lưu trữ vĩnh viễn vào MySQL. Một số dự án như

- Dự án các bài báo khoa học
- Dự án các nhân viên
- Dự án các sơ yếu lý lịch

6.2. PHẦN 2: HIBERNATE & SPRING DATA JPA



Hình So sánh giữa Hibernate và cài đặt JPA mặc định

6.2.1. Ôn tập lý thuyết với các PROMPT

+ Hãy giải thích mối quan hệ giữa JPA và Hibernate bằng một ví dụ thực tế (như Bản thiết kế xe hơi và Chiếc xe hơi thực tế). Tại sao chúng ta thường khai báo code bằng các interface của JPA nhưng bên dưới lại để Hibernate xử lý?

+ Hibernate quản lý vòng đời của một Entity như thế nào qua các trạng thái: Transient, Managed, Detached, và Removed? Hãy giải thích cơ chế Dirty Checking giúp Hibernate tự động cập nhật Database mà không cần gọi lệnh update thủ công.

+ Vấn đề N+1 Select trong Hibernate là gì? Tại sao nó lại xảy ra khi chúng ta sử dụng @OneToMany hoặc @ManyToMany? Hãy đề xuất các giải pháp để khắc phục như sử dụng Join Fetch hoặc EntityGraph.

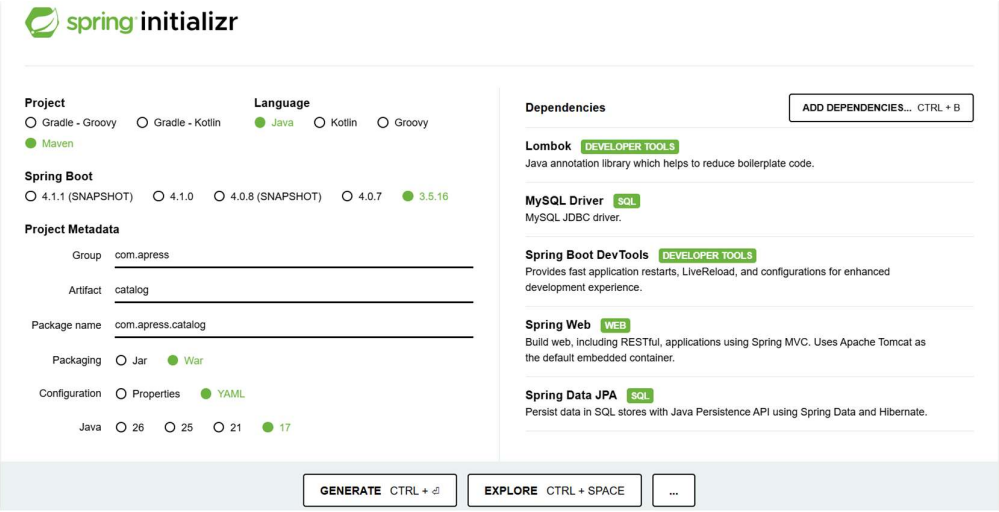
+ So sánh sự khác biệt giữa FetchType.LAZY và FetchType.EAGER. Tại sao trong các dự án thực tế, LAZY luôn được ưu tiên sử dụng? Điều gì xảy ra nếu bạn truy cập vào một Proxy của Hibernate sau khi Session đã đóng (LazyInitializationException)?

+ Tại sao việc sử dụng @Data của Lombok trên các class @Entity của Hibernate lại được coi là một 'bad practice' (thói quen xấu)? Hãy giải thích rủi ro về lỗi vòng lặp vô tận trong hashCode() và equals() khi có quan hệ hai chiều và cách khắc phục bằng @Getter / @Setter.

6.2.2. Bài làm mẫu

Bài 1: Dự án làm quen với một số tính năng cơ bản của Spring Data JPA

Bước 1: Tạo dự án Spring Boot trên Spring initializr (https://start.spring.io/)



Bước 2: Cập nhật tập tin pom.xml để bổ sung một số thư viện không có thể thêm trực tiếp bằng Spring initializr như sau

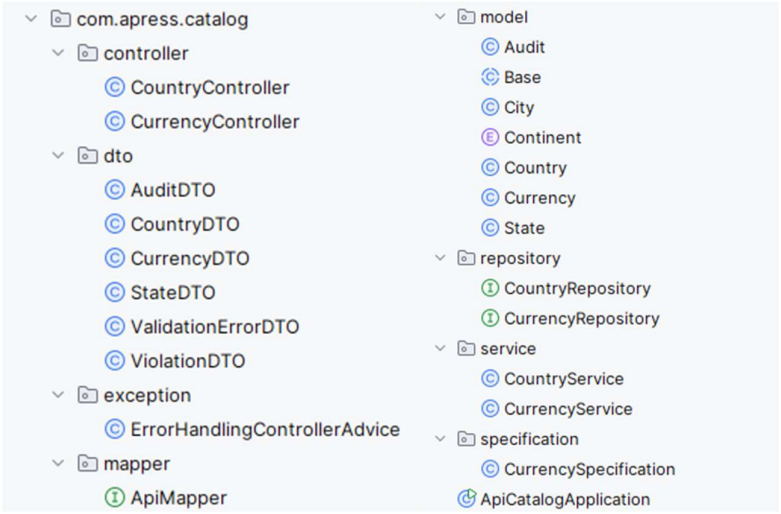
```
<dependency>
    <groupId>org.mapstruct</groupId>
    <artifactId>mapstruct</artifactId>
    <version>1.5.2.Final</version>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<dependency>
    <groupId>jakarta.persistence</groupId>
    <artifactId>jakarta.persistence-api</artifactId>
</dependency>

<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.8.17</version>
</dependency>
```

Bước 3: Tạo gói, lớp và interface như hình bên dưới



Các bước xây dựng tầng dữ liệu (có sử dụng Spring Data JPA)

Bước 3: Chuẩn bị cơ sở dữ liệu có 2 cách

Cách 1: thực thi script được cung cấp

- Mở MySQL Workbench

- Tạo cơ sở dữ liệu là catalog

- Chạy tập tin V1.0__init_database.sql để tạo các bảng vào trong cơ sở dữ liệu

- Cấu hình thêm các tùy biến vào các bảng dữ liệu bằng cách chạy tập tin V2.0__audit_tables.sql

- Thêm dữ liệu ban đầu cho các bảng bằng cách chạy tập tin V1.1__insert_rows.sql

Cách 2: chạy dự án với tùy chọn tạo ra các bảng dữ liệu jpa.hibernate.ddl-auto = none, và sau đó nhập thủ công các dòng dữ liệu ban đầu hoặc chạy 2 tập tin là V2.0__audit_tables.sql và tập tin V1.1__insert_rows.sql

Bước 4: Cấu hình thông tin liên quan đến cơ sở dữ liệu quan hệ bằng cách cập nhật thông tin có trong tập tin application.yml (các dự án trước sử dụng application.properties)

```
…
spring:
  datasource:
    url: "jdbc:mysql://localhost:3306/catalog?autoReconnect=true"
    username: root
    password: root
    driverClassName: com.mysql.cj.jdbc.Driver
    validation-query: select 1;
    maxActive: 100
  jpa:
    show-sql: true
    generate-ddl: false
    hibernate:
      ddl-auto: none
…
```

Bước 5: Viết mã cho các lớp bean dữ liệu hỗ trợ phần thông tin audit trên các thực thể

```
// Giữ thông tin liên quan đến phần audit của các thực thể
@Embeddable
public class Audit implements Serializable {

    @Column(name = "created_on", nullable = false)
    private LocalDateTime createdOn;

    @Column(name = "updated_on", nullable = true)
    private LocalDateTime updatedOn;

    //Getter / Setter /equals /hashCode
}


// Giữ thông tin liên quan đến phần audit của các thực thể
@MappedSuperclass
public abstract class Base implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Embedded
    private Audit audit;

    public Base(){}

    public Base(Long id) {
        this.id = id;
```

```
    }

    @PrePersist // => …
    public void fillCreatedOn() {
        audit.setCreatedOn(LocalDateTime.now());
    }

    @PreUpdate // => …
    public void fillUpdatedOn() {
        audit.setUpdatedOn(LocalDateTime.now());
    }

    //Getter / Setter /equals /hashCode
}
```

Bước 6: Viết mã cho phần bean dữ liệu và cấu hình mapping giữa thực thể với các bảng dữ liệu trong cơ sở dữ liệu (đây là một chức năng của JPA)

```
// Có thể cập nhật để sử dụng Lombok
public class City implements Serializable {
    private Long id;
    private String name;
    private Boolean enabled;
    private State state;

    public City() {}

    public City(Long id, String name, Boolean enabled, State state) {
       this.id = id;
       this.name = name;
       this.enabled = enabled;
       this.state = state;
    }
    //Getter / Setter /equals /hashCode
}

public enum Continent {
    SOUTH_AMERICA,
    NORTH_AMERICA,
    EUROPE,
    ASIA,
    AFRICA,
    ANTARCTIC;
}


@Entity
@Table(name = "currency")
public class Currency extends Base implements Serializable {

    @NotBlank(message = "Code is mandatory")
    @Column(name = "code", nullable = false, length = 4)
    private String code;

    @NotBlank(message = "Symbol is mandatory")
    @Column(name = "symbol", nullable = false, length = 4)
    private String symbol;

    @NotBlank(message = "Description is mandatory")
    @Column(name = "description", nullable = false, length = 30)
    private String description;
    @NotNull(message = "The state of the currency is mandatory")
    @Column(name = "enabled", nullable = false)
    private Boolean enabled = Boolean.TRUE;

    @Min(value = 1, message = "The minimum value is 1")
    @Max(value = 5 , message = "The maximum value is 5")
    @Column(name = "decimal_places")
```

```
    private int decimalPlaces;

    public Currency() {}

    public Currency(Long id, String code, String description, Boolean enabled,
                    int decimalPlaces, String symbol) {
       super(id);
       this.code = code;
       this.description = description;
       this.enabled = enabled;
       this.decimalPlaces = decimalPlaces;
       this.symbol = symbol;
    }

    //Getter /Setter /equals /hashCode
}


@Entity
@Table(name= "state")
public class State implements Serializable {
    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE)
    private Long id;
    @Column(name = "code", nullable = false, length = 4)
    private String code;
    @Column(name = "name", nullable = false, length = 30)
    private String name;
    @Column(name = "enabled", nullable = false)
    private Boolean enabled = Boolean.TRUE;
    @ManyToOne
    @JoinColumn(name = "country_id", insertable = false, updatable = false)
    private Country country;

    public State() {}

    public State(Long id, String code, String name,
                 Boolean enabled, Country country) {
       this.id = id;
       this.code = code;
       this.name = name;
       this.enabled = enabled;
       this.country = country;
    }

    //Getter /Setter /equals /hashCode
}


@Entity
@Table(name= "country")
public class Country implements Serializable {
    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE)
    private Long id;
    @Column(name = "code", nullable = false, length = 4)
    private String code;
    @Column(name = "name", nullable = false, length = 30)
    private String name;
    @Column(name = "locale", nullable = false, length = 6)
    private String locale;
    @Column(name = "time_zone", nullable = false)
    private String timezone;
    @Column(name = "enabled", nullable = false)
    private Boolean enabled = Boolean.TRUE;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "currency_id", nullable = false)
    private Currency currency;
```

```
    @OneToMany(fetch = FetchType.LAZY)
    @JoinColumn(name = "country_id", nullable = false, updatable = false,
                                        insertable = false)
    @OrderBy(value = "code")
    private List<State> states;

    public Country() {}
    public Country(Long id, String code, String name, String locale, String
             timezone, Boolean enabled, Currency currency, List<State> states) {
       this.id = id;
       this.code = code;
       this.name = name;
       this.locale = locale;
       this.timezone = timezone;
       this.enabled = enabled;
       this.currency = currency;
       this.states = states;
    }

    //Getter /Setter /equals /hashCode
}
```

Bước 7: Viết mã thực hiện các thao tác liên quan đến cơ sở dữ liệu sử dụng tính năng của Spring Data JPA. Với Spring Data JPA giúp cho việc viết code JPA dễ dàng hơn và giảm đi rất nhiều mã boilerplate.

```
public interface CountryRepository extends CrudRepository<Country, Long> {
    List<Country> findByCode(String code);
}


public interface CurrencyRepository extends CrudRepository<Currency, Long> {
    // General queries
    List<Currency> findByCode(String code);
    List<Currency> findByCodeAndDescription(String code, String description);

    // Order queries
    List<Currency> findByDescriptionOrderByCodeAsc(String description);
    List<Currency> findByDescriptionOrderByCodeDesc(String description);

    // Manual query
    @Query("SELECT c FROM Currency c where c.code = :code")
    Currency retrieveByCode(@Param("code") String code);
}
```

Các bước xây dựng tầng dịch vụ

Bước 8: Viết mã các lớp trong gói dto, mapper (có sử dụng mapstruct đã được trình bày chi tiết ở Chương 5)

```
public class AuditDTO implements Serializable {

    private LocalDateTime createdOn;

    private LocalDateTime updatedOn;

    //Getter, Setter, equals, hashCode
}

public class CurrencyDTO implements Serializable {
    private Long id;
    private String code;
    private String description;
    private Boolean enable;
    private Integer decimalPlaces;
    private String symbol;
```

```
    private AuditDTO audit;

    public CurrencyDTO() {}

    public CurrencyDTO(Long id, String code, String description,
                       Boolean enable, Integer decimalPlaces) {
        this.id = id;
        this.code = code;
        this.description = description;
        this.enable = enable;
        this.decimalPlaces = decimalPlaces;
    }
    //Getter, Setter, equals, hashCode
}

public class StateDTO implements Serializable {
    private Long id;

    private String code;
    private String name;
    private Boolean enabled;

    private CountryDTO country;

    public StateDTO() {}

    public StateDTO(Long id, String code, String name, Boolean enabled,
                    CountryDTO country) {
       this.id = id;
       this.code = code;
       this.name = name;
       this.enabled = enabled;
       this.country = country;
    }
    //Getter, Setter, equals, hashCode
}

public class CountryDTO implements Serializable {
    private Long id;
    private String code;
    private String name;
    private String locale;
    private String timezone;
    private Boolean enabled;

    private CurrencyDTO currency;

    private List<StateDTO> states;

    public CountryDTO() {}

    public CountryDTO(Long id, String code, String name, String locale,
                  String timezone, Boolean enabled, CurrencyDTO currency) {
       this.id = id;
       this.code = code;
       this.name = name;
       this.locale = locale;
       this.timezone = timezone;
       this.enabled = enabled;
       this.currency = currency;
    }
    //Getter, Setter, equals, hashCode
}

public class ViolationDTO {
    private String field;
    private String message;
    public ViolationDTO(String field, String message) {
        this.field = field;
        this.message = message;
    }
```

```
    //Getter, Setter, equals, hashCode
}

public class ValidationErrorDTO {
    private List<ViolationDTO> violations;

    public ValidationErrorDTO() {
        violations = new ArrayList<>();
    }

    public List<ViolationDTO> getViolations() {
        return violations;
    }

    public void setViolations(List<ViolationDTO> violations) {
        this.violations = violations;
    }
}

@Mapper(componentModel = "spring")
public interface ApiMapper {
    ApiMapper INSTANCE = Mappers.getMapper( ApiMapper.class );

    CurrencyDTO entityToDTO(Currency currency);
    Currency DTOToEntity(CurrencyDTO currency);

    CountryDTO entityToDTO(Country country);
    Country DTOToEntity(CountryDTO country);

    @Mapping(target="country", ignore = true)
    StateDTO stateToStateDTO(State state);

    State stateDTOToState(StateDTO state);
}
```

Bước 9: Viết mã các chức năng ở tầng dịch vụ (có sử dụng tính năng Validate dữ liệu được trình bày tại Chương 3)

```
@Service
public class CurrencyService {

    CurrencyRepository repository;
    Validator validator;

    @Autowired
    public CurrencyService(CurrencyRepository repository, Validator validator) {
       this.repository = repository;
       this.validator = validator;
    }

    public CurrencyDTO getById(Long id) {
       CurrencyDTO response = null;
       Optional<Currency> currency = repository.findById(id);

       if(currency.isPresent()) {
          response = ApiMapper.INSTANCE.entityToDTO(currency.get());
       }

       return response;
    }

    public CurrencyDTO save(CurrencyDTO currency) {
       return saveInformation(currency);
    }

    public CurrencyDTO update(CurrencyDTO currency) {
       return saveInformation(currency);
    }
```

```
    public void delete(Long id) {
       Optional<Currency> currency = repository.findById(id);

       if(currency.isPresent()) {
          currency.get().setEnabled(Boolean.FALSE);
          repository.save(currency.get());
       }
    }

    private CurrencyDTO saveInformation(CurrencyDTO currency) {
       Currency entity = ApiMapper.INSTANCE.DTOToEntity(currency);

       Set<ConstraintViolation<Currency>> violations = validator.validate(entity);
       if(!violations.isEmpty()) {
          throw new ConstraintViolationException(violations);
       }

       Currency savedEntity = repository.save(entity);

       return ApiMapper.INSTANCE.entityToDTO(savedEntity);
    }
}


@Service
public class CountryService {

    CountryRepository repository;
    Validator validator;

    @Autowired
    public CountryService(CountryRepository repository,  Validator validator) {
       this.repository = repository;
       this.validator = validator;
    }

    public CountryDTO getById(Long id) {
       CountryDTO response = null;
       Optional<Country> country = repository.findById(id);

       if(country.isPresent()) {
          response = ApiMapper.INSTANCE.entityToDTO(country.get());
       }

       return response;
    }

    public CountryDTO save(CountryDTO currency) {
       return saveInformation(currency);
    }

    public CountryDTO update(CountryDTO currency) {
       return saveInformation(currency);
    }

    public void delete(Long id) {
       Optional<Country> country = repository.findById(id);

       if(country.isPresent()) {
          country.get().setEnabled(Boolean.FALSE);
          repository.save(country.get());
       }
    }

    private CountryDTO saveInformation(CountryDTO country) {
       Country entity = ApiMapper.INSTANCE.DTOToEntity(country);
       Country savedEntity = repository.save(entity);

       Set<ConstraintViolation<Country>> violations = validator.validate(entity);
       if(!violations.isEmpty()) {
          throw new ConstraintViolationException(violations);
```

```
        }

        return ApiMapper.INSTANCE.entityToDTO(savedEntity);
    }
}
```

Các bước xây dựng tầng ứng dụng (RESTful API)

Bước 10:

```
@RestController
@RequestMapping("/currency")
public class CurrencyController {

    private CurrencyService currencyService;

    @Autowired
    public CurrencyController(CurrencyService currencyService) {
        this.currencyService = currencyService;
    }

    @GetMapping(value = "/{id}")
    public ResponseEntity<CurrencyDTO> getById(@PathVariable Long id) {
        CurrencyDTO response = currencyService.getById(id);
        return new ResponseEntity<>(response, HttpStatus.OK);
    }

    @PostMapping
    public ResponseEntity<CurrencyDTO> save(@RequestBody CurrencyDTO currencyDTO) {
        CurrencyDTO response = currencyService.save(currencyDTO);
        return new ResponseEntity<>(response, HttpStatus.CREATED);
    }

    @PutMapping(value = "/{id}")
    public ResponseEntity<CurrencyDTO> update(@RequestBody CurrencyDTO currencyDTO)
    {
        CurrencyDTO response = currencyService.update(currencyDTO);
        return new ResponseEntity<>(response, HttpStatus.OK);
    }

    @DeleteMapping(value = "/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        currencyService.delete(id);
        return ResponseEntity.ok().build();
    }
}


@RestController
@RequestMapping("/country")
public class CountryController {

    private CountryService countryService;

    @Autowired
    public CountryController(CountryService countryService) {
        this.countryService = countryService;
    }

    @GetMapping(value = "/{id}")
    public ResponseEntity<CountryDTO> getById(@PathVariable Long id) {
        CountryDTO response = countryService.getById(id);
        return new ResponseEntity<>(response, HttpStatus.OK);
    }

    @PostMapping
    public ResponseEntity<CountryDTO> save(@RequestBody CountryDTO countryDTO) {
        CountryDTO response = countryService.save(countryDTO);
        return new ResponseEntity<>(response, HttpStatus.CREATED);
```

```
    }

    @PutMapping(value = "/{id}")
    public ResponseEntity<CountryDTO> update(@RequestBody CountryDTO countryDTO) {
        CountryDTO response = countryService.update(countryDTO);
        return new ResponseEntity<>(response, HttpStatus.OK);
    }

    @DeleteMapping(value = "/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        try {
            //Start the transaction
            //Call the service method
            //Commit the transaction
        } catch(RuntimeException e) {
            //Rollback the transaction
        }

        countryService.delete(id);
        return ResponseEntity.ok().build();
    }
}
```

Các bước xử lý phần ngoại lệ và các phần chung khác

Bước 11:

```
@ControllerAdvice
public class ErrorHandlingControllerAdvice {

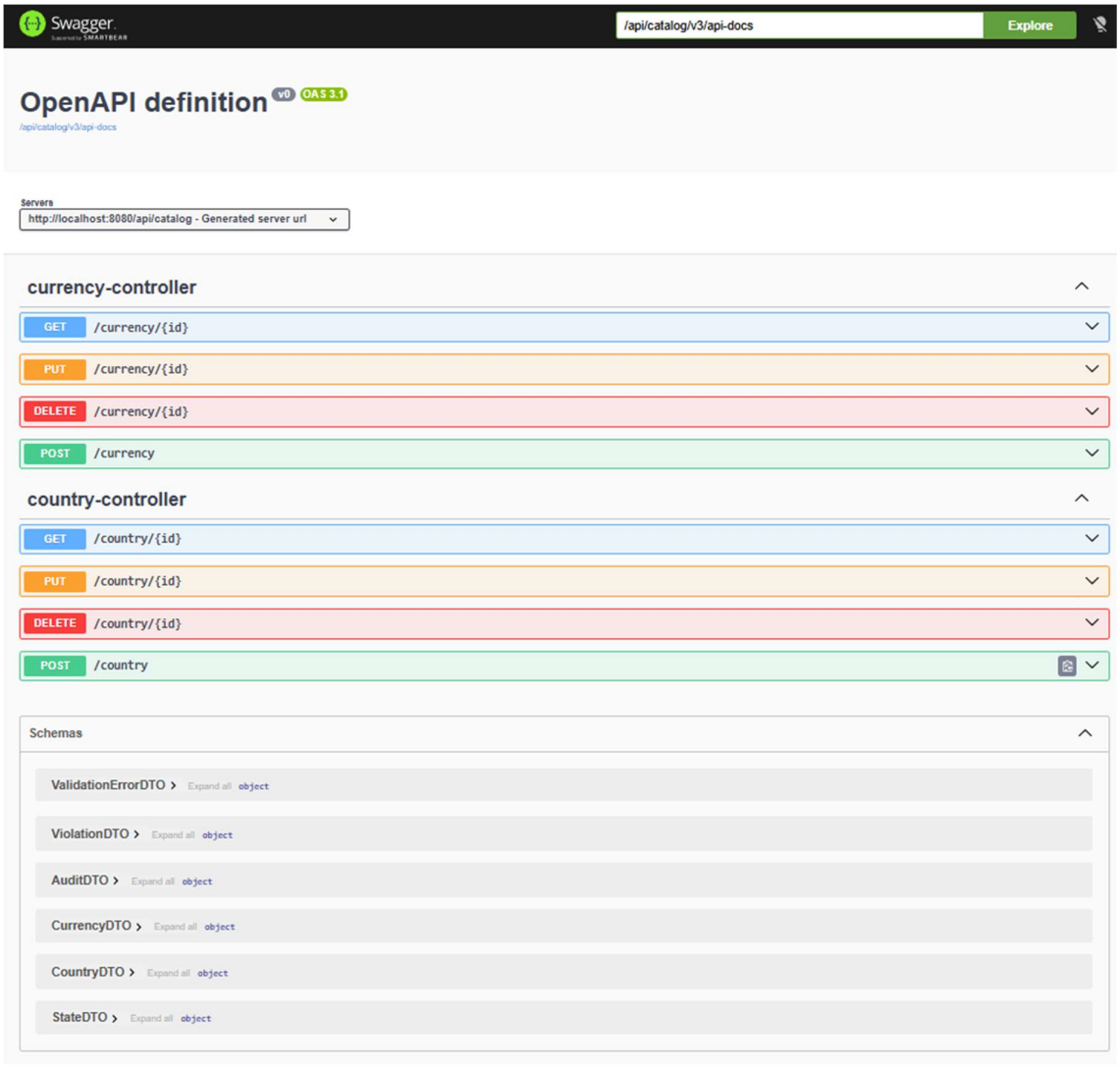
    @ExceptionHandler(ConstraintViolationException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    @ResponseBody
    ValidationErrorDTO onConstraintValidationException(
            ConstraintViolationException e) {
        ValidationErrorDTO error = new ValidationErrorDTO();
        for (ConstraintViolation violation : e.getConstraintViolations()) {
            error.getViolations().add(
                    new ViolationDTO(violation.getPropertyPath().toString(),
                    violation.getMessage())
            );
        }
        return error;
    }
}
```

Kiểm tra đơn vị tự động cho dự án:

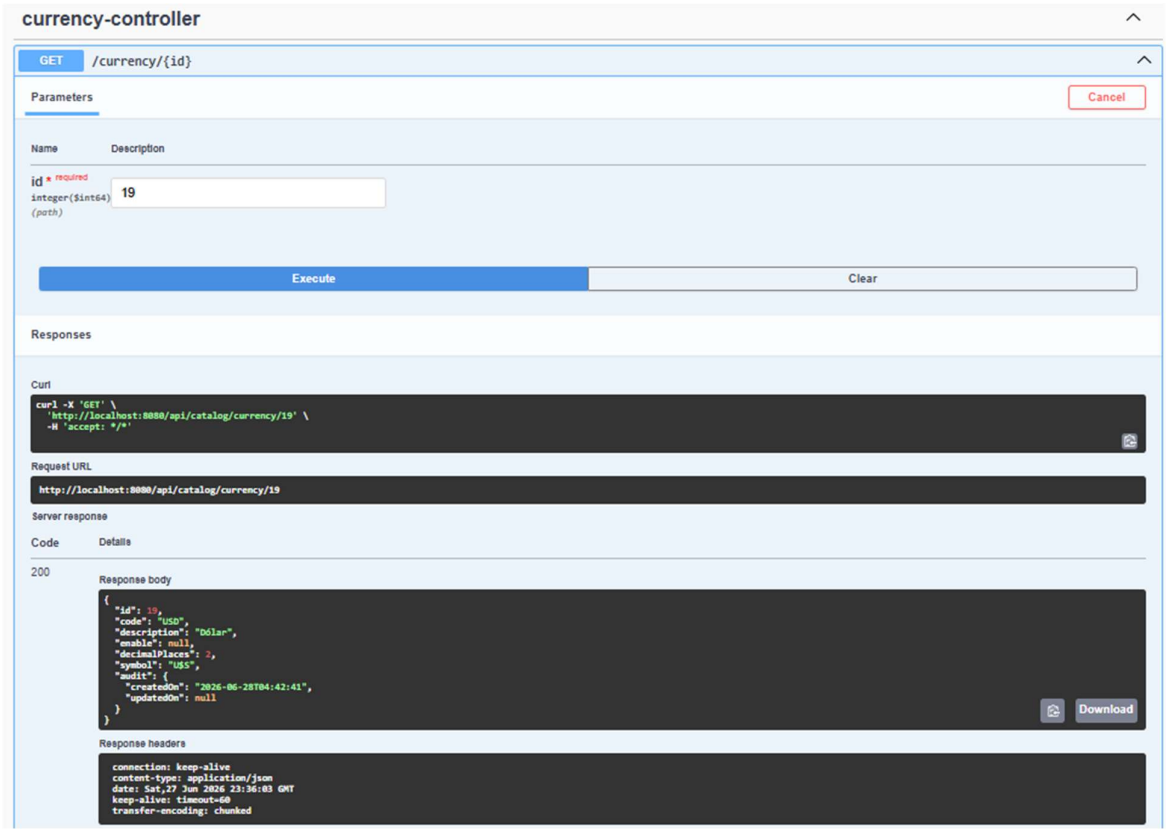
Trong bài này chúng ta chưa thực hiện việc triển khai tự động đơn vị trên dự án này. Các bạn có thể tự triển khai phần này như một bài thực hành.

Kiểm tra hệ thống bằng thủ công cho dự án

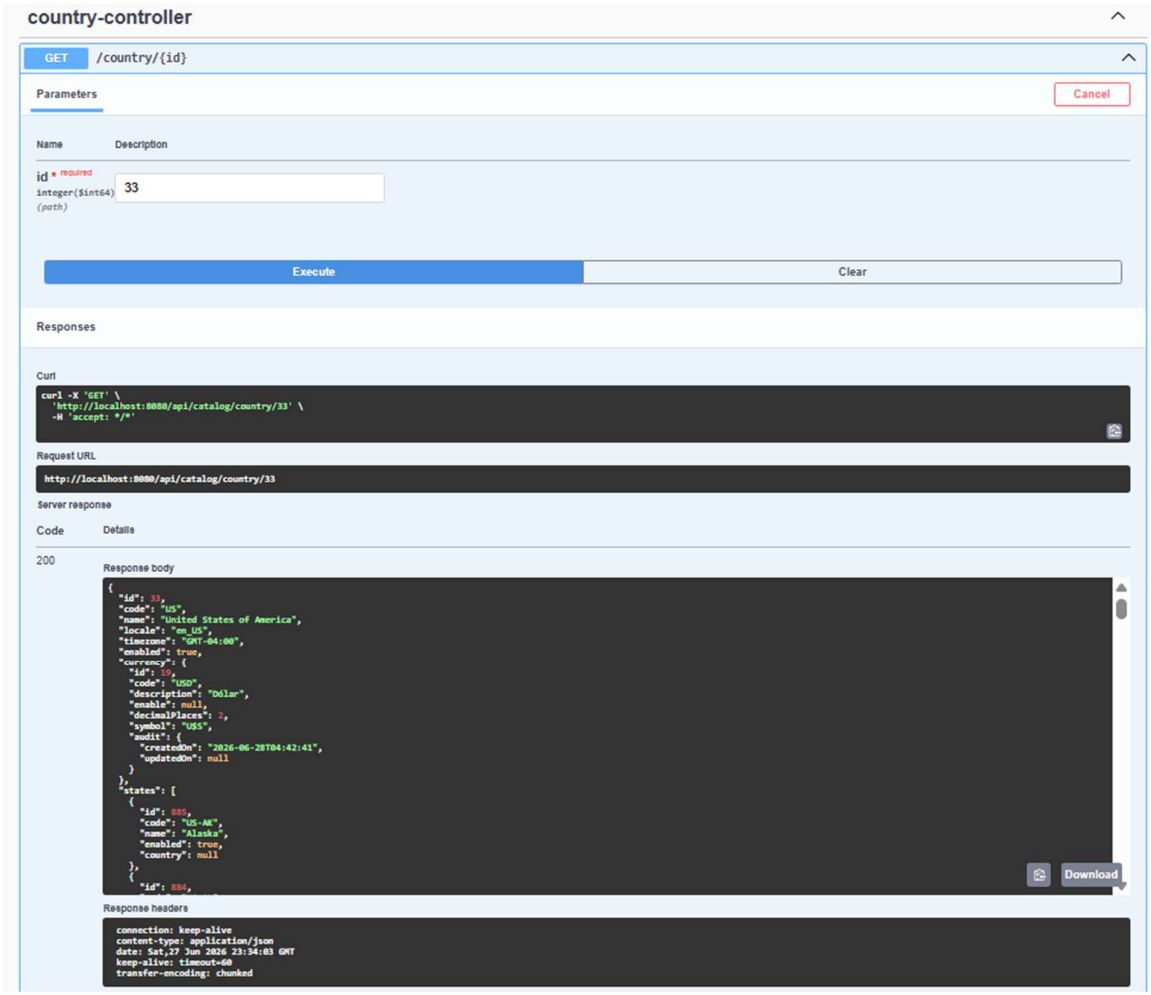
Bước 12: Xem danh sách các RESTful API thông qua Swagger bằng cách Duyệt URL:
http://localhost:8080/api/catalog/swagger-ui/index.html


Swagger
/api/catalog/v3/api-docs
Explore
OpenAPI definition
v0
OAS 3.1
/api/catalog/v3/api-docs
Servers
http://localhost:8080/api/catalog - Generated server url
currency-controller
GET /currency/{id}
PUT /currency/{id}
DELETE /currency/{id}
POST /currency
country-controller
GET /country/{id}
PUT /country/{id}
DELETE /country/{id}
POST /country
Schemas
ValidationErrorDTO
ViolationDTO
AuditDTO
CurrencyDTO
CountryDTO
StateDTO

Bước 13: Kiểm tra các RESTful API liên quan đến thực thể Currency

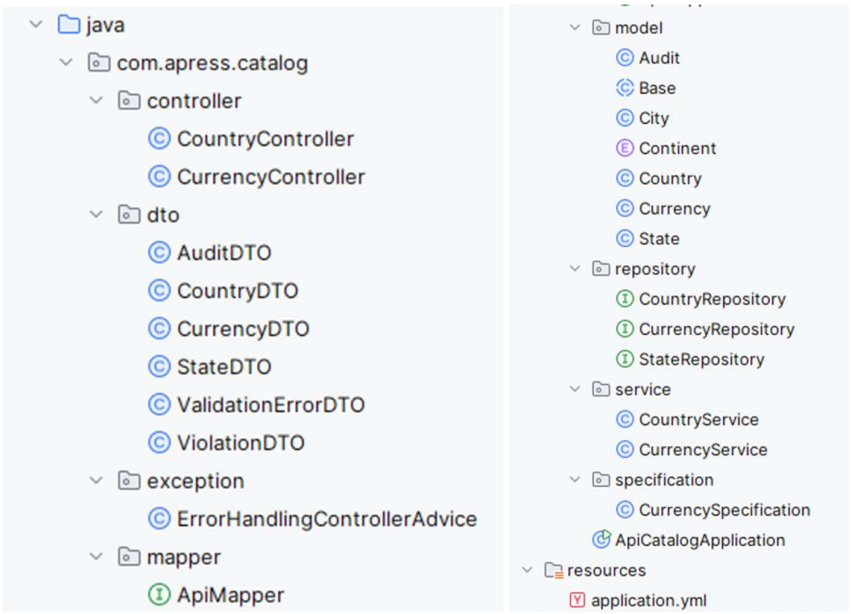


Bước 14: Kiểm tra các RESTful API liên quan đến thực thể Country



Bài 2: Dự án làm quen với xử lý giao tác trong Spring Data JPA

Bước 1: Tải dự án Tải dự án 6_2_2_ex_01_blank.zip. Dự án có cấu trúc như sau:



Bước 2: Phần xử lý các thao tác làm thay đổi dữ liệu thường đặt vào trong một giao tác để đảm bảo thao tác được thực hiện theo nguyên tắc ACID.

Cách 1:

```
@Service
public class CountryService {

    CountryRepository countryRepository;

    StateRepository stateRepository;

    Validator validator;

    @Autowired
    public CountryService(CountryRepository countryRepository,
              StateRepository stateRepository, Validator validator) {
       this.countryRepository = countryRepository;
       this.stateRepository = stateRepository;
       this.validator = validator;
    }

    // Sử dụng Annotation @Transactional
    @Transactional(readOnly = true)
    public CountryDTO getById(Long id) {
       CountryDTO response = null;
       Optional<Country> country = countryRepository.findById(id);

       if(country.isPresent()) {
          response = ApiMapper.INSTANCE.entityToDTO(country.get());
       }

       return response;
    }

    public CountryDTO save(CountryDTO currency) {
       return saveInformation(currency);
    }

    public CountryDTO update(CountryDTO currency) {
       return saveInformation(currency);
    }

    @Transactional(isolation = Isolation.DEFAULT)
    public void delete(Long id) throws InterruptedException {
       Optional<Country> country = countryRepository.findById(id);
       List<State> states =
            stateRepository.findAllByCountryId(country.get().getId());

       if(country.isPresent()) {
          country.get().setEnabled(Boolean.FALSE);
          countryRepository.save(country.get());

          Thread.sleep(2000L);

          for (State state: states) {
             state.setEnabled(Boolean.FALSE);
             stateRepository.save(state);
          }
       }
    }

    private CountryDTO saveInformation(CountryDTO country) {
       Country entity = ApiMapper.INSTANCE.DTOToEntity(country);
       Country savedEntity = countryRepository.save(entity);

       Set<ConstraintViolation<Country>> violations = validator.validate(entity);
       if(!violations.isEmpty()) {
          throw new ConstraintViolationException(violations);
       }
```

```
        return ApiMapper.INSTANCE.entityToDTO(savedEntity);
    }
}
```

Cách 2:

```
@Service
public class CountryService {
    CountryRepository countryRepository;
    StateRepository stateRepository;
    Validator validator;
    @Autowired
    private final TransactionTemplate transactionTemplate;

    @Autowired
    public CountryService(PlatformTransactionManager transactionManager,
                CountryRepository countryRepository,
                StateRepository stateRepository,
                Validator validator) {
        this.countryRepository = countryRepository;
        this.stateRepository = stateRepository;
        this.validator = validator;
        this.transactionTemplate = new TransactionTemplate(transactionManager);
        transactionTemplate.setReadOnly(false);
        transactionTemplate.setTimeout(1000);
    }

    @Transactional(readOnly = true)
    public CountryDTO getById(Long id) {
        CountryDTO response = null;
        Optional<Country> country = countryRepository.findById(id);

        if(country.isPresent()) {
            response = ApiMapper.INSTANCE.entityToDTO(country.get());
        }

        return response;
    }

    public CountryDTO save(CountryDTO currency) {
        return saveInformation(currency);
    }

    public CountryDTO update(CountryDTO currency) {
        return saveInformation(currency);
    }

    private CountryDTO saveInformation(CountryDTO country) {
        Country entity = ApiMapper.INSTANCE.DTOToEntity(country);
        Country savedEntity = countryRepository.save(entity);

        Set<ConstraintViolation<Country>> violations = validator.validate(entity);
        if(!violations.isEmpty()) {
            throw new ConstraintViolationException(violations);
        }

        return ApiMapper.INSTANCE.entityToDTO(savedEntity);
    }

    public void delete(Long id) throws InterruptedException {
        CountryDTO response = null;
        Optional<Country> country = countryRepository.findById(id);
        List<State> states =
              stateRepository.findAllByCountryId(country.get().getId());

        this.transactionTemplate.execute(new TransactionCallbackWithoutResult() {
            public void doInTransactionWithoutResult(TransactionStatus status) {
                try {
```

```
                if(country.isPresent()) {
                    country.get().setEnabled(Boolean.FALSE);
                    countryRepository.save(country.get());

                    Thread.sleep(2000L);

                    for (State state: states) {
                        state.setEnabled(Boolean.FALSE);
                        stateRepository.save(state);
                    }
                }
            } catch(NoSuchElementException | InterruptedException ex) {
                status.setRollbackOnly();
            }
        }
    });
  }
}
```

Bước 3: Kiểm tra các chức năng liên quan đến cập nhật dữ liệu

6.2.3. Bài tập làm thêm

Dự án 6.3. Tạo dự án như trong bài viết https://camunda.com/blog/2025/05/how-to-build-a-rest-api-with-spring-boot-a-step-by-step-guide

Dự án 6.4. Tạo dự án như trong bài viết https://o7planning.org/11661/spring-boot-jpa-and-spring-transaction

Dự án 6.5. Dựng, tìm hiểu và trình bày các công nghệ được sử dụng trong *https://github.com/spring-projects/spring-petclinic*. Dự án này có thể được xem là dự án điển hình sử dụng hệ sinh thái Spring để phát triển hệ thống web. Dựa vào cách tổ chức có trong dự án này các bạn có thể sử dụng nó để phát triển dự án đồ án cuối kỳ

Dự án 6.6. Cập nhật dự án đồ án cuối kỳ có các chức năng

- Lưu trữ vĩnh viễn dữ liệu vào MySQL
- Cài đặt các tính năng của Repository sử dụng Spring Data JPA
- Xử lý phần giao tác cho các chức năng liên quan đến phần cập nhật cơ sở dữ liệu

D. TÓM TẮT CHƯƠNG

Chương 6 đã đánh dấu một bước ngoặt quan trọng khi đưa ứng dụng của sinh viên từ việc xử lý dữ liệu tạm thời sang khả năng lưu trữ bền vững và chuyên nghiệp. Thông qua việc tìm hiểu về JPA và Hibernate, người học không chỉ nắm vững kỹ thuật ánh xạ đối tượng (ORM) để thu hẹp khoảng cách giữa Java và SQL, mà còn làm chủ được các phương thức truy vấn mạnh mẽ từ Spring Data JPA. Sinh viên đã hiểu rằng Hibernate không chỉ là một công cụ tự động sinh mã, mà là một bộ máy quản lý dữ liệu tinh vi, đòi hỏi sự hiểu biết sâu sắc về vòng đời của thực thể và tối ưu hóa hiệu suất truy vấn.

Tài liệu tham khảo

[1] https://www.geeksforgeeks.org/advance-java/spring; 2026

[2] https://www.javaguides.net/p/spring-core-tutorial.html; 2026

[3] Modern API Development with Spring 6 and Spring Boot 3 - Sourabh Sharma - <Packt>BIRMINGHAM—MUMBAI – 2023 – ISBN 978-1-80461-327-6

[4] Spring Boot 3 Recipes - Marten Deinum – Apress – 2024 – ISBN-13 (pbk): 979-8-8688-0112-9

[5] Spring Boot 3.0 Cookbook - Felip Miguel Puig – Packt – 2024

[6] https://eazybytes.com; 2026

[7] Beginning Spring Data - Andres Sacco – Apress – 2023 - ISBN-13 (pbk): 978-1-4842-8763-7